



POLITECNICO DI BARI

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA ELETTRONICA E
DELLE TECNOLOGIE INTERNET

TESI DI LAUREA IN
FONDAMENTI DI ELETTRONICA

PROGETTAZIONE, IMPLEMENTAZIONE E TEST DI UN
SOFT-PROCESSOR RISC-V SU FPGA

RELATORE:
PROF. AGOSTINO GIORGIO

LAUREANDO:
ADRIANO AMOROSINI

ANNO ACCADEMICO 2025-2026

Ai miei genitori

Abstract

Progettazione, in linguaggio VHDL, di una CPU *pipelined* RISC-V a 32 bit capace di eseguire tutte le istruzioni dell'*instruction set* RV32I, corredata da diverse test-bench con cui sarà possibile valutarne e studiarne il funzionamento. Il progetto viene poi sintetizzato e implementato su una scheda FPGA utilizzando l'ambiente *Quartus*, eseguendo il debug tramite strumenti avanzati come *Timequest Timing Analyzer* e *Signal Tap Logic Analyzer*. Viene poi effettuato un successivo confronto delle prestazioni della CPU realizzata con quelle del processore *Nios V*, *soft-core* sviluppato da Intel per dispositivi FPGA.

INDICE

ELENCO DELLE TABELLE

ELENCO DELLE FIGURE 1

INTRODUZIONE 5

1 AMBIENTE DI SVILUPPO QUARTUS: INTRODUZIONE E STRUMENTI PRINCIPALI 7

1.1	Introduzione	7
1.2	Creazione di un progetto e step iniziali	8
1.2.1	Dalla creazione del progetto alla sintesi del circuito	8
1.2.2	Design della rete	14
1.3	Strumenti per la simulazione	14
1.3.1	Utilizzo e panoramica del Waveform Editor	14
1.3.2	Esempio di applicazione di Questa Starter Edition	16
1.4	Analisi Temporale in ambiente Quartus	18
1.4.1	Cenni teorici	18
1.4.2	Esempio di utilizzo del <i>Timing Analyzer</i>	22
1.5	Analisi di potenza in ambiente Quartus	26
1.5.1	Cenni Teorici	26
1.5.2	Esempio di utilizzo del <i>Power Analyzer Tool</i>	27
1.6	Utilizzo della FPGA	28
1.6.1	Assegnamento dei pin	28
1.6.2	Programmazione della scheda	30
1.6.3	Debug e verifica con <i>Signal Tap Logic Analyzer</i>	31

2 IL LINGUAGGIO VHDL 36

2.1	Introduzione	36
2.2	Creare un componente	36
2.2.1	Generalità	36
2.2.2	Entity	37
2.2.3	Architecture	38
2.3	Tipi base del linguaggio VHDL	39
2.3.1	Il tipo <code>bit</code>	39
2.3.2	Tipi IEEE	39

2.3.3	Package <code>numeric_std</code> [1]	40
2.3.4	Il tipo <code>integer</code>	41
2.3.5	Definizione di nuovi tipi	41
2.4	Manipolazione dei segnali	42
2.4.1	Operatori di base	42
2.4.2	Slice e concatenamento	42
2.4.3	Assegnamento condizionale	43
2.5	VHDL per reti sequenziali: i processi	44
2.5.1	Definizione di un processo	44
2.5.2	Statement <code>if</code>	45
2.5.3	Statement <code>case</code>	46
2.5.4	Statement <code>for</code>	47
2.6	Descrizione strutturale	48
2.7	Package	49
3	DESIGN DEL SOFT-CORE RISC-V	50
3.1	Architettura RISC-V	50
3.1.1	Introduzione	50
3.1.2	Assembly e linguaggio macchina	50
3.1.3	Registri, memorie ed immediati	51
3.1.4	Tipologie di istruzioni	53
3.2	Microarchitettura del soft-core	55
3.2.1	Introduzione	55
3.2.2	Componenti di base	56
3.2.3	Pipeline e gestione degli hazard	63
3.2.4	Architettura strutturale della CPU in VHDL	68
3.3	Implementazione del soft-processor su FPGA	75
3.3.1	Inizializzazione dei segnali	75
3.3.2	Gestione delle memorie	75
3.3.3	Implementazione della CPU	76
4	TEST E VERIFICA DEL SOFT-PROCESSOR	77
4.1	Verifica del funzionamento della CPU	77
4.1.1	Introduzione	77
4.1.2	Simulazione funzionale	77
4.1.3	Analisi temporale	81
4.1.4	Analisi di potenza	83
4.1.5	Programmazione della FPGA	84
4.2	Utilizzo del soft-core <i>Nios V</i>	89
4.2.1	Introduzione	89
4.2.2	Inclusione del soft-processor all'interno di un progetto Quartus	90
4.2.3	Generazione di un programma eseguibile dal soft-processor	92
4.2.4	Integrazione del soft-core nel progetto	95
4.2.5	Analisi temporale e di potenza	96
4.2.6	Verifica del funzionamento tramite <i>Signal Tap</i>	98

CONCLUSIONI	100
BIBLIOGRAFIA E SITOGRAFIA	101
RINGRAZIAMENTI	102

ELENCO DELLE TABELLE

2.1	Valori logici del tipo <code>std_logic</code>	40
3.1	Registri dell'architettura RISC-V e loro utilizzo [2]	51
3.2	Istruzioni del sub-set RV32I [2]	55
3.3	Legenda delle sigle utilizzate nelle istruzioni descritte [2]	56
4.1	Elenco dei registri di controllo e relativi indirizzi CSR.	90
4.2	Istruzioni della <i>M-extension</i> [3].	90
4.3	Istruzioni di controllo del set <i>Zicsr</i> [3].	91
4.4	Istruzioni miscellanee in Nios V [3].	91

ELENCO DELLE FIGURE

1.1	Interfaccia principale dell'ambiente di sviluppo Quartus al suo primo avvio [3].	9
1.2	Menù a tendina dal quale sarà possibile sia creare o aprire nuovi file e progetti, sia uscire dall'ambiente di lavoro.	10
1.3	Selezione del nome e filepath per la directory contenente il progetto	11
1.4	Selezione del tipo di progetto.	11
1.5	Selezione di eventuali file esistenti da includere nel progetto.	12
1.6	Selezione del dispositivo FPGA di progetto.	12
1.7	Selezione delle impostazioni di progetto riguardo HDL ed software di terze parti utilizzati.	13
1.8	Interfaccia principale dopo la creazione di un progetto.	13
1.9	Circuito del codificatore di Hamming sintetizzato.	14
1.10	Waveform Editor alla sua prima apertura.	15
1.11	Finestra del <i>Node Finder</i>	15
1.12	Risultato della simulazione funzionale del codificatore.	16
1.13	Selezione della testbench nel profilo di simulazione.	17
1.14	Risultato della simulazione RTL, in Questa, a partire dalla testbench scritta.	17
1.15	Risultato della simulazione RTL con la presenza di alcuni segnali interni opportunamente aggiunti.	18
1.16	Illustrazione grafica del calcolo del <i>setup slack</i> [4]	21
1.17	Illustrazione grafica del calcolo dell' <i>hold slack</i> [4]	21
1.18	Circuito sintetizzato per illustrare il funzionamento del timing analyzer.	22
1.19	Voci presenti nel report dell'analisi temporale.	23
1.20	Diagramma temporale ottenuto con frequenza di clock pari a 1000 MHz	23
1.21	Finestra per la creazione di un nuovo clock.	24
1.22	Finestra per associare il nuovo clock al suo corrispettivo nel design.	24
1.23	Diagramma temporale ottenuto con frequenza di clock pari a 250 MHz	25
1.24	Finestra utile ad impostare il file *.sdc creato nel Timing Analyzer come riferimento per la compilazione.	25
1.25	Andamento della corrente in una FPGA durante le diverse fasi operative [5]	26
1.26	Finestra utile ad impostare le condizioni di esecuzione dell'analisi temporale.	28

1.27	<i>Power Analyzer Summary</i> ottenuto dopo la compilazione del circuito d'esempio.	28
1.28	Layout della <i>DE1-SoC</i> [6].	29
1.29	<i>Assignment Editor</i> alla sua prima apertura [3].	30
1.30	<i>Assignment Editor</i> dopo aver associato i pin fisici ai nodi.	30
1.31	<i>Programmer</i> dopo aver impostato correttamente la sua finestra.	31
1.32	Esempio di utilizzo della scheda: l'input degli switch è pari a 0001, mentre l'output dei LED è, come prevedibile, 1101001.	32
1.33	Esempio di utilizzo della scheda: l'input degli switch è pari a 0110, mentre l'output dei LED è, come prevedibile, 1100110.	33
1.34	Finestra di <i>Signal Tap</i> alla sua prima apertura.	33
1.35	Finestra di <i>Signal Tap</i> dopo la prima modifica effettuata.	34
1.36	Forme d'onda catturate con il primo set di impostazioni.	34
1.37	Finestra di <i>Signal Tap</i> con la condizione di trigger di tipo sequenziale.	34
1.38	Forme d'onda catturate con il secondo set di impostazioni. Il momentaneo ritorno a 0 è causato dal fenomeno di <i>debounce</i> degli switch	35
1.39	Finestra per la personalizzazione della condizione di trigger.	35
1.40	Forme d'onda catturate con il terzo set di impostazioni.	35
3.1	Formato delle istruzioni di tipo R [2]	53
3.2	Formato delle istruzioni di tipo I [2]	53
3.3	Formato delle istruzioni di tipo S/B [2]	54
3.4	Formato delle istruzioni di tipo U/J [2]	54
3.5	Microarchitettura del soft-processor. In nero le componenti; in giallo gli ingressi; in verde i segnali del datapath; in rosso i segnali di controllo; in blu i segnali per la gestione degli hazard.	57
4.1	Primo programma per il test della CPU [2].	78
4.2	Waveform relative al test del primo programma.	81
4.3	Waveform relative al test del secondo programma.	81
4.4	Waveform relative al test del terzo programma.	81
4.5	Waveform relative al test del quarto programma.	82
4.6	Waveform relative al test del quinto programma.	82
4.7	Contenuto del Register File al concludersi della simulazione.	82
4.8	Contenuto della Data Memory al concludersi della simulazione.	83
4.9	Impostazioni del <i>Power Analyzer</i> per la CPU.	84
4.10	Report dell'analisi di potenza della CPU.	84
4.11	FPGA con display a 7 segmenti inizialmente spenti.	85
4.12	Valore scritto durante l'esecuzione del primo programma, visibile sui display a 7 segmenti.	85
4.13	Stato di reset della CPU dopo la pressione del bottone <code>ctrl</code> , con valore nullo sui display.	86
4.14	Output del secondo programma visualizzato sui display a 7 segmenti.	87
4.15	Output del terzo programma mostrato sui display a 7 segmenti.	87

4.16	Output del quarto programma visualizzato sui display a 7 segmenti.	88
4.17	Output del quinto programma visualizzato sui display a 7 segmenti.	88
4.18	Analisi di alcuni dei segnali reali della CPU attraverso <i>Signal Tap</i> .	89
4.19	Finestra principale del <i>Platform Designer</i> .	91
4.20	Finestra principale del <i>Platform Designer</i> dopo aver effettuato i procedimenti necessari.	92
4.21	Finestra della <i>Nios V Command Shell</i> .	93
4.22	Finestra iniziale del <i>Riscfree IDE</i> relativa alla selezione del <i>work- space</i> .	95
4.23	Schermata principale del <i>Riscfree IDE</i> .	95
4.24	Impostazioni relative all'inizializzazione della memoria del Nios V.	96
4.25	Implementazione fisica del progetto con Nios V con segnale di reset basso.	97
4.26	Implementazione fisica del progetto con Nios V con segnale di reset alto.	98
4.27	Analisi di potenza del progetto con Nios V.	98
4.28	Elenco dei segnali d'interesse per <i>Signal Tap</i> nel progetto con Nios V.	99
4.29	Condizione di trigger per <i>Signal Tap</i> nel progetto con Nios V.	99
4.30	Waveform rilevate con <i>Signal Tap</i> nel progetto con Nios V.	99

Introduzione

Le architetture open-source, negli ultimi anni, hanno acquisito un ruolo sempre più rilevante nel mondo dell'elettronica digitale. Una delle più rilevanti è, indubbiamente, l'architettura RISC-V, divenuta celebre grazie alla sua modularità e flessibilità, che la rendono l'ideale per numerosi progetti ed attività didattiche e professionali. Parallelamente, le FPGA (Field Programmable Gate Array) sono dei dispositivi logici programmabili che rappresentano, ormai da lungo tempo, la piattaforma principale per la prototipazione rapida e la sperimentazione di sistemi digitali complessi, permettendo di verificare in hardware il funzionamento di numerosi progetti senza dover ricorrere alla costruzione del circuito.

Il presente lavoro di tesi si propone di progettare, implementare e testare un soft-processor basato sull'architettura RISC-V, utilizzando il linguaggio VHDL per la descrizione hardware e l'ambiente di sviluppo Quartus di Intel/Altera per la simulazione, il debug e la sintesi su FPGA. L'obiettivo principale è realizzare un core funzionante, capace di eseguire istruzioni di base e di essere verificato con l'ausilio di testbench opportunamente scritte e la board FPGA *DE1-SoC*, consentendo così di comprendere l'intero processo di progettazione e test di un processore.

La tesi è articolata nei seguenti capitoli:

- **Capitolo 1** – Fornisce una panoramica sull'ambiente di sviluppo Quartus, illustrando i principali strumenti disponibili per la simulazione (come il waveform editor e Questa Starter Edition), il debug (Timing Analyzer, Signal Tap Logic Analyzer) e l'analisi delle prestazioni (Power Analyzer), spiegandone il loro utilizzo ed il loro funzionamento.
- **Capitolo 2** – Introduce il linguaggio VHDL, fornendo una panoramica di base esaustiva del linguaggio di progettazione. Vengono descritti i concetti fondamentali, le strutture di controllo disponibili, i tipi e la gestione dei segnali e l'utilizzo di questi strumenti per la prototipazione di reti logiche combinatorie e sequenziali.
- **Capitolo 3** – Presenta il progetto del core RISC-V e la sua implementazione in VHDL. Si descrivono le scelte architetturali, l'organizzazione e il funzionamento delle unità funzionali e l'interazione tra i vari componenti del processore nel datapath, volta a garantire la corretta esecuzione dell'intero Instruction Set RV32I.

- **Capitolo 4** – Illustra l’implementazione del core su FPGA, la simulazione tramite il tool Questa, e la verifica funzionale in hardware. Vengono approfonditi gli aspetti di debug, le misure di prestazione e le problematiche riscontrate durante la realizzazione del prototipo sulla board DE1-SoC. Successivamente, vengono confrontati i risultati ottenuti con quelli del NIOS V, il soft processor basato su RISC-V implementato dalla Intel.

Infine, le Conclusioni riassumono i risultati ottenuti, evidenziando le potenzialità e i limiti del progetto, e suggeriscono possibili sviluppi futuri per ulteriori ottimizzazioni o implementazioni di estensioni del processore.

Capitolo 1

AMBIENTE DI SVILUPPO QUARTUS: INTRODUZIONE E STRUMENTI PRINCIPALI

1.1 Introduzione

Quartus è l'ambiente di sviluppo integrato (IDE) prodotto da Intel/Altera, ed offre ad i suoi utenti un insieme completo di strumenti per progettare, simulare e implementare sistemi digitali su FPGA.

L'IDE qui discusso mette a disposizione numerosi tool che andremo successivamente a discutere nel dettaglio, come¹:

- Editor: consente di descrivere il progetto digitale tramite diagrammi schematici o codice HDL. Nel progetto in esame si farà ricorso al linguaggio VHDL.
- Compilatore: trasforma il progetto in una rappresentazione compatibile con il dispositivo FPGA utilizzato.
- Programmer: consente di caricare il file di programmazione sulla FPGA.
- Waveform Editor: permette di creare e analizzare forme d'onda, in modo da verificare il corretto funzionamento logico del sistema progettato.
- RTL Viewer: consente di visualizzare la struttura logica del progetto dopo la fase di sintesi del codice, mostrando i blocchi funzionali e le connessioni tra i vari componenti.
- Questa (Starter Edition): strumento di simulazione avanzata che permette di verificare il comportamento del sistema, utilizzando eventuali testbench scritte ed inserite nel progetto.

¹Elenco tratto parzialmente da [7]

- Timing Analyzer: analizza i ritardi e le temporizzazioni del progetto, permettendo di verificare che tutte le operazioni logiche rispettino i vincoli temporali previsti.
- Signal Tap Logic Analyzer: consente di catturare e visualizzare segnali interni della FPGA in tempo reale, permettendo il debug direttamente sull'hardware.
- Power Analyzer: valuta il consumo energetico del progetto, permettendo di ottimizzare l'efficienza e di garantire il rispetto dei limiti di potenza previsti dal progetto.

In generale, il tipico flusso di lavoro è caratterizzato dal seguente schema:

1. *Design*: attraverso l'editor, viene specificato il circuito desiderato.
2. *Sintesi*: il progetto inserito viene sintetizzato in un circuito costituito dagli elementi logici disponibili nel chip FPGA.
3. *Simulazione funzionale*: il circuito sintetizzato viene testato per verificarne la correttezza funzionale, ignorando momentaneamente i vincoli temporali.
4. *Fitting*: lo strumento CAD (*Fitter*) determina il posizionamento degli elementi logici della netlist all'interno degli elementi logici reali dell'FPGA; creando le connessioni (*routing*) necessarie tra di essi.
5. *Analisi temporale*: vengono analizzati i ritardi di propagazione lungo i vari percorsi del circuito per stimarne le prestazioni.
6. *Simulazione temporale*: il circuito "fittato" viene testato per verificare sia la correttezza funzionale sia quella temporale.
7. *Programmazione e configurazione*: il circuito progettato viene implementato fisicamente nel chip FPGA.

1.2 Creazione di un progetto e step iniziali

1.2.1 Dalla creazione del progetto alla sintesi del circuito

Come già anticipato, Quartus mette a disposizione strumenti di simulazione dotati di numerose funzionalità e caratteristiche. Per rendere più chiaro il loro utilizzo e il loro scopo, si può considerare una semplice rete di esempio, come quella di un codificatore di Hamming $H(7, 4)$.

Ogni circuito logico progettato con Quartus è chiamato progetto. L'IDE lavora su un solo progetto alla volta e mantiene tutte le informazioni relative ad esso in una singola cartella nel file system del computer su cui si sta lavorando. Avviando Quartus per la prima volta, appare una schermata simile a quella mostrata in Figura 1.1.

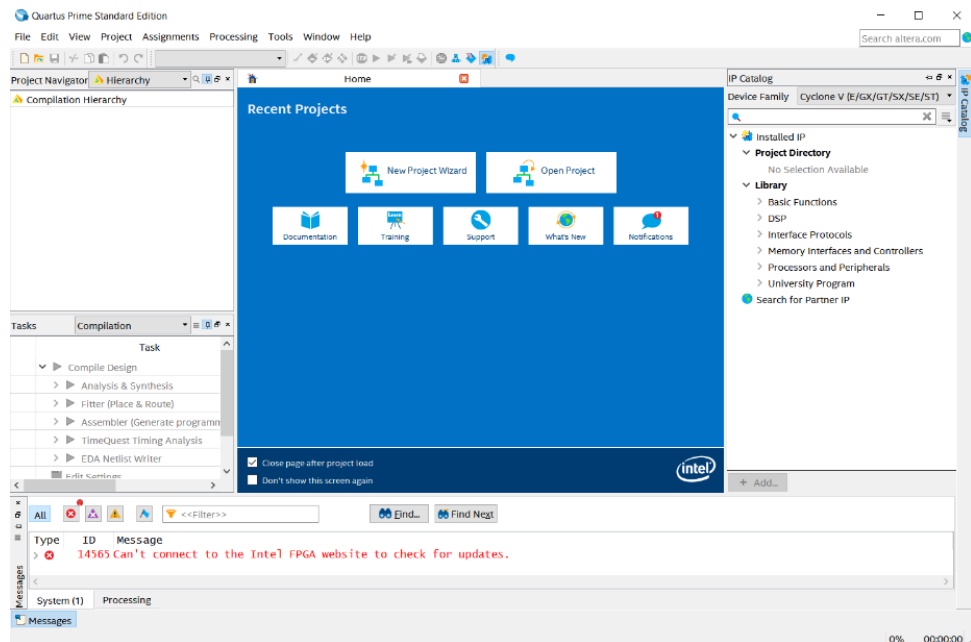


Figura 1.1: Interfaccia principale dell'ambiente di sviluppo Quartus al suo primo avvio [3].

Questa schermata è composta da diverse finestre che forniscono accesso a tutte le funzionalità del software. La maggior parte dei comandi in Quartus è accessibile tramite una serie di menù situati subito sotto la barra col nome dell'IDE (o del progetto nel caso in cui ci si stia già lavorando). Ad esempio, facendo clic con il pulsante sinistro del mouse sulla voce *File* si apre il menù mostrato in Figura 1.2 da cui, selezionando la voce *Exit*, sarà possibile uscire dall'ambiente di sviluppo.

Per iniziare a lavorare sulla nostra rete sarà necessario definire un nuovo progetto. Il software utilizzato facilita questo compito fornendo una procedura guidata, divisa nei seguenti step:

1. Selezionare *File > New Project Wizard* e fare clic su *Next* per raggiungere la finestra mostrata in Figura 1.3.
2. In questa sezione, sarà necessario assegnare al progetto un nome, che solitamente coincide con quello della *top entity* inclusa nel progetto, ed il percorso della cartella associata. Dopo aver completato questa parte, si potrà cliccare su *Next*.
3. La successiva finestra dedicata al *Project Type*, mostrata in Figura 1.4, consente di scegliere tra le opzioni *Empty project* e *Project template*. Per questo esempio si può selezionare la prima opzione, poiché il progetto verrà creato da zero.
4. Successivamente, la procedura guidata, come mostrato in Figura 1.5, permette di specificare eventuali file esistenti da includere nel progetto. Poiché,

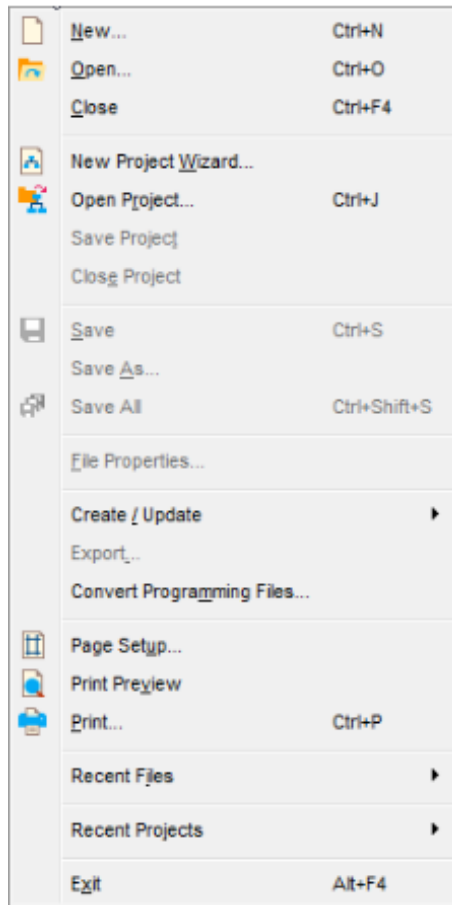


Figura 1.2: Menù a tendina dal quale sarà possibile sia creare o aprire nuovi file e progetti, sia uscire dall'ambiente di lavoro.

nel nostro caso, non ne abbiamo, si clicca su *Next* per passare alla finestra in Figura 1.6.

5. È necessario specificare il tipo di dispositivo su cui verrà implementato il circuito progettato. In questo esempio, come in tutto il lavoro di tesi, è stata scelta la famiglia di dispositivi *Cyclone V*. È possibile lasciare che Quartus Prime selezioni automaticamente un dispositivo specifico oppure sceglierlo manualmente. In questo caso si seleziona esplicitamente la scheda FPGA che andremo ad utilizzare, ovvero la *5CSEMA5F31C6*, presente sulla board *DE1-SoC* [6].
6. Come mostrato in Figura 1.7, è possibile specificare eventuali strumenti di terze parti da utilizzare e il linguaggio HDL scelto. Selezioniamo VHDL, lasciamo come EDA il software di default Questa e clicchiamo su *Next*.
7. Verrà, infine, mostrato un riepilogo delle impostazioni scelte. Premere *Finish* per tornare alla finestra principale di Quartus Prime, con *codificatoreham-*

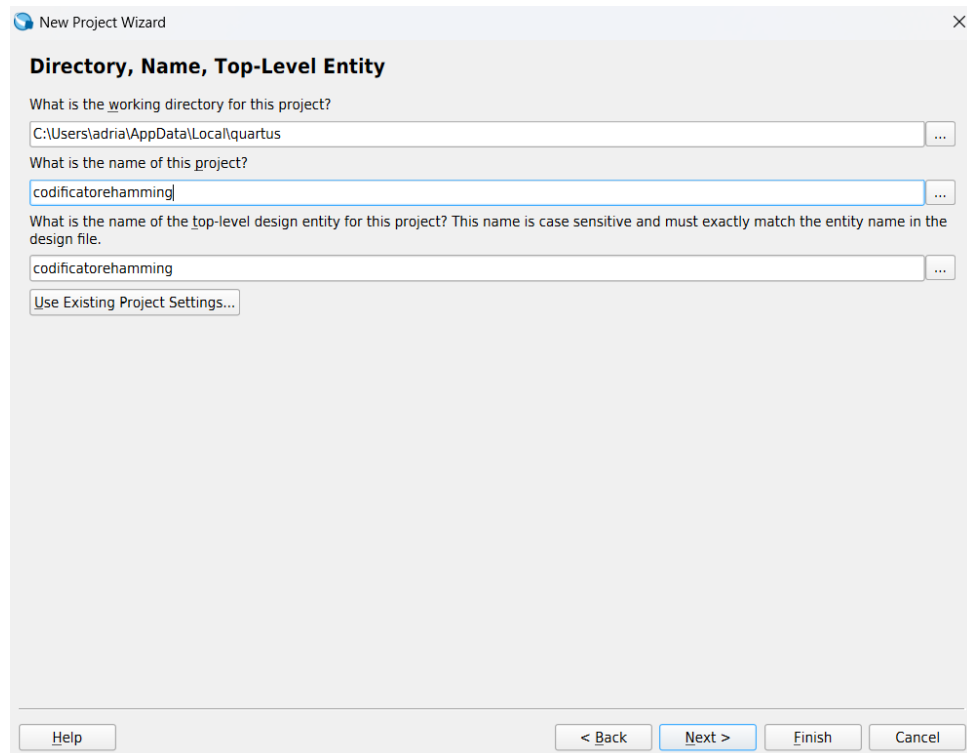


Figura 1.3: Selezione del nome e filepath per la directory contenente il progetto

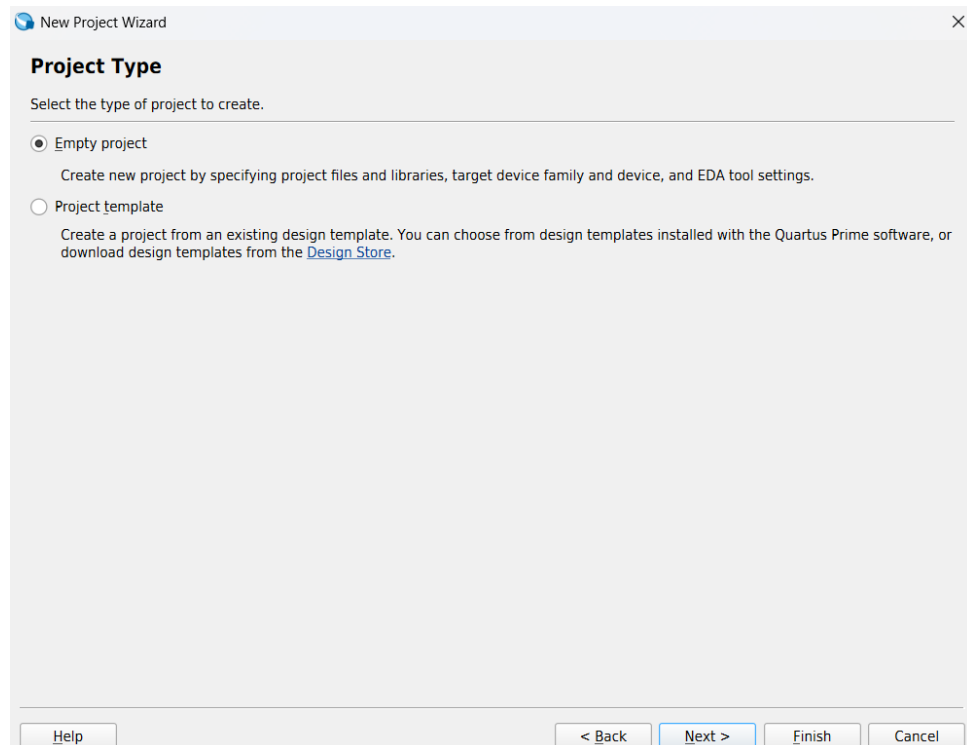


Figura 1.4: Selezione del tipo di progetto.

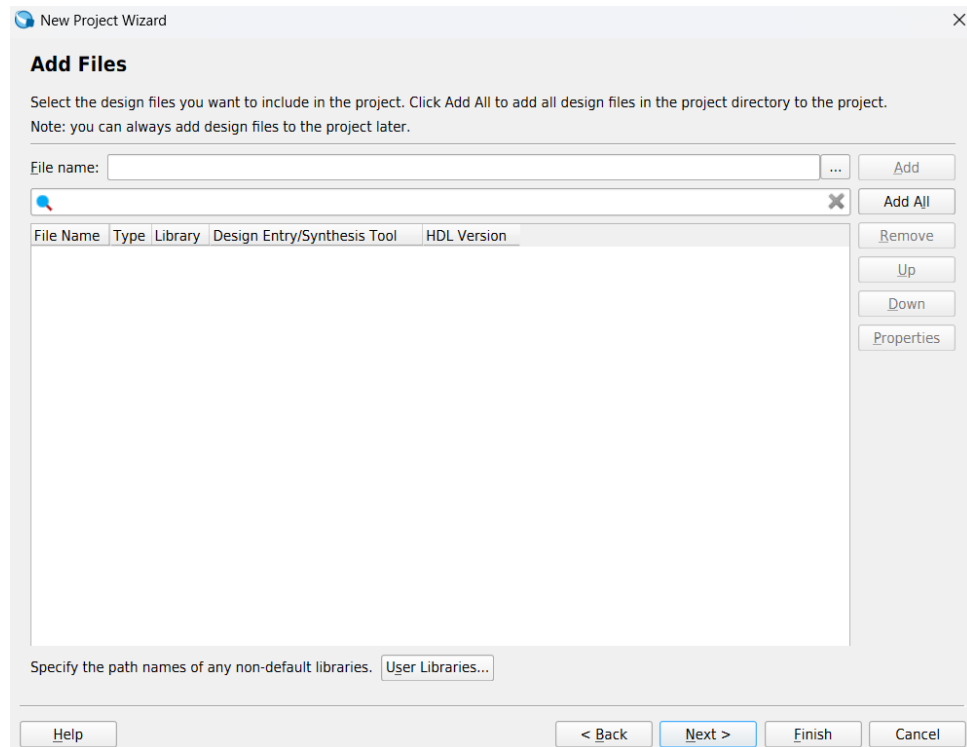


Figura 1.5: Selezione di eventuali file esistenti da includere nel progetto.

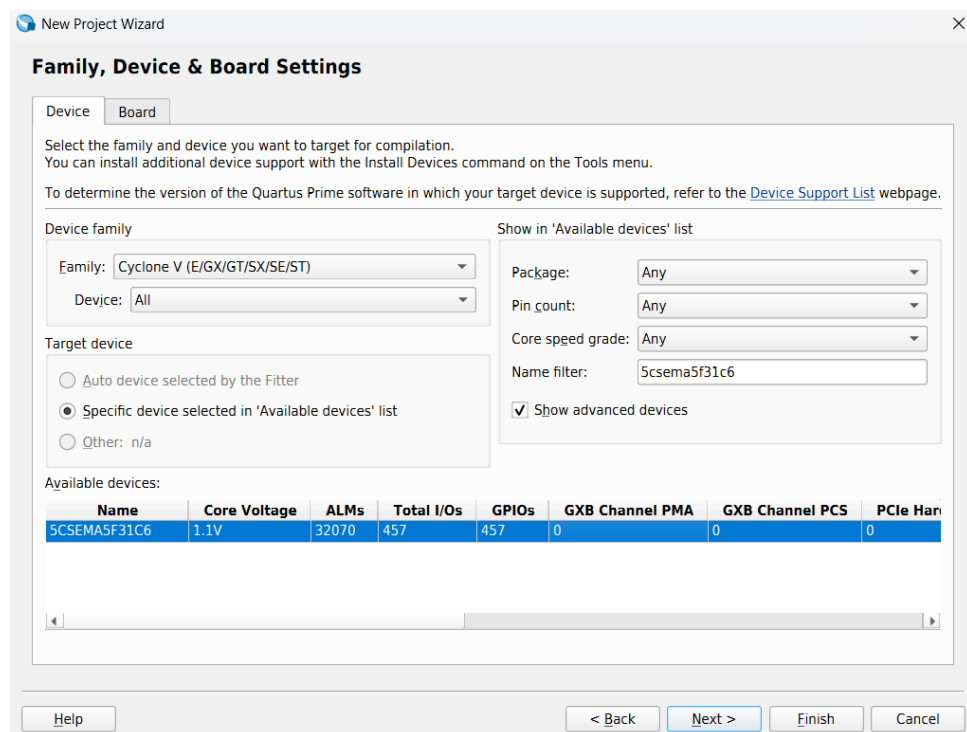


Figura 1.6: Selezione del dispositivo FPGA di progetto.

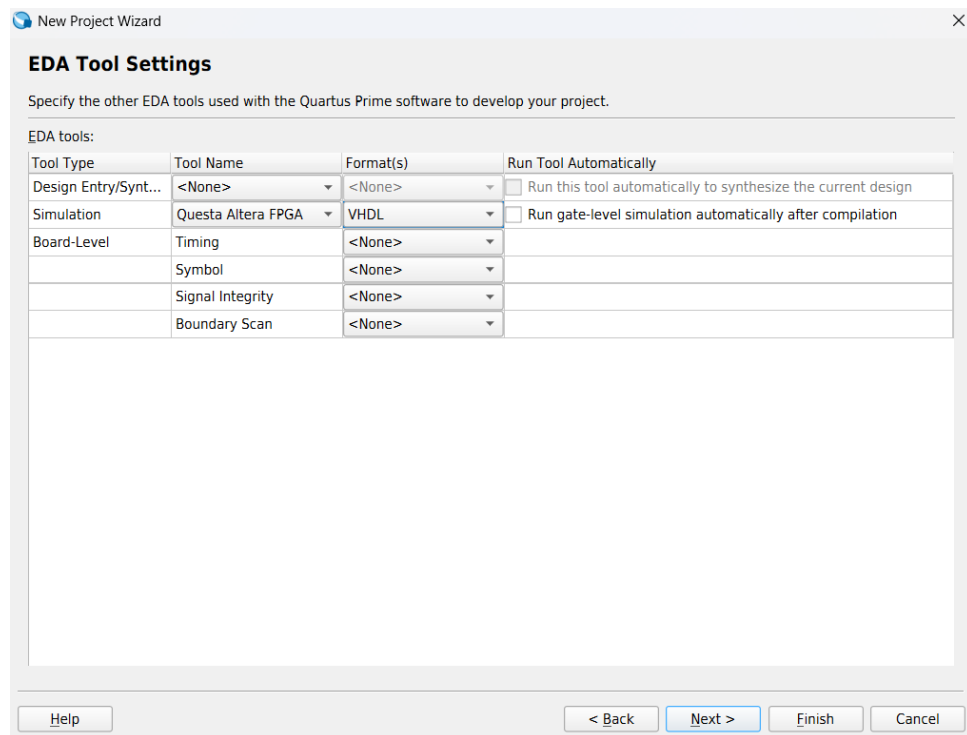


Figura 1.7: Selezione delle impostazioni di progetto riguardo HDL ed software di terze parti utilizzati.

ming impostato come progetto corrente nella barra del titolo, come mostrato in Figura 1.8.

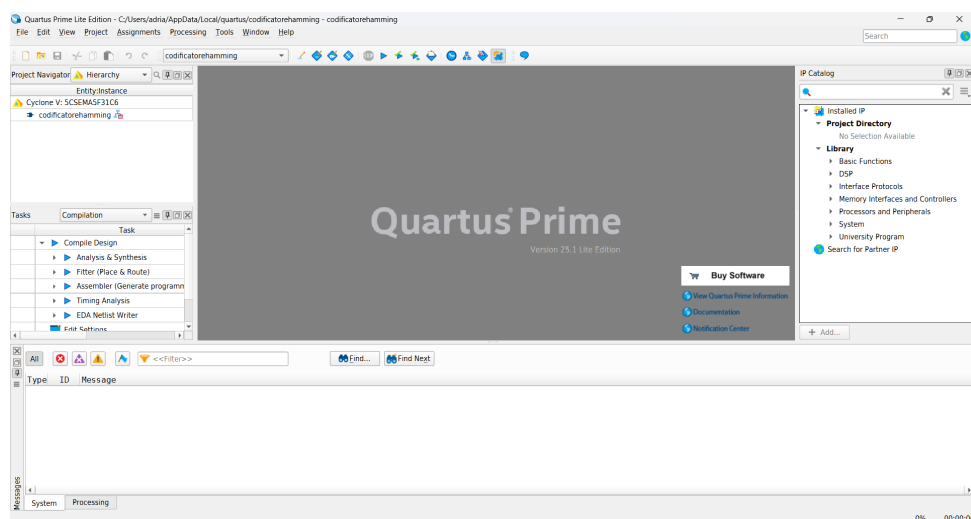


Figura 1.8: Interfaccia principale dopo la creazione di un progetto.

1.2.2 Design della rete

Dopo aver creato un nuovo progetto, il primo passo consiste nel descrivere il circuito di riferimento utilizzando il linguaggio di progettazione VHDL, che verrà approfondito in seguito. Senza entrare troppo nel dettaglio, è sufficiente sapere che il sistema avrà come ingresso un vettore di 4 bit chiamato `input` e come uscita un vettore di 7 bit chiamato `output`. Quest'ultimo sarà costituito, come previsto dal codice, dai tre bit di parità generati (nelle posizioni 1, 2 e 4, rispettivamente relativi alle quadruple 1357, 2367 e 4567) insieme ai quattro bit originali (nelle posizioni 3, 5, 6 e 7). Sempre dal menù a tendina mostrato in Figura 1.2, sarà possibile creare un nuovo file e specificarne il tipo, che in questo caso sarà, ovviamente, *VHDL File*. Dopo la scrittura ed il salvataggio del codice, si procede con l'analisi e la sintesi (selezionabile dall'omonima icona presente nella barra in alto), in modo da individuare eventuali errori e poter studiare il sistema ottenuto. Una delle prime e più semplici operazioni che si possono eseguire dopo la sintesi consiste nel visualizzare il circuito equivalente appena generato tramite l'RTL Viewer messo a disposizione dall'IDE nei suoi *Tool*, selezionabili nell'omonimo menù a tendina: come mostrato in Figura 1.9, lo schema risulterà proprio quello previsto dalla logica di partenza.

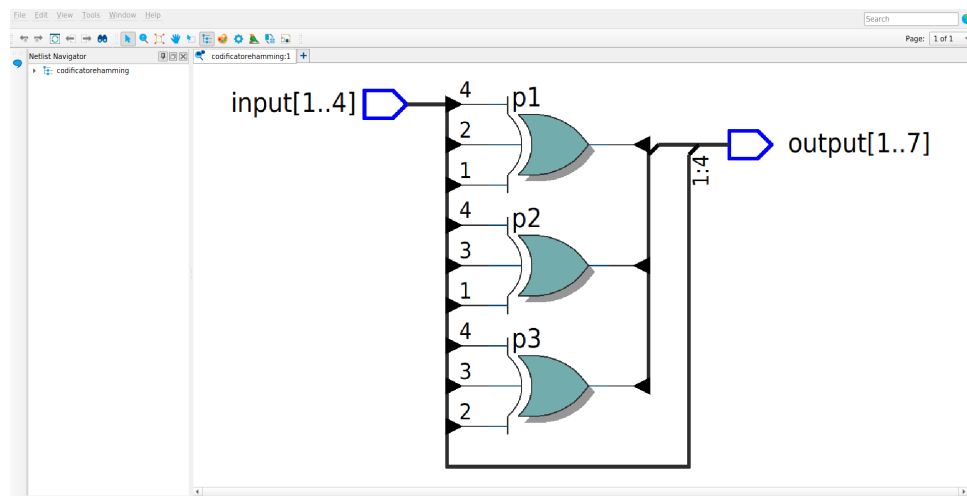


Figura 1.9: Circuito del codificatore di Hamming sintetizzato.

1.3 Strumenti per la simulazione

1.3.1 Utilizzo e panoramica del Waveform Editor

Per verificare il corretto funzionamento della rete, una delle numerose possibilità consiste nel ricorrere alle funzionalità del Waveform Editor. Questo strumento, come parzialmente anticipato, permette di far variare nel tempo in varie modalità i segnali in ingresso e visualizzare il conseguente andamento dei segnali di uscita.

Per prima cosa, si andrà a creare ed inserire nel progetto, sempre dal menù a tendina di Figura 1.2, un file di tipo *University Program VWF*, dall'estensione *.vwf. Quando questo verrà aperto, si aprirà automaticamente la finestra del Waveform Editor, come mostrato in Figura 1.10.

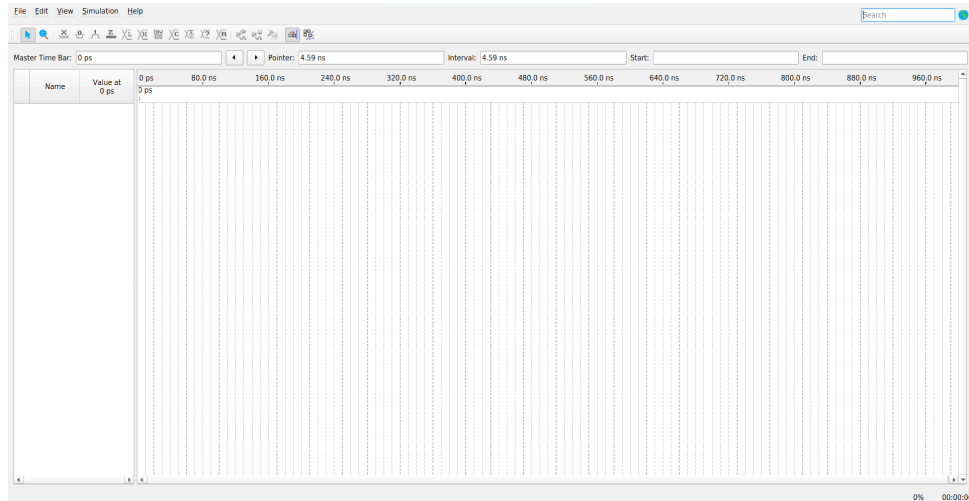


Figura 1.10: Waveform Editor alla sua prima apertura.

Si procede cliccando due volte nella colonna a sinistra: nel pop-up che verrà mostrato, cliccare su *Node Finder* per aprire l'omonima finestra, illustrata in Figura 1.11.

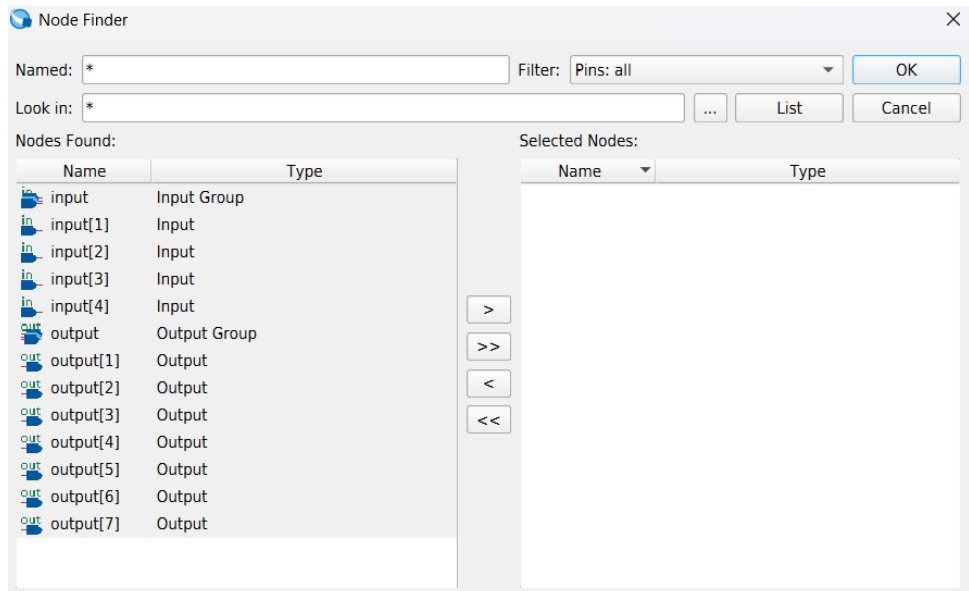


Figura 1.11: Finestra del *Node Finder*.

Si procede cliccando su *List* per rendere disponibili tutti i nodi presenti: nel nostro caso, clicchiamo due volte sui singoli elementi dei due vettori e confermiamo

la nostra scelta con *Ok* sia in questa finestra, che nel piccolo pop-up immediatamente precedente. Successivamente, sarà necessario assegnare un andamento ad i valori in ingresso: come mostrato nella barra in alto di Figura 1.10, sarà possibile assegnare ad un segnale, dopo averlo selezionato, un valore costante o variabile nel tempo, di natura arbitraria o anche randomica. Nel nostro esempio, faremo seguire ad ogni bit del vettore in ingresso un segnale di *clock* (cliccando sulla corrispettiva icona) dal periodo multiplo del successivo, in modo da avere progressivamente un valore complessivo via via crescente. Concluso ciò, si potrà eseguire la simulazione funzionale (*Simulation > Run Functional Simulation*) e visualizzare, come mostrato in Figura 1.12, i bit del segnale in uscita.

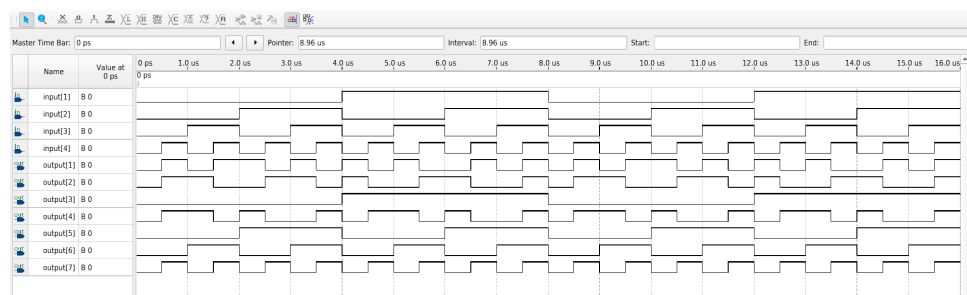


Figura 1.12: Risultato della simulazione funzionale del codificatore.

1.3.2 Esempio di applicazione di Questa Starter Edition

Un altro modo per verificare il corretto funzionamento del progetto consiste nello scrivere delle testbench: si tratta di file in codice HDL creati appositamente per testare il comportamento del circuito, fornendo stimoli in ingresso e controllando le uscite. Procederemo quindi a scriverne una adatta al nostro codificatore, concentrandoci, per semplicità, solo sui primi tre casi di test. Una volta completata la scrittura della testbench, sarà sufficiente indicare il nome di quest'ultima e il suo file nelle impostazioni del software di sintesi (*Assignments > Settings > EDA Tool Settings > Simulation*, poi cliccare su *Testbenches > New* nel successivo pop-up, inserire il file corretto e premere su *Ok*, per poi chiudere tutto con *Apply*) in modo che venga compilata correttamente, come mostrato in Figura 1.13.

Successivamente, dopo la compilazione, potremo avviare una *RTL Simulation* (*Tools > Run Simulation Tool > RTL Simulation*), ovvero una simulazione funzionale basata sui dati inseriti nella testbench. Durante la simulazione, si aprirà automaticamente il software ausiliario Questa, che permette di visualizzare le forme d'onda dei segnali generate, modificarle e analizzarne i risultati. Ad esempio, è possibile utilizzare dei cursori per osservare, in un determinato istante di tempo, i valori assunti dai segnali di ingresso e di uscita, oltre che regolare l'asse dei tempi e cambiare l'estetica delle waveform presenti. Inoltre, come mostrato in Figura 1.14, è possibile cambiare la rappresentazione numerica dei segnali: nel nostro caso, abbiamo scelto la rappresentazione binaria al posto della predefinita esadecimale, in modo da rendere più agevole la verifica dei risultati ottenuti. Queste modifiche

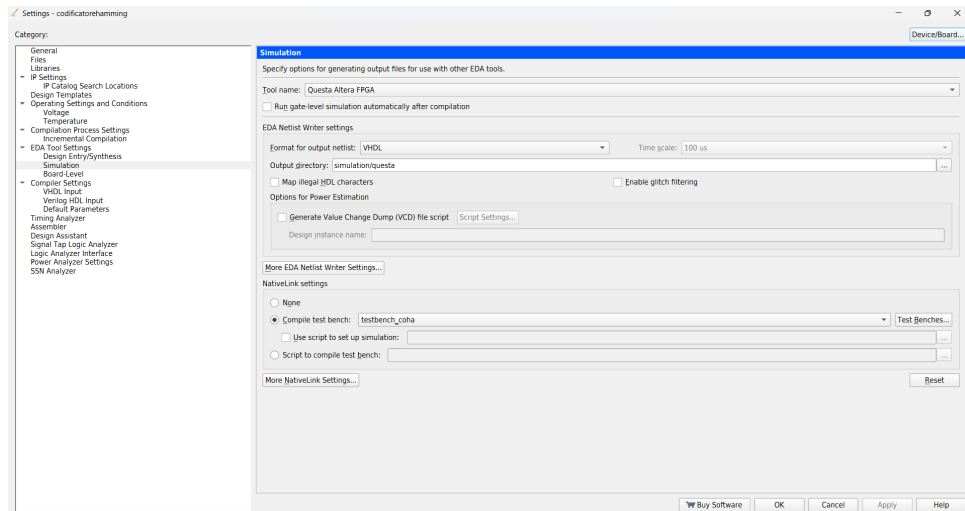


Figura 1.13: Selezione della testbench nel profilo di simulazione.

sui singoli segnali si effettuano cliccando col tasto destro sulla grandezza d'interesse, nella finestra aperta in automatico dedicata alle waveform chiamata *Wave*, e selezionando la voce *Properties*.

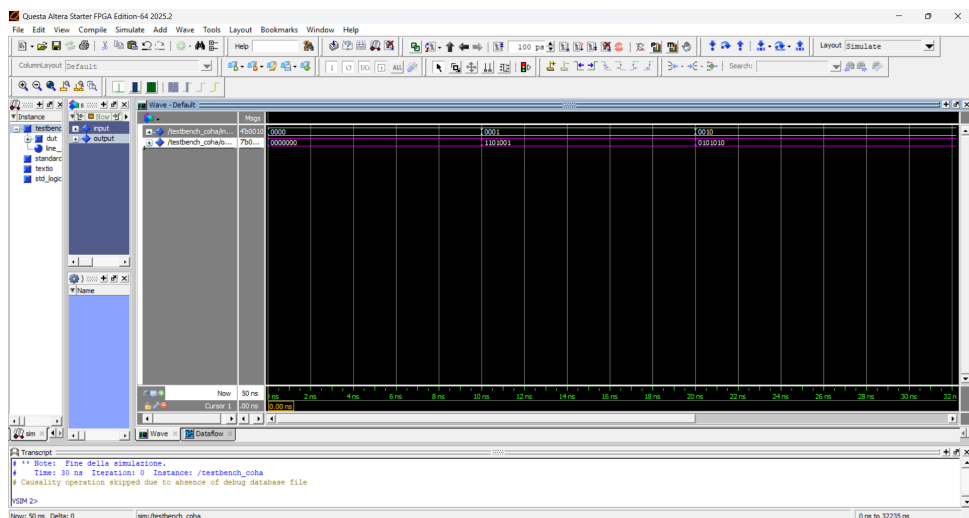


Figura 1.14: Risultato della simulazione RTL, in Questa, a partire dalla testbench scritta.

Il seguente software permette anche la visualizzazione di segnali interni, ovvero non immediatamente visibili nella simulazione funzionale in quanto non di I/O. Per fare ciò, dalla finestra *Objects*, che contiene tutti i segnali presenti nel design, si possono trascinare con l'ausilio del mouse gli eventuali segnali interni che si vogliono visualizzare all'interno della finestra *Wave*, in cui saranno già presenti gli ingressi e le uscite generali. Dopodichè, cliccare dal menù *Simulate* in alto la voce *Restart* e successivamente, sempre dalla stessa sezione, cliccare su *Run > Run*

-all. Come si può vedere in Figura 1.15, saranno mostrati anche gli andamenti dei segnali interni inseriti.

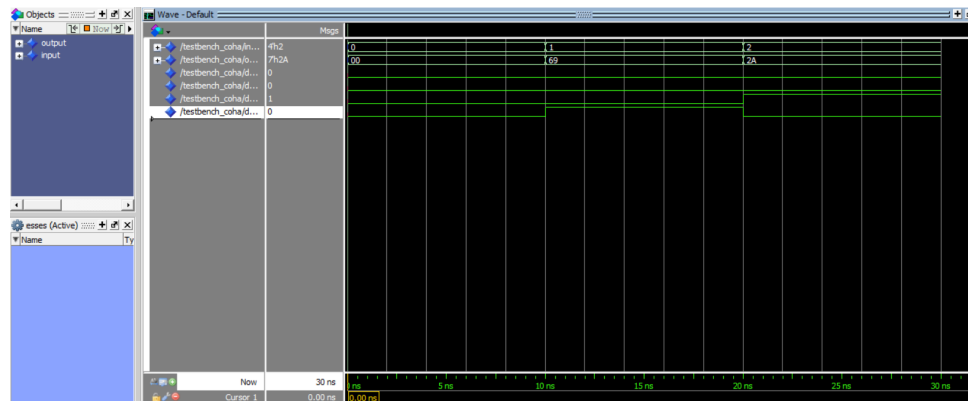


Figura 1.15: Risultato della simulazione RTL con la presenza di alcuni segnali interni opportunamente aggiunti.

Un'altra funzionalità molto interessante del seguente Tool, che vedremo applicata successivamente, è quella di poter visualizzare il contenuto di eventuali memorie presenti nel design. Ciò sarà possibile selezionando la voce *View > Memory List*.

1.4 Analisi Temporale in ambiente Quartus

1.4.1 Cenni teorici

1.4.1.1 Introduzione

L'analisi temporale rappresenta un processo fondamentale nello studio dei circuiti logici digitali, volto alla determinazione delle condizioni temporali in cui il circuito può funzionare in maniera affidabile.

I percorsi temporali all'interno di un FPGA possono essere classificati in tre categorie principali:

- *Clock Path*: percorso del segnale di clock dalla sorgente ai registri.
- *Data Path*: percorso dei dati dall'uscita Q del *source register* all'ingresso D del *destination register*.
- *Asynchronous Control Path*: percorso dei segnali di controllo asincroni, capaci di modificare l'uscita del registro indipendentemente dal clock (come eventuali segnali di reset).

Proprio per via della diversa natura dei percorsi analizzabili, si distinguono due principali tipologie di analisi temporale:

- Analisi sincrona: confronta il *clock path* con il *data path* per garantire che i segnali non raggiungano simultaneamente il registro di destinazione, evitando condizioni di metastabilità.
- Analisi asincrona: analizza la relazione tra *clock path* e *asynchronous control path*, al fine di prevenire instabilità causate da segnali asincroni.

1.4.1.2 Analisi sincrona

Nell'ambito della prima tipologia, i parametri fondamentali relativi al singolo registro sono:

- *Setup Time*: intervallo minimo prima del *clock edge* durante il quale il dato deve essere stabile.
- *Hold Time*: intervallo minimo successivo al *clock edge* durante il quale il dato deve rimanere stabile.

Questi parametri, presenti nel datasheet di ogni FPGA, definiscono una finestra temporale, detta *data required window*, entro la quale il segnale dati non deve variare.

Successivamente, sempre riguardo ai registri, si definiscono:

- *Launch Edge*: fronte di clock che avvia la propagazione dei dati dal *source register*.
- *Latch Edge*: fronte di clock che campiona i dati nel *destination register*.

Si possono così introdurre le seguenti grandezze:

- *Data Arrival Time*: tempo necessario affinché il dato raggiunga il registro di destinazione. Tale grandezza è quindi data dalla somma di:
 - T_{clk1} , il tempo che impiega il segnale di clock per arrivare al registro di partenza.
 - t_{co} , il ritardo di output del primo registro.
 - T_{data} , il tempo che impiegano i dati uscenti dal primo registro ad arrivare all'ingresso del secondo.

In particolare, sarà presente un valore di massimo associato al percorso più "lungo" (che indicheremo con $T_{arrival}^{max}$), ed un valore di minimo associato al percorso più "corto" (che indicheremo con $T_{arrival}^{min}$).

- *Clock Arrival Time*: tempo di arrivo del clock al registro di destinazione. Si ottiene dalla somma della differenza fra il *Latch Edge* ed il *Launch Edge* (che, nel caso di condivisione dello stesso clock da parte dei due registri, è generalmente il periodo), e il ritardo con cui il fronte arriva al registro di destinazione.

Il corretto trasferimento dell'informazione tra due registri dipende dal rispetto di due vincoli fondamentali: quello di *setup* e quello di *hold*. In particolare:

- Il primo impone che il dato sia stabile per un certo intervallo di tempo prima dell'arrivo del *Latch Edge*. Se il dato arriva troppo tardi, ovvero non rispetta questo anticipo minimo, il registro potrebbe acquisire un valore errato o entrare in una condizione di metastabilità.
- Il secondo impone invece che il dato in ingresso al registro di destinazione rimanga stabile per un certo intervallo di tempo dopo il *Latch Edge*. Se il dato cambia troppo presto dopo il campionamento, il registro potrebbe non mantenere correttamente il valore acquisito.

Utilizzando le grandezze introdotte in precedenza, il tempo richiesto per il rispetto del vincolo di setup (detto, in entrambi i vincoli, *Data required time*) è dato da:

$$T_{required}^{setup} = T_{(clock\ arrival\ time)} - t_{setup} - t_{uncertainty} \quad (1.1)$$

Questa espressione rappresenta l'istante massimo entro cui il dato deve arrivare al registro di destinazione per essere correttamente campionato.

Analogamente, per il vincolo di hold:

$$T_{required}^{hold} = T_{clk2} + t_{hold} + t_{uncertainty} \quad (1.2)$$

In questo caso, l'equazione definisce l'istante minimo prima del quale il dato non deve cambiare, al fine di non compromettere la stabilità del valore acquisito. T_{clk2} , analogamente a T_{clk1} , rappresenta il tempo che impiega il segnale di clock per arrivare al registro di destinazione.

La verifica del rispetto dei vincoli temporali avviene, in modo immediato, tramite il calcolo dello *slack* per ognuno dei due vincoli, ovvero il margine temporale disponibile tra il comportamento reale del circuito e i limiti imposti dai vincoli. Le formule e il significato di queste grandezze sono, rispettivamente:

- *Setup Slack*:

$$Slack_{setup} = T_{required}^{setup} - T_{arrival}^{max} \quad (1.3)$$

Esprime quanto il dato arriva in anticipo rispetto al limite massimo consentito. Un valore positivo indica che il vincolo di setup è soddisfatto, mentre un valore negativo segnala una violazione. L'illustrazione grafica del calcolo di questa grandezza è illustrato in Figura 1.16

- *Hold Slack*:

$$Slack_{hold} = T_{arrival}^{min} - T_{required}^{hold} \quad (1.4)$$

Indica quanto il dato rimane stabile oltre il limite minimo richiesto dopo il fronte di clock. Anche in questo caso, un valore positivo implica il rispetto del vincolo di hold, mentre un valore negativo rimane, come nel caso precedente, un riscontro negativo. La Figura 1.17 mostra il calcolo del suddetto valore.

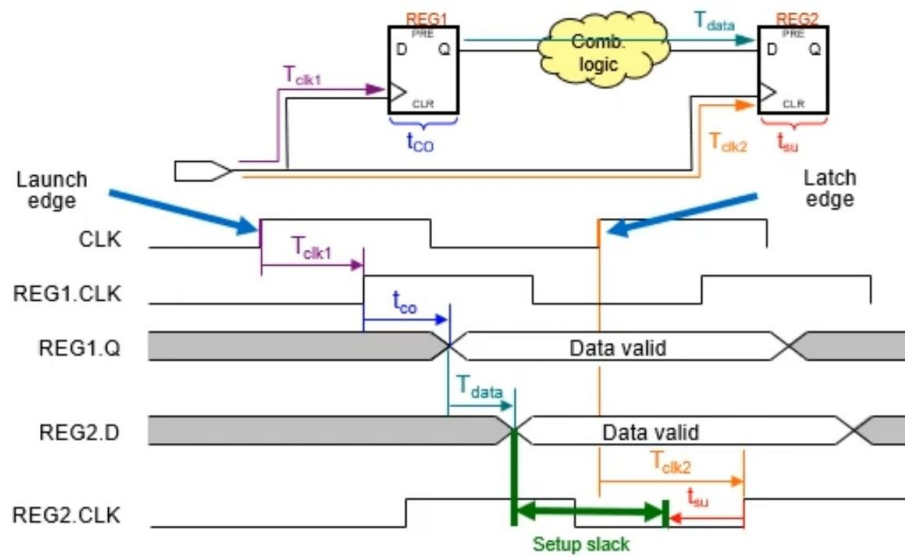


Figura 1.16: Illustrazione grafica del calcolo del *setup slack* [4]

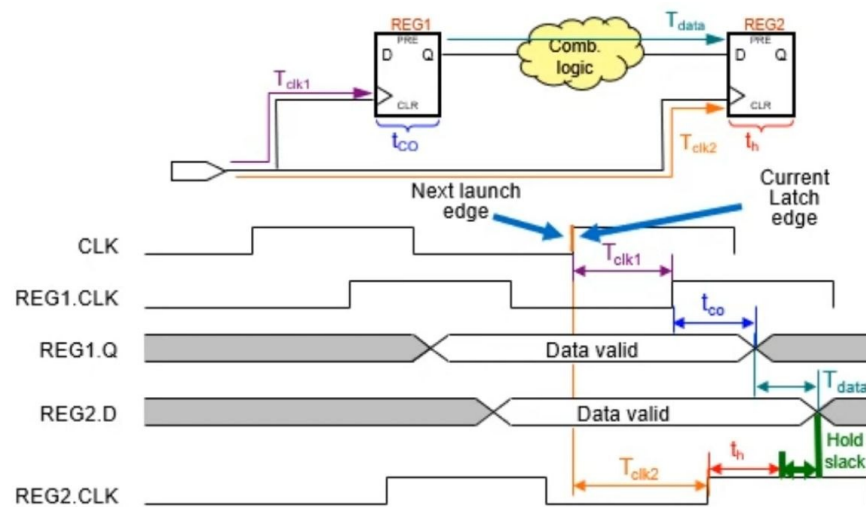


Figura 1.17: Illustrazione grafica del calcolo dell'*hold slack* [4]

1.4.1.3 Analisi asincrona

Nel caso di segnali asincroni, si considerano:

- *Recovery Time*: tempo minimo prima del *clock edge* in cui deve avvenire la disattivazione del segnale asincrono.
- *Removal Time*: tempo minimo dopo il *clock edge* in cui deve avvenire la disattivazione del segnale asincrono.

Anche in questo caso, ad ognuna di queste grandezze, verrà assegnato uno *slack*. In particolare, seguendo sempre le stesse dinamiche viste precedentemente, si definiscono:

- *Recovery Slack*:

$$Slack_{recovery} = T_{required}^{recovery} - T_{arrival}^{max} \quad (1.5)$$

- *Removal Slack*:

$$Slack_{removal} = T_{arrival}^{min} - T_{required}^{removal} \quad (1.6)$$

In entrambi i casi, come nei precedenti, valori positivi indicano il rispetto dei vincoli, mentre valori negativi ne segnalano l'infrazione.

1.4.2 Esempio di utilizzo del *Timing Analyzer*

Dopo aver fornito le basi teoriche per comprendere lo svolgersi dell'analisi temporale, per rendere più chiaro l'utilizzo di base del *timing analyzer* messo a disposizione da Quartus, risulta molto agevole considerare un semplice esempio: torniamo quindi al codificatore di Hamming precedentemente implementato ed aggiungiamo, a monte e a valle di esso, due registri sincroni sensibili al fronte di salita dello stesso segnale di clock, come mostrato in Figura 1.18.

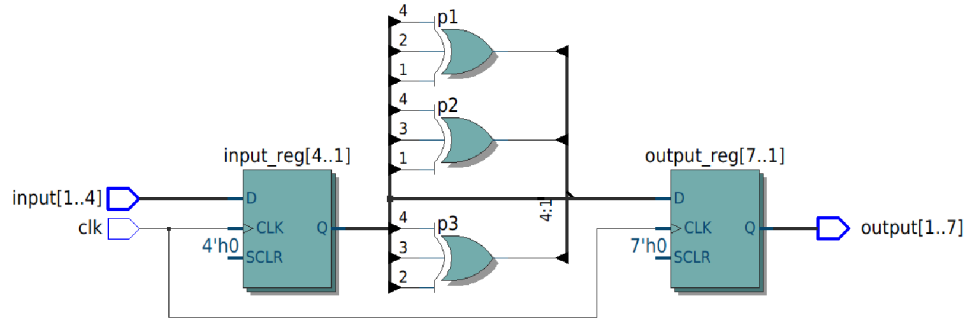


Figura 1.18: Circuito sintetizzato per illustrare il funzionamento del timing analyzer.

Si procede quindi, come di consueto, con la sintesi e la compilazione del programma appena realizzato. Tra le varie *tasks* presenti, nella finestra principale di Quartus, in basso a sinistra, è possibile aprire e studiare il contenuto di quella relativa all'analisi temporale, mostrato in Figura 1.19.

Tra i dati più interessanti presenti nel *Report*, sono presenti la frequenza e periodo del clock (impostata di default a 1000 MHz) nella voce *Clocks*; la frequenza massima teoricamente applicabile (pari, in questo caso, a 863.56 MHz); i *reports* relativi ai precedentemente descritti tempi di *recovery* e *removal* (che risulteranno

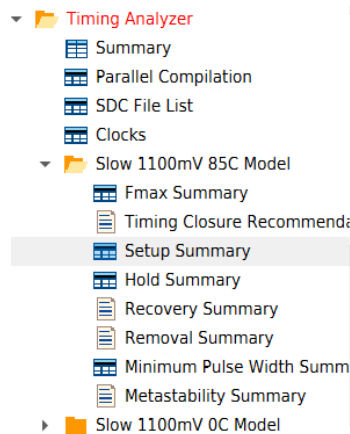


Figura 1.19: Voci presenti nel report dell'analisi temporale.

vuoti per l'assenza di segnali asincroni), e lo *slack time* sia di *setup* che di *hold*. Il secondo risulterà positivo e pari a 0.244 ns, mentre il primo, dal valore negativo di -0.176 ns, segnerà una violazione del suo rispettivo vincolo.

Si osservi che tali valori sono ottenuti considerando il *worst-case* tra le quattro diverse analisi effettuate sui possibili *corner* (combinazione tra le diverse condizioni di funzionamento del dispositivo in termini di variazioni di processo, tensione e temperatura), ovvero la configurazione chiamata *Slow 1100mV 85C Model*, in modo da garantire il funzionamento della rete anche nelle condizioni più critiche.

A partire dai dati ottenuti, sarà poi possibile aprire il *Time Analyzer* ed ottenere una panoramica più completa del vincolo problematico di *setup*. Per fare ciò, nel menù mostrato in Figura 1.19, selezionare *Setup summary*. Poi, nella tabella che mostra il risultato, cliccare col tasto destro sul valore e selezionare *Report Timing...(In Timing Analyzer UI)*. Infine, nel pop-up che appare, selezionare il segnale di clock corretto nella voce *From clock* e cliccare su *Report Timing*, lasciando il resto invariato. Il diagramma ottenuto è mostrato in Figura 1.20.

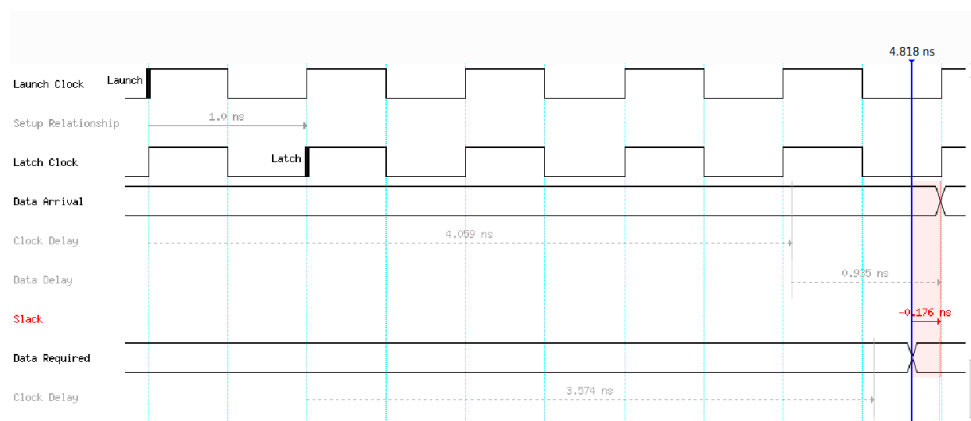


Figura 1.20: Diagramma temporale ottenuto con frequenza di clock pari a 1000 MHz

Come si può osservare, sono presenti tutte le grandezze finora descritte, come i due ritardi di clock, il *Data Delay*, e lo *slack*. Inoltre, tabellati, sono riportati anche il *Data Arrival Time* pari a 4.994 ns e il *Data Required Time* pari a 4.818 ns.

Dato il feedback negativo ottenuto, sarà necessario provare a ridurre la frequenza del segnale di temporizzazione. Si procede creando un nuovo profilo per il segnale di clock seguendo il percorso *Constraints > Create Clock*, ottenendo il pop-up mostrato in Figura 1.21. Da questa finestra, potremo cambiare il periodo (nel nostro esempio, si utilizzerà una durata pari a 4 ns) ed associare il seguente clock al suo corrispettivo nel progetto dalla sezione *Target*, trovando e selezionando, tra i vari segnali disponibili, quello di interesse, come mostrato in Figura 1.22.

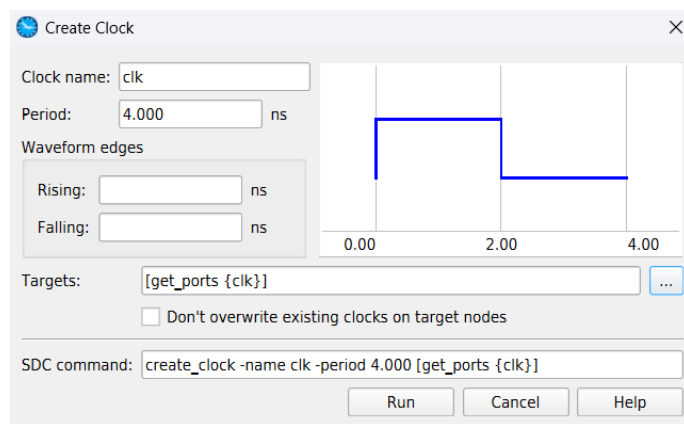


Figura 1.21: Finestra per la creazione di un nuovo clock.

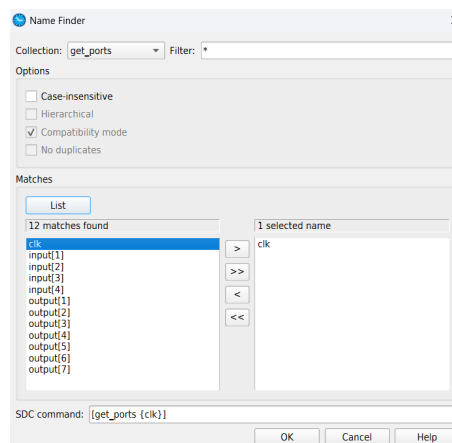


Figura 1.22: Finestra per associare il nuovo clock al suo corrispettivo nel design.

Si procede cliccando su *Run*, ottenendo in seguito una segnalazione relativa alla necessità di rigenerare le analisi compiute. Cliccare col tasto destro su una delle voci del report a sinistra e selezionare *Regenerate all out of date*. Così facendo, sarà possibile visualizzare i risultati ottenuti, mostrati in Figura 1.23. Come si può osservare, il nuovo *slack time* è positivo e pari a 2.842 ns.

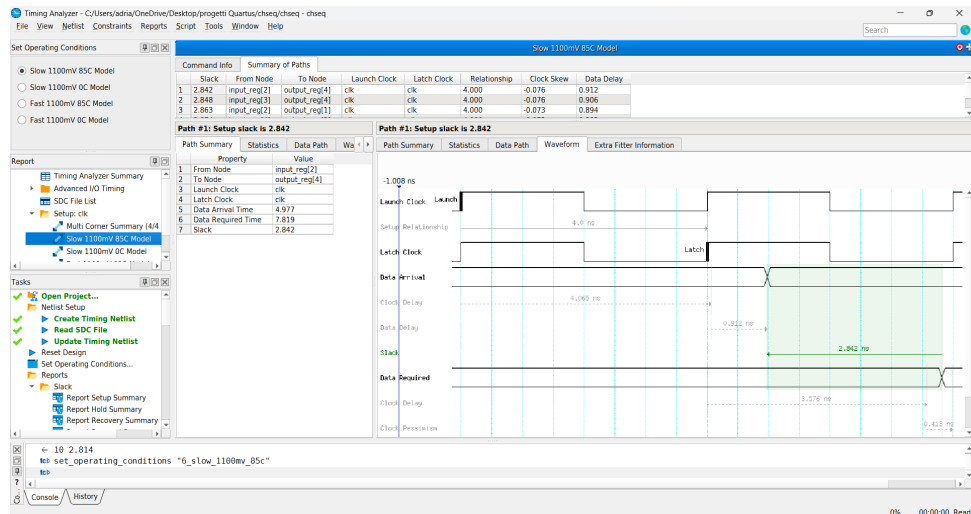


Figura 1.23: Diagramma temporale ottenuto con frequenza di clock pari a 250 MHz

Infine, per salvare le impostazioni di clock scelte per le prossime compilazioni, si procede creando un file *.sdc selezionando, tra le *Tasks* presenti in basso a sinistra, la voce *Write SDC File*. Dopodichè si chiude il Timing Analyzer per tornare all'interfaccia di Quartus. Si procede seguendo il percorso *Assignments > Settings > Timing Analyzer* e si inserisce, attraverso la finestra mostrata in Figura 1.24, il file *.sdc appena creato. Dopo aver confermato la scelta cliccando su *Apply*, le successive compilazioni utilizzeranno come clock di riferimento quello definito in precedenza, mostrando i nuovi valori di slack e di periodo.

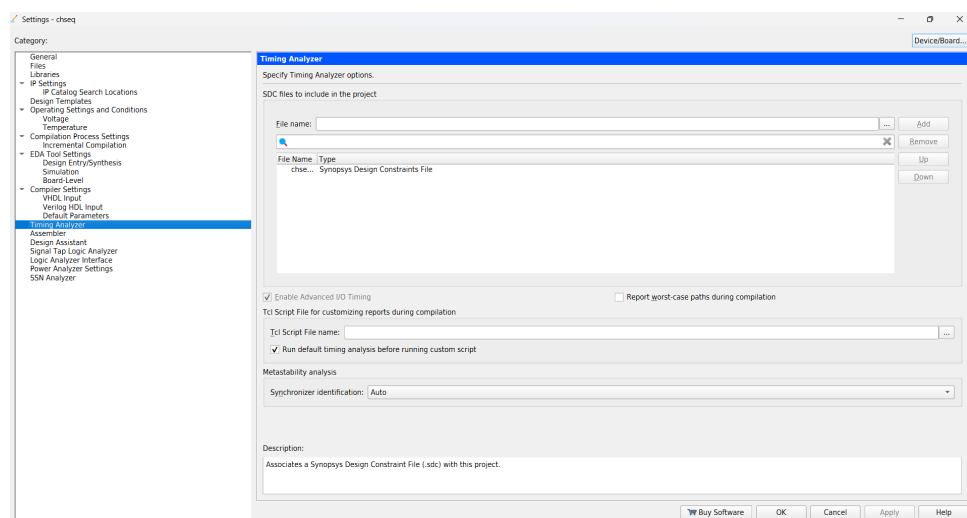


Figura 1.24: Finestra utile ad impostare il file *.sdc creato nel Timing Analyzer come riferimento per la compilazione.

1.5 Analisi di potenza in ambiente Quartus

1.5.1 Cenni Teorici

L'analisi della potenza, necessaria per verificare se il progetto realizzato rispetta il budget energetico imposto, richiede innanzitutto la comprensione delle diverse componenti di potenza necessaria al dispositivo nel corso del tempo, a partire dall'accensione fino al normale funzionamento.

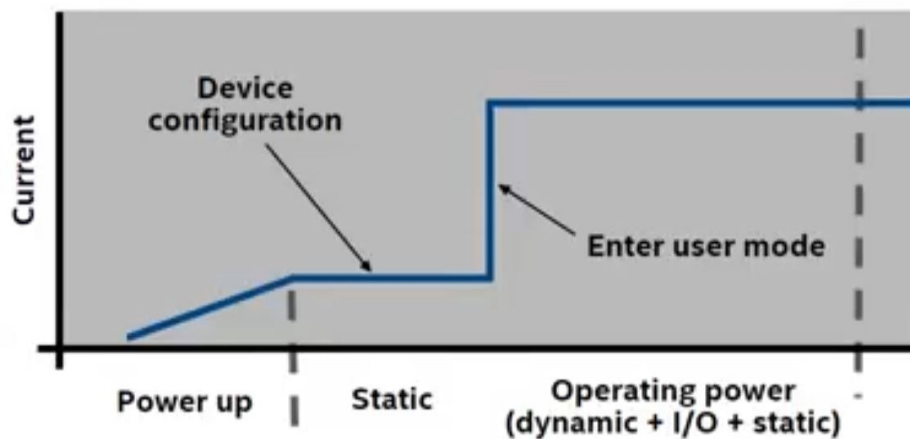


Figura 1.25: Andamento della corrente in una FPGA durante le diverse fasi operative [5]

Quando il dispositivo FPGA viene alimentato e programmato con un determinato design, la corrente assorbita aumenta progressivamente fino a raggiungere un livello iniziale di potenza definito come *potenza statica*. Durante questa fase iniziale, i clock del sistema sono mantenuti in stato inattivo.

Successivamente avviene la configurazione del dispositivo, dopodiché i clock vengono abilitati e il sistema entra nella cosiddetta modalità *user mode*, ovvero la condizione di funzionamento normale. In questa fase, il consumo totale di potenza è dato dalla somma di più contributi:

- potenza statica;
- potenza dinamica;
- potenza associata alle risorse di I/O, anche detta *standby power*.

La potenza statica è principalmente determinata dalle caratteristiche fisiche del dispositivo e dall'ambiente operativo. Essa include tra i suoi contributi principali gli effetti energetici delle correnti di leakage dei blocchi funzionali interni. Esclude, invece, il consumo dovuto ai pin di I/O o il consumo delle risorse analogiche, come i transceiver: quest'ultime componenti, infatti, sono spesso raggruppate sotto il termine di *standby power*.

In particolare, un ruolo fondamentale nella determinazione del valore della potenza statica lo gioca la temperatura. Infatti, la dipendenza da questa variabile è di tipo esponenziale: un aumento della temperatura di giunzione comporta un incremento significativo della potenza statica. Per questo motivo, è fondamentale adottare soluzioni di raffreddamento adeguate e considerare i casi peggiori nella progettazione dell'alimentazione.

La potenza dinamica rappresenta invece il contributo addizionale dovuto all'attività del circuito durante il funzionamento. Essa dipende da diversi fattori, e, a differenza della potenza statica, varia significativamente da progetto a progetto ed è in parte controllabile dal progettista. I principali parametri su cui è possibile intervenire sono, ad esempio, la frequenza di clock o la attività dei segnali.

Quest'ultima è un parametro chiave nella stima della potenza dinamica e si basa su due grandezze principali:

- Il *toggle rate* rappresenta la frequenza con cui un segnale cambia stato logico. Può essere espresso come percentuale rispetto al clock o, alternativamente, come numero di transizioni al secondo.
- La *static probability* indica la frazione di tempo in cui un segnale si trova al livello logico alto durante un intervallo di osservazione. Ovviamente, tale grandezza può assumere solo valori compresi tra 0 e 1.

1.5.2 Esempio di utilizzo del *Power Analyzer Tool*

Quartus permette di effettuare un'analisi di potenza di base direttamente in simulazione, senza la necessità di avere a disposizione una board FPGA fisica. Consideriamo nuovamente il circuito utilizzato nella precedente sezione: il primo passo da compiere è quello di inserire, nelle impostazioni (seguendo sempre il solito percorso), i valori dei parametri relativi alle condizioni operative nella sezione *Operating Settings Conditions*, ovvero le tensioni e la temperatura di lavoro. Per semplicità, si procede a lasciare i valori già presenti presenti "*di default*". Successivamente, sarà necessario sia includere l'analisi temporale durante la compilazione, sia fornire le direttive di lavoro al *power analyzer*. Entrambe le operazioni sono effettuabili nella sezione chiamata *Power Analyzer Settings* (mostrata in Figura 1.26), per cui saranno disponibili due opzioni:

- Fornire al tool un file *.vcd, generato a partire dalla simulazione di una testbench.
- Inserire, direttamente nelle impostazioni, i valori del *toggle rate* dei segnali di I/O e non.

Ovviamente, è facile dedurre che la prima strada permetta di avere valori più attendibili ed utili; tuttavia, poichè in esame si ha una rete molto semplice, si decide di optare per la seconda possibilità, inserendo come valore del *toggle rate* lo standard 12.5%. Si procede quindi con la compilazione e la lettura del *summary* relativo all'analisi di potenza presente, sempre, nel *compilation report*: come si

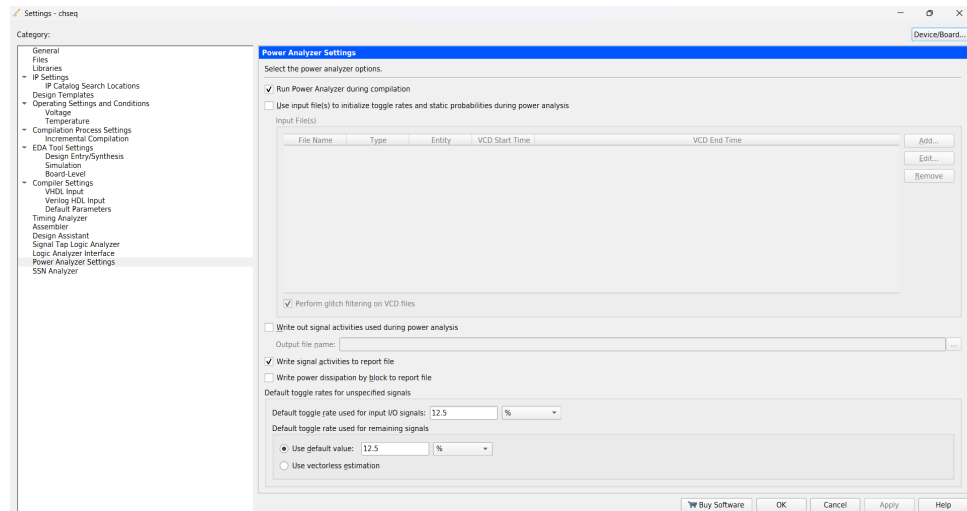


Figura 1.26: Finestra utile ad impostare le condizioni di esecuzione dell'analisi temporale.

può osservare da Figura 1.27, sarà possibile leggere i valori della potenza dinamica, statica e di I/O.

Power Analyzer Summary	
<<Filter>>	
Power Analyzer Status	Successful - Tue Apr 7 14:41:53 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	chseq
Top-level Entity Name	chseq
Family	Cyclone V
Device	5CSEMA5F31C6
Power Models	Final
Total Thermal Power Dissipation	428.44 mW
Core Dynamic Thermal Power Dissipation	1.05 mW
Core Static Thermal Power Dissipation	411.28 mW
I/O Thermal Power Dissipation	16.12 mW

Figura 1.27: *Power Analyzer Summary* ottenuto dopo la compilazione del circuito d'esempio.

1.6 Utilizzo della FPGA

1.6.1 Assegnamento dei pin

Dopo aver verificato il funzionamento logico della rete ed il rispetto dei vincoli temporali ed energetici, si può passare all'implementazione del circuito sulla FPGA scelta. Tuttavia, prima di programmare la scheda, sarà necessario assegnare gli input ed output del progetto a dei pin fisici presenti sulla board scelta. Il dispositivo fisico che verrà utilizzato durante il lavoro di tesi prende il nome di *DE1-SoC*². Un *System on Chip* (da qui la sigla utilizzata nel nome, *SoC*) rap-

²Tutte le informazioni relative alla scheda sono state reperite nel manuale ufficiale [6].

presenta un sistema integrato in cui diversi componenti elettronici fondamentali sono racchiusi all'interno di un'unica scheda. Nel nostro caso specifico, la FPGA *5CSEMA5F31C6* viene collegata con un insieme di dispositivi e periferiche già pronte all'uso, semplificando notevolmente la progettazione e la sperimentazione, oltre a creare un vero e proprio "laboratorio" localizzato sulla singola board. Le risorse messe a disposizione dal dispositivo sono mostrate in Figura 1.28.

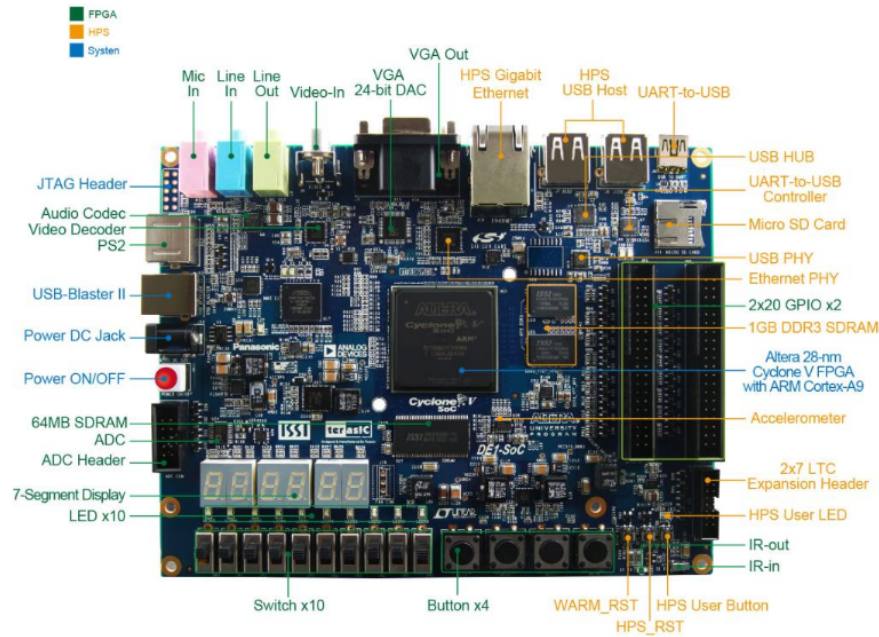


Figura 1.28: Layout della *DE1-SoC* [6].

Ipotizziamo di voler implementare il seguente sistema: i 4 bit di input saranno rappresentati da 4 *Slide Switches* corrispondenti, mentre i 7 bit in uscita saranno rappresentati da 7 dei 10 LED disponibili. Per prima cosa, sarà necessario aprire la finestra che ci permetterà di effettuare l'assegnamento tramite il percorso *Assignment > Assignment Editor*, mostrata in Figura 1.29.

Dopo aver selezionato come *Category* la tipologia *All*, si procede cliccando due volte sulla voce *To*, aprendo il classico pop-up del *Node Finder*. Come di consueto, si procede ad importare tutti i nodi esterni della rete. Successivamente si modifica, per ogni voce, la sezione *Assignment Name*, selezionando *Location (Accept Wildcards/Group)*, per poi assegnare, ad ogni nodo, il suo pin corrispondente, facendo riferimento al datasheet ufficiale [6]. In particolare, oltre agli I/O assegnati agli switch e ai LED, si assegnerà al clock della rete uno di quelli messi a disposizione dalla scheda: nel nostro caso, ne sceglieremo uno dalla frequenza di 50 MHz, ben al di sotto del valore critico. Inoltre, sarà possibile, nella sezione *Comment*, assegnare dei commenti testuali a ciascun nodo per salvare annotazioni ed informazioni utili. Il risultato finale è mostrato in Figura 1.30.

Si può quindi procedere con il salvataggio delle nuove impostazioni e la compilazione del progetto: tutte le informazioni relative ad i pin scelti saranno salvate

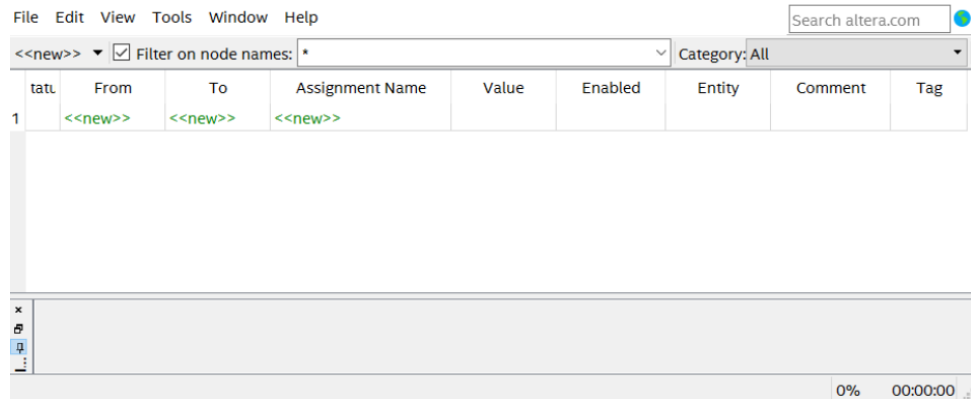


Figura 1.29: *Assignment Editor* alla sua prima apertura [3].

<<new>> Filter on node names: * Category: All									
tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag	
1	✓	in clk	Location	PIN_AF14	Yes		50MHz		
2	✓	in input[1]	Location	PIN_AF10	Yes		SW[3]		
3	✓	in input[2]	Location	PIN_AF9	Yes		SW[2]		
4	✓	in input[3]	Location	PIN_AC12	Yes		SW[1]		
5	✓	in input[4]	Location	PIN_AB12	Yes		SW[0]		
6	✓	out output[1]	Location	PIN_W20	Yes		LED[7]		
7	✓	out output[2]	Location	PIN_Y19	Yes		LED[6]		
8	✓	out output[3]	Location	PIN_W19	Yes		LED[5]		
9	✓	out output[4]	Location	PIN_W17	Yes		LED[4]		
10	✓	out output[5]	Location	PIN_V18	Yes		LED[3]		
11	✓	out output[6]	Location	PIN_V17	Yes		LED[2]		
12	✓	out output[7]	Location	PIN_W16	Yes		LED[1]		
13	<<new>>	<<new>>	<<new>>						

Figura 1.30: *Assignment Editor* dopo aver associato i pin fisici ai nodi.

in un file *.qsf che, nel caso di necessità, potrà essere importato anche in altri progetti.

1.6.2 Programmazione della scheda

Prima di programmare la scheda, sarà necessaria collegarla al PC ed all'alimentazione. Le schede della serie DE consentono la configurazione del dispositivo FPGA secondo due modalità distinte: *JTAG* e *Active Serial (AS)*. In entrambi i casi, i dati di configurazione vengono trasferiti da un computer *host*, sul quale è installato il software *Quartus Prime*, alla scheda tramite un cavo che collega una porta USB del computer al connettore *USB-Blaster* presente sulla board. Quando la FPGA viene configurata tramite *JTAG*, mantiene la configurazione finché il dispositivo resta alimentato; tuttavia, questa viene persa allo spegnimento. In alternativa, è possibile utilizzare la modalità *Active Serial*, dove i dati di configurazione vengono memorizzati in un dispositivo di configurazione dotato di memoria flash, presente sulla scheda DE-series. Il software *Quartus Prime* si occupa di programmare tale memoria, dalla quale i dati vengono automaticamente caricati nella FPGA all'accensione o durante una riconfigurazione. Questo approccio consente al sistema di mantenere la programmazione della scheda anche dopo lo spegnimento e la successiva riaccensione, senza la necessità di un nuovo caricamento da parte del computer. Nel presente lavoro si farà uso della prima modalità. Si procede

aprendo il *Programmer* presente nei *Tools* di Quartus. Per prima cosa, si verifica che la *Mode* corrente sia *JTAG* e che la voce *Hardware Setup* sia impostata sul dispositivo collegato, ovvero *DE-SoC*: in caso contrario, bisognerà modificare le sezioni menzionate per impostare le voci corrette. Successivamente, per permettere la programmazione della scheda, sarà necessario includere il suo *HPS*, processore integrato necessario per la gestione delle periferiche che verranno utilizzate: si procede quindi con l'aggiunta seguendo il percorso *Add Device > Soc Series V > SOCVHPS*. Infine, si verifica che sia presente nella finestra ed incluso nel progetto il file **.sof* relativo, necessario per la configurazione della FPGA: se ciò non è verificato, sarà possibile aggiungerlo manualmente sempre dal tool aperto. Alla fine di queste modifiche, la finestra dovrebbe apparire come in Figura 1.31.

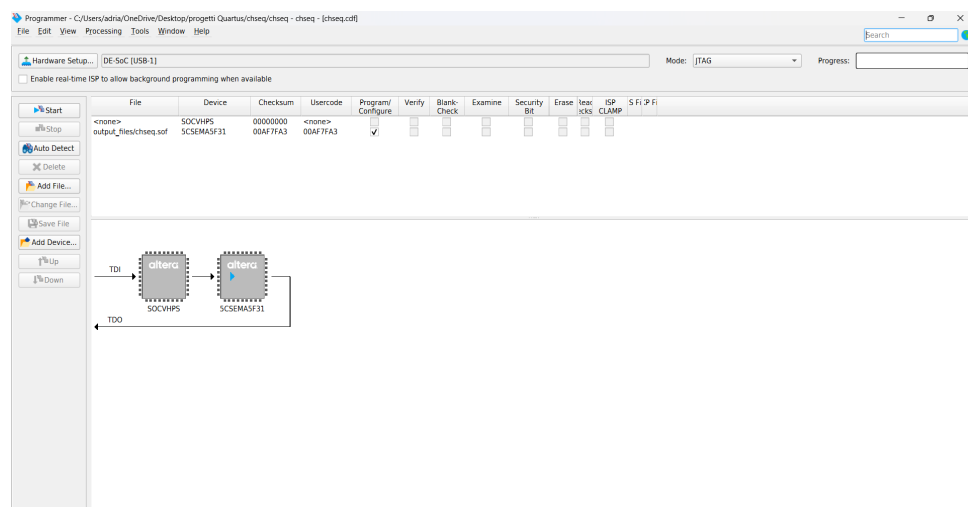


Figura 1.31: *Programmer* dopo aver impostato correttamente la sua finestra.

Concluso ciò, si può procedere col caricare il programma all'interno della scheda: cliccare su *Start* ed aspettare che il processo finisca. In Figura 1.32 ed in Figura 1.33 sono mostrati alcuni esempi pratici di utilizzo degli switch di input e di conseguente accensione dei LED di output.

1.6.3 Debug e verifica con *Signal Tap Logic Analyzer*

Il *Signal Tap Embedded Logic Analyzer* [8] è un analizzatore di logica utilizzabile sulla FPGA, particolarmente utile poichè non richiede l'ausilio di strumenti esterni. Infatti, a differenza dei metodi tradizionali che necessitano, ad esempio, di pin di I/O dedicati e sonde fisiche collegate alla scheda che potrebbero influenzare il comportamento della rete, Signal Tap viene integrato direttamente all'interno della FPGA, permettendo di monitorare direttamente i segnali "sepolti" all'interno del design in tempo reale. Per utilizzare questo strumento, la prima cosa da fare è creare un file adatto ad esso: come di consueto, si procede con la creazione di un file, questa volta di tipo *Signal Tap Logic Analyzer File*. Quest'ultimo, dall'estensione **.stp*, dovrà essere salvato possibilmente con lo stesso nome del

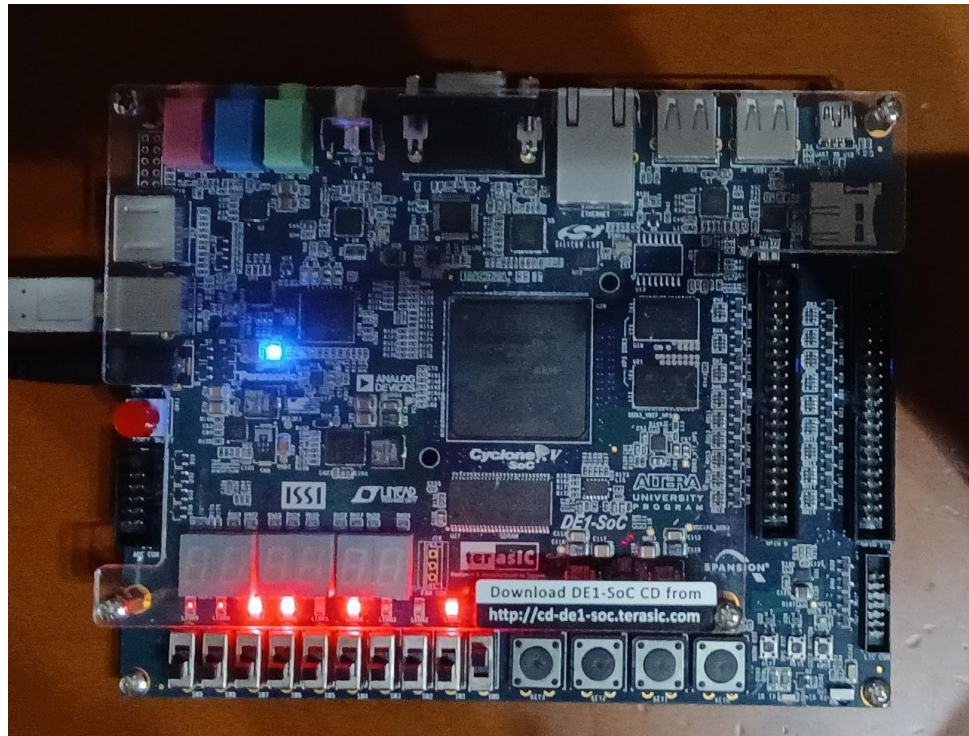


Figura 1.32: Esempio di utilizzo della scheda: l'input degli switch è pari a 0001, mentre l'output dei LED è, come prevedibile, 1101001.

progetto e poi impostato come file del suo tipo del progetto corrente: poichè alla prima creazione questa operazione viene fatta in automatico, non sarà necessario fare ulteriori operazioni. Si procede quindi con l'apertura del tool (cliccando due volte sul file o aprendo lo strumento dal menù dei *Tools*), che si presenterà come in Figura 1.34.

Per prima cosa, bisogna indicare quali segnali osservare durante l'esecuzione fisica del progetto: nella sezione di *Setup*, cliccare due volte su *Double click to add nodes*, procedendo ad inserire i nodi di interesse con il classico sistema del *Node Finder*, ed utilizzando come *Filter* l'impostazione *Signal Tap: pre-synthesis*. In questo esempio, ci limiteremo ad utilizzare i 4 bit di ingresso. Successivamente, è necessario indicare allo strumento il segnale di clock da utilizzare: nella sezione *Signal Configuration*, inseriamo il segnale di temporizzazione della rete. Ritornando alla sezione di *Setup*, i 4 nodi saranno adesso presenti in una tabella: la voce *Trigger Condition* permette di specificare la condizione che, se soddisfatta, farà catturare allo strumento i valori dei segnali indicati subito prima e dopo il suo avvenimento. Nel caso usato in Figura 1.35, si pone come condizione di trigger il valore alto del solo primo bit di input: infatti, un AND (*Basic AND*) tra una variabile e 3 *Don't Care* è pari alla variabile stessa.

Si verifica, infine, che la connessione sia di tipo JTAG e che la board sia letta e collegata al PC, per poi ricompilare il programma. Dopo la compilazione e la programmazione della scheda, si può riaprire *Signal Tap* ed effettuare il rilevamento:

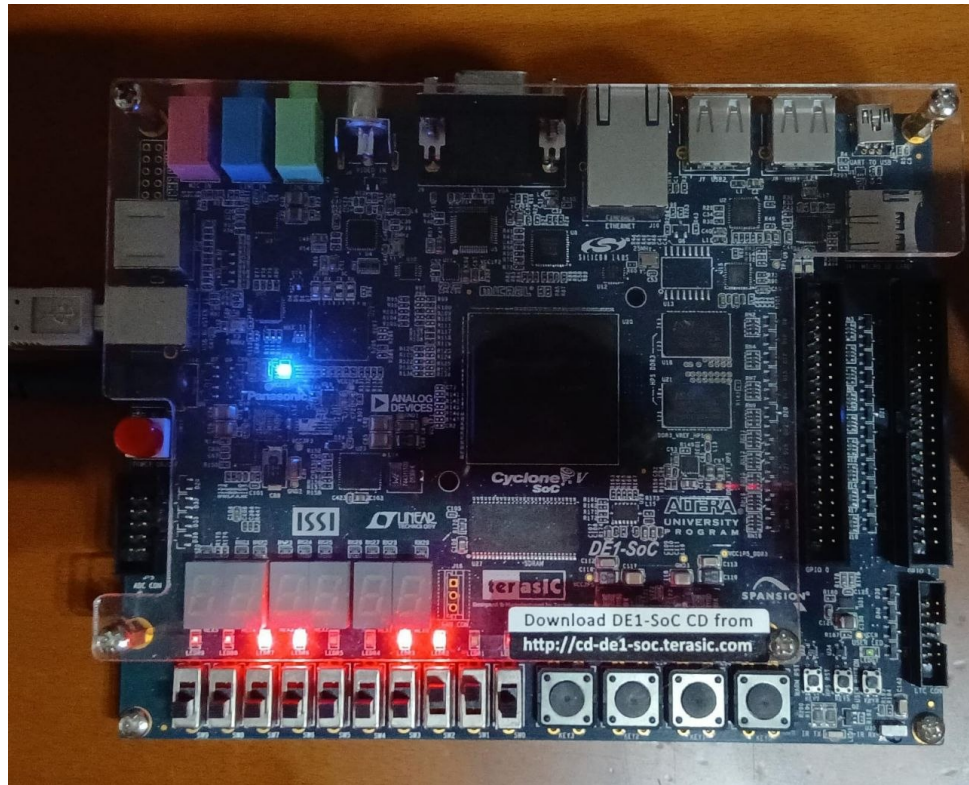


Figura 1.33: Esempio di utilizzo della scheda: l'input degli switch è pari a 0110, mentre l'output dei LED è, come prevedibile, 1100110.

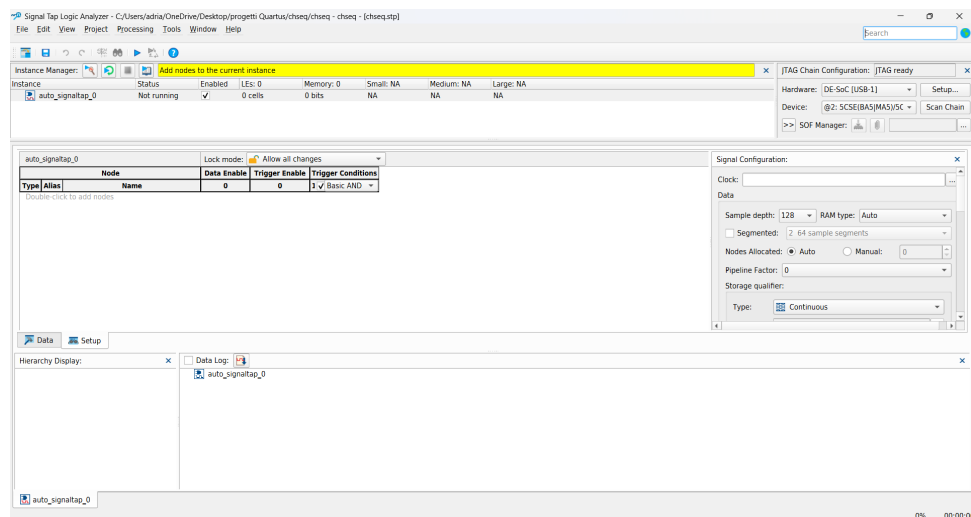


Figura 1.34: Finestra di *Signal Tap* alla sua prima apertura.

clickare su *Processing > Run Analysis* e portare a valore alto lo switch associato al primo input. Dopo aver effettuato quest'operazione, aprire la sezione *Data* per osservare le forme d'onda rilevate: come mostrato in Figura 1.36, è stato catturato un breve intervallo di tempo intorno alla condizione di trigger impostata e poi

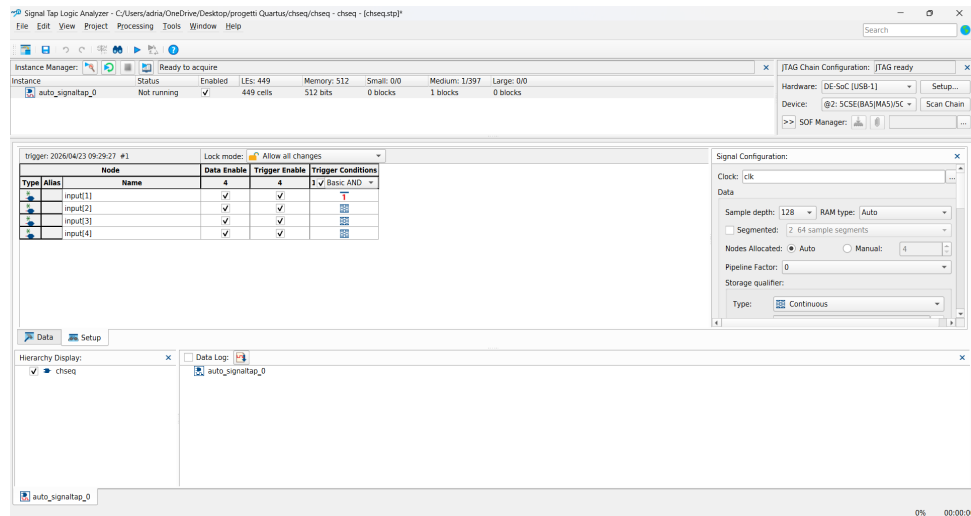


Figura 1.35: Finestra di *Signal Tap* dopo la prima modifica effettuata.

avvenuta.

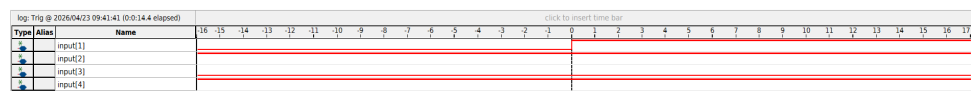


Figura 1.36: Forme d'onda catturate con il primo set di impostazioni.

La condizione di trigger può essere anche di natura sequenziale: per esempio, nella sezione *Signal Configuration*, trovare la sottosezione *Trigger* e selezione 4 nella voce *Trigger Condition*. Si può così procedere a modificare la tabella della finestra come mostrato in Figura 1.37: in questo modo si avrà la cattura delle forme d'onda nel momento in cui vengono rilevati, in modo sequenziale, i fronti di salita del primo, secondo, terzo e quarto bit di ingresso.

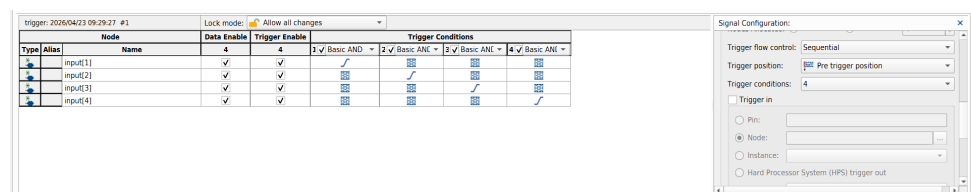


Figura 1.37: Finestra di *Signal Tap* con la condizione di trigger di tipo sequenziale.

Si ricompila il progetto e si riefettua l'analisi come in precedenza: con l'avvenimento della condizione di trigger, saranno catturate le forme d'onda mostrate in Figura 1.38.

Tornando alle condizioni combinatorie, quest'ultime possono essere rese più complesse: nella colonna della tabella chiamata *Trigger Condition*, scegliere dal menù a tendina la voce *Advanced*, che farà aprire immediatamente una finestra dedicata. In quest'ultima, si potranno trascinare con il mouse, all'interno della

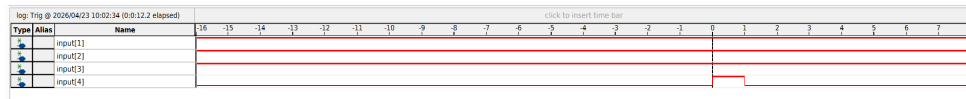


Figura 1.38: Forme d'onda catturate con il secondo set di impostazioni. Il momentaneo ritorno a 0 è causato dal fenomeno di *debounce* degli switch

window di lavoro, i segnali presenti ed i numerosi elementi logici e non messi a disposizione per personalizzare al massimo la condizione: in Figura 1.39 è mostrata la scelta che verrà utilizzata come esempio.

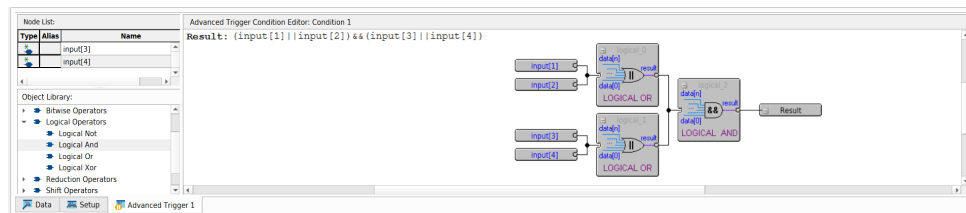


Figura 1.39: Finestra per la personalizzazione della condizione di trigger.

In questo caso, ad esempio, per attivare la condizione di trigger potremo portare a valore alto gli switch associati al bit 1 ed al bit 4: il risultato è mostrato in Figura 1.40.

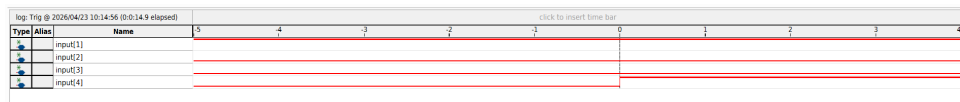


Figura 1.40: Forme d'onda catturate con il terzo set di impostazioni.

Capitolo 2

IL LINGUAGGIO VHDL

2.1 Introduzione

Il linguaggio VHDL (VLSI Hardware Description Language), come suggerisce il nome, è un linguaggio per la descrizione dell'hardware, ovvero permette di descrivere una rete logica ed i suoi elementi. A differenza dei linguaggi di programmazione, gli HDL non vengono usati per scrivere programmi *eseguibili*, ma ne permettono la *simulazione*, ovvero la simulazione del circuito digitale descritto dalle specifiche del linguaggio.

Nel seguente capitolo, verranno illustrate le fondamenta di questo linguaggio, insieme a tutte le funzionalità utilizzate nella realizzazione della CPU.

2.2 Creare un componente

2.2.1 Generalità

Un sistema più o meno complesso è caratterizzato da diverse unità funzionali, dette anche *moduli*, che ricoprono uno o più ruoli della rete completa, collaborando con le altre. Inoltre, è bene fare distinzione fra *modulo* e *istanza*: il primo è un'entità composta da:

- Interfaccia: descrive i segnali di ingresso e di uscita, permettendo di trattare ogni elemento come una "*Black Box*" e connetterlo agli altri
- Comportamento: descrive la trasformazione degli input in output, quindi il funzionamento del componente stesso

La seconda, invece, è un utilizzo di tale modulo nella realizzazione della rete.

In VHDL, ognuno di questi concetti assume un nome ben preciso:

- Un modulo prende il nome di *Design Entity*
- Un'interfaccia prende il nome di *Entity*

- Il comportamento prende il nome di *Architecture*

2.2.2 Entity

L'interfaccia di ciascun modulo viene definita tramite la dichiarazione di una *entity*. Questo costrutto definisce il nome del modulo, le sue porte e, opzionalmente, un insieme di parametri aggiuntivi. La forma generale di una dichiarazione di una *entity* è la seguente¹:

```
entity entity_name is
    [generic (generic_list);]
    [port (port_list);]
end entity_name;
```

Il nome *entity_name* identifica univocamente la design entity all'interno del progetto. La *port_list* rappresenta l'insieme delle porte e specifica i segnali di ingresso e uscita del modulo. Ogni porta viene dichiarata secondo la sintassi:

```
port_name [, port_name, ...] : {in | out | inout} port_type;
```

Il campo *port_name* indica il nome della porta, che verrà utilizzato nell'architettura per riferirsi al segnale associato. La parola chiave successiva specifica la direzione della porta: *in* per ingressi, *out* per uscite e *inout* per segnali bidirezionali.

In modo analogo, la lista dei parametri *generic_list* segue la forma:

```
generic_name [, generic_name, ...] : generic_type;
```

A differenza delle porte, i *generic* non hanno direzione e possono assumere anche tipi di dato complessi. Si riporta di seguito un esempio elementare di dichiarazione di una *entity*, relativa a una porta logica AND a due ingressi, parametrizzata tramite un *generic*:

```
entity and_gate is
    generic (
        N : integer := 1
    );
    port (
        a, b : in  std_logic_vector(N-1 downto 0);
        y    : out std_logic_vector(N-1 downto 0)
    );
end and_gate;
```

¹Il seguente modello generale, così come tutti i presenti ad eccezione di quello utilizzato nel Paragrafo 2.7, sono tratti da [9].

In questo esempio, il parametro *N* definisce la dimensione dei vettori di ingresso e uscita (le caratteristiche del tipo utilizzato verranno analizzate in seguito). Le porte *a* e *b* rappresentano quindi due segnali di ingresso a *N* bit, mentre la porta *y* rappresenta il segnale di uscita, anch'esso a *N* bit. Come si può osservare, l'utilizzo dei *generic* consente di rendere il modulo riutilizzabile e facilmente configurabile a seconda delle necessità.

2.2.3 Architecture

La descrizione del comportamento di un modulo in VHDL avviene tramite una *architecture declaration*, la cui forma generale è la seguente:

```
architecture architecture_name of entity_name is
    [declarations]
begin
    [implementation]
end architecture_name;
```

Da tale struttura si può osservare che ogni *architecture* è associata ad una sola *entity*, mentre non è valido il contrario. L'associazione tra una specifica *architecture* e la relativa *entity* viene definita come *configuration declaration*.

La sezione iniziale dell'*architecture*, indicata come **declarations** e opzionale, contiene un insieme di dichiarazioni utilizzate nella descrizione del modulo. Tali dichiarazioni, che godono di visibilità limitata alla sola *architecture* in cui sono definite possono appartenere a diverse categorie:

- Dichiarazioni di costanti: definiscono nomi, tipi e valori di costanti simboliche impiegate nel progetto.
- Dichiarazioni di segnali: specificano i nomi e i tipi dei segnali utilizzati nella descrizione funzionale.
- Dichiarazioni di tipi: introducono tipi di dato definiti dall'utente, utilizzati all'interno della specifica.
- Dichiarazioni di componenti: descrivono i moduli (componenti) che vengono istanziati all'interno dell'*architecture*.

Tra le parole chiave **begin** ed **end** è invece presente la sezione di **implementation**, nella quale viene descritta la funzionalità del modulo.

La parte implementativa è generalmente costituita da un insieme di *statement*, quali assegnamenti di segnali, assegnamenti condizionali, costrutti di selezione e istanziazioni di componenti. Tali istruzioni vengono valutate in parallelo e, di conseguenza, l'ordine con cui gli *statement* sono scritti all'interno dell'*architecture* non influisce sul comportamento finale del circuito.

Si riporta di seguito, sempre come esempio, la *architecture* associata alla *entity* precedentemente definita. In questo caso viene descritta l'implementazione di una porta logica AND vettoriale:

```

architecture func of and_gate is
begin
  y <= a and b;
end;

```

In questo esempio, l'architecture denominata `func` descrive il comportamento del modulo, determinando il valore del segnale di uscita `y` partendo dai due ingressi.

Si osserva come non siano necessarie dichiarazioni aggiuntive nella sezione `declarations`, in quanto il comportamento del modulo può essere espresso direttamente nella parte implementativa.

2.3 Tipi base del linguaggio VHDL

2.3.1 Il tipo `bit`

Il tipo `bit` rappresenta il tipo di dato più elementare disponibile in VHDL. Esso consente di modellare un valore binario e, di conseguenza, può assumere esclusivamente i valori logici '0' e '1'. Gli operatori applicabili a questo tipo sono limitati e comprendono essenzialmente operatori di assegnamento, operatori di confronto e operatori logici.

In molte applicazioni risulta utile considerare insiemi di segnali trattati come un'unica entità. A tal fine, il linguaggio VHDL mette a disposizione il tipo `bit_vector`, che rappresenta un array di elementi di tipo `bit`.

La sintassi per la dichiarazione di un segnale di tipo `bit_vector` è la seguente:

```

signal_name : bit_vector(index1 {to | downto} index2);

```

Questa dichiarazione definisce un segnale composto da un numero di elementi pari a `index2 - index1 + 1`, ciascuno dei quali è di tipo `bit`. L'ordinamento degli elementi è determinato dalla parola chiave utilizzata nella specifica degli indici. In particolare, utilizzando la forma `to`, l'indice cresce da `index1` a `index2` e il bit più significativo è quello associato a `index1`. Al contrario, con la forma `downto`, l'indice decresce e il bit più significativo corrisponde a `index2`.

Gli indici utilizzati per definire il vettore possono essere scelti liberamente, purché siano valori interi non negativi e coerenti con la direzione specificata. Per accedere a un singolo elemento del vettore si utilizza la notazione con parentesi tonde, indicando l'indice desiderato.

Nonostante la loro semplicità, i tipi `bit` e `bit_vector` presentano alcune limitazioni. In particolare, non consentono di rappresentare lo stato di *don't care* o quello di alta impedenza, che risultano invece fondamentali nella modellazione di circuiti digitali più complessi.

2.3.2 Tipi IEEE

La libreria `IEEE` introduce in VHDL una serie di tipi di dato avanzati, basati su un sistema logico a nove valori. Tali valori permettono una modellazione più realistica

dei segnali digitali, tenendo conto anche di condizioni non rappresentabili con il semplice tipo `bit`. I nove valori logici sono mostrati nella Tabella 2.1:

Valore	Descrizione
'0'	Valore logico basso
'1'	Valore logico alto
'Z'	Stato di alta impedenza
'X'	Valore indeterminato, che può corrispondere a 0 oppure 1
'U'	Valore non inizializzato, mai assegnato
'W'	Segnale debole non interpretabile come 0 o 1
'L'	Segnale debole interpretabile come 0
'H'	Segnale debole interpretabile come 1
'-'	Condizione di <i>don't care</i>

Tabella 2.1: Valori logici del tipo `std_logic`

Tra questi valori, quelli maggiormente utilizzati nella pratica progettuale, oltre ai livelli logici '0' e '1', sono lo stato di alta impedenza 'Z' e i simboli relativi allo stato di *don't care*, ovvero 'X' e '-', di cui il primo è generato automaticamente dai simulatori quando un segnale risulta indeterminato, mentre il secondo può essere solo assegnato dal progettista. Tale differenza, comunque, non influisce sul processo di sintesi.

Come tutte le librerie VHDL, anche la libreria IEEE è organizzata in *package* (concetto che approfondiremo in seguito), ciascuno dei quali mette a disposizione specifici elementi riutilizzabili. Il più rilevante è il package `std_logic_1164`, che definisce i tipi logici multi-valore e i corrispondenti tipi vettoriali, ampiamente utilizzati nella descrizione di sistemi digitali.

La libreria IEEE include inoltre ulteriori package, quali `std_logic_unsigned`, `std_logic_signed` e `std_logic_arith`, che estendono il comportamento degli operatori aritmetici e di confronto.

Anche per i tipi IEEE sono disponibili i vettori, come mostrato nella `entity` usata come esempio precedentemente. Sono quindi valide, per questi tipi, tutte le regole logiche ed implementative precedentemente spiegate per il tipo `bit`

2.3.3 Package `numeric_std` [1]

Il package `numeric_std`, definito nella libreria IEEE, non fa parte dello standard VHDL base. Esso è stato creato per fornire tipi di dati numerici standard e portabili, insieme a operazioni aritmetiche chiaramente definite, facilitando così la progettazione e la sintesi di circuiti numerici.

Il package introduce due nuovi tipi di dati:

- **unsigned**: array di `std_logic` che rappresenta valori interi non negativi;
- **signed**: array di `std_logic` che rappresenta valori interi con segno, codificati in complemento a due.

Per entrambi i tipi, il bit più a sinistra viene considerato il più significativo (MSB). Il package definisce inoltre un insieme di operazioni aritmetiche e logiche direttamente utilizzabili su questi tipi, tra cui, ad esempio, somma, sottrazione, confronto e operazioni di shift, come sarà possibile osservare nel Capitolo successivo.

2.3.4 Il tipo integer

Il tipo `integer` consente di rappresentare valori interi su 32 bit, con, tuttavia, alcune criticità.

In primo luogo, è necessario stabilire come interpretare il segno dei valori rappresentati: in assenza di specifiche diverse, essi vengono considerati come interi senza segno.

In secondo luogo, l'utilizzo di variabili o segnali di tipo `integer` può comportare un uso inefficiente delle risorse hardware. Infatti, gli strumenti di sintesi trattano tali segnali come bus a 32 bit, generando automaticamente connessioni e componenti dimensionati di conseguenza, anche quando ciò non sarebbe strettamente necessario.

2.3.5 Definizione di nuovi tipi

Un tipo definito dall'utente in VHDL è un nuovo tipo di dato le cui proprietà vengono stabilite direttamente dal progettista. I principali meccanismi per la definizione di tipi personalizzati sono il *subtyping* e i tipi enumerativi.

Il *subtyping* permette di derivare un nuovo tipo a partire da uno esistente, restringendone l'intervallo dei valori ammissibili. Questo approccio risulta particolarmente utile nel caso di tipi numerici, e in particolare del tipo `integer`. La sintassi generale è la seguente:

```
subtype new_type_name is type_name range val1 to val2;
```

Con tale dichiarazione, il nuovo tipo `new_type_name` mantiene le caratteristiche del tipo di partenza `type_name`, ma può assumere esclusivamente valori compresi tra `val1` e `val2`, permettendo un utilizzo ottimizzato delle risorse.

Si propone quindi un semplice esempio:

```
subtype small_int is integer range 0 to 15;
```

Il nuovo tipo `small_int` rappresenta un sottoinsieme del tipo `integer`, limitato ai valori compresi tra 0 e 15. Tale restrizione consente, nel caso di variabili con dominio ridotto, una chiara ottimizzazione delle risorse, allocando solo 4 bit per segnale invece di 32.

Un secondo approccio consiste nella definizione di tipi enumerativi, che permettono di specificare esplicitamente l'insieme dei valori simbolici che il tipo può assumere. La sintassi è la seguente:

```
type new_type_name is (val0, val1, ..., valN);
```

I valori enumerati non sono associati automaticamente a rappresentazioni numeriche né a una specifica codifica in bit; per questo motivo, il loro utilizzo richiede particolare attenzione.

Suddetta possibilità implementativa risulta particolarmente pratica nell'implementazione di macchine a stati finiti. Ad esempio, se si volesse realizzare una *FSM* relativa ad un semaforo stradale, si può definire il seguente tipo per definirne gli stati.

```
type state_type is (GREEN, YELLOW, RED);
```

2.4 Manipolazione dei segnali

2.4.1 Operatori di base

In VHDL, l'operatore `<=` viene utilizzato per assegnare un valore a un segnale, mentre l'operatore `=` permette di confrontare due segnali in modo da verificare che siano uguali. L'assegnamento, come già detto, è concorrente, e aggiorna il valore del segnale nella simulazione; mentre il confronto restituisce un valore booleano che indica se i due segnali sono uguali. Un'altra funzione di base di VHDL è la possibilità di formulare espressioni logiche, che possono essere costruite utilizzando gli operatori `and`, `or`, `not` e `xor`. La valutazione di tali espressioni avviene secondo le regole di precedenza e di associatività definite dall'algebra booleana. Tra i costrutti più elementari, uno dei più utili, dalla seguente sintassi:

```
signal_name <= (others => const);
```

permette assegnare lo stesso valore a tutti gli elementi di un vettore, indipendentemente dal fatto che la sua dimensione sia nota o meno. Tale espressione appena formulata indica che ogni elemento del vettore `signal_name` assumerà il valore `const`. Questo costrutto verrà ampiamente utilizzato nell'implementazione dei moduli della CPU, con l'obiettivo di inizializzare i valori dei segnali utilizzati e ridurre al minimo le ambiguità in fase di sintesi.

2.4.2 Slice e concatenamento

Una *slice* consiste in un sottoinsieme di un vettore, la cui selezione avviene mediante la sintassi:

```
signal_name(index1 {to | downto} index2)
```

dove `index1` e `index2` devono essere indici validi per il vettore `signal_name` e coerenti con l'ordinamento specificato tramite le parole chiave `to` o `downto`.

Di seguito è riportato un esempio pratico di questa operazione:

```

signal a : std_logic_vector(7 downto 0);
signal b : std_logic_vector(3 downto 0);
-- [...]
b <= a(3 downto 0);

```

In questo esempio, il vettore `b` viene assegnato con i 4 bit meno significativi del vettore `a`.

Il concatenamento rappresenta un'operazione complementare, in quanto consente di combinare più segnali in un'unica struttura vettoriale. In VHDL, tale operazione è realizzata mediante l'operatore `&`.

Si osservi la seguente applicazione, tenendo sempre in conto dei precedentemente definiti `a` e `b`:

```

signal c : std_logic_vector(7 downto 0);
-- [...]
c <= b & b;

```

Qui, l'operatore `&` viene utilizzato per concatenare due copie del vettore `b`, generando un nuovo vettore di dimensione doppia.

2.4.3 Assegnamento condizionale

VHDL offre dei costrutti per l'assegnamento condizionale di segnali. La forma più comune è la seguente:

```

signal_name <= const_1 when cond_1 else
               const_2 when cond_2 else
               ...
               const_N when cond_N else
               const_default;

```

In questo caso, il segnale `signal_name` assume il valore della costante corrispondente alla prima condizione vera (`cond_1`, ..., `cond_N`); se nessuna condizione è verificata, viene assegnato il valore di default `const_default`. Bisogna ricordare, tuttavia, che VHDL richiede che tutti i possibili casi siano esplicitamente gestiti.

Un'alternativa più compatta è il costrutto `with ... select`, dalla seguente struttura

```

with test_signal select
signal_name <= const_1 when case_1,
               const_2 when case_2,
               ...
               const_N when case_N,
               const_default when others;

```

Lo scopo è lo stesso: il valore di `test_signal` viene confrontato con ciascuna costante `case_1`, ..., `case_N` e il segnale assume la costante corrispondente. Se nessun caso corrisponde, viene assegnato il valore `const_default`. La parola chiave `others` rappresenta tutti i casi non esplicitamente specificati.

Uno dei più comuni casi di applicazioni dell'assegnamento condizionale consiste nella realizzazione dei multiplexer. Ad esempio, nel caso in cui si voglia realizzare un multiplexer 4×1 , dove `d0`, `d1`, `d2`, `d3` sono gli ingressi alle corrispondenti porte, `y` è l'uscita, e `sel` è il vettore di selezione di 2 bit, si può optare o per il primo costruito:

```
y <= d0 when sel = "00" else
      d1 when sel = "01" else
      d2 when sel = "10" else
      d3 when sel = "11";
```

O, invece, per il secondo:

```
with sel select
  y <= d0 when "00",
      d1 when "01",
      d2 when "10",
      d3 when "11";
```

2.5 VHDL per reti sequenziali: i processi

2.5.1 Definizione di un processo

I processi sono costrutti fondamentali per la progettazione di reti sequenziali, dove è necessario che determinati elementi vengano controllati da alcuni segnali, come, ad esempio, il classico *clock*.

Un *processo* in VHDL è uno statement concorrente a tutti gli assegnamenti e istanziazioni di componenti presenti. Tuttavia, esso rappresenta anche un costrutto composto, in quanto contiene al suo interno una sequenza di istruzioni.

A differenza degli altri statement concorrenti, le istruzioni presenti nel corpo di un processo vengono eseguite in modo sequenziale. Questo consente di descrivere comportamenti in cui l'ordine delle operazioni è rilevante e, proprio per questa peculiarità, vengono utilizzati anche per ottimizzare eventuali reti combinatorie.

La sintassi generale per la definizione di un processo è la seguente:

```
[process_name]: process (sensitivity_list)
  [declarations]
begin
  [body]
end process;
```

Un processo può essere identificato da un nome opzionale `process_name`, utile principalmente per migliorare la leggibilità del codice. Esso può includere una sezione dichiarativa (`declarations`), analoga a quella presente nelle *architecture*, e un corpo (`body`), che contiene le istruzioni da eseguire.

Elemento fondamentale è la *sensitivity list*, ovvero l'insieme dei segnali che determinano l'attivazione del processo: ogni volta che si verifica una variazione su uno di tali segnali, il processo viene eseguito; in assenza di eventi, esso rimane inattivo.

I costrutti sequenziali che possono essere utilizzati nel corpo di un processo sono i classici già conosciuti in ogni linguaggio di programmazione come espressioni ed assegnamenti, costrutti condizionali e cicli.

2.5.2 Statement if

Tra i principali costrutti condizionali del linguaggio VHDL è lo statement `if`, la cui forma generale è la seguente:

```
if condition_1 then
    statement_1
[elsif condition_2 then
    statement_2]
...
[else
    statement_N]
end if;
```

I rami `elsif` e `else` sono opzionali, e il numero di condizioni `elsif` può essere arbitrario. Tuttavia, analogamente a quanto avviene per gli assegnamenti condizionali, è necessario considerare esplicitamente tutti i possibili casi. Ciò implica, nella maggior parte delle situazioni, la presenza del ramo `else`.

Il comportamento è analogo a quello dei linguaggi di programmazione tradizionali: se la condizione è verificata, vengono eseguiti gli statement associati al ramo `then`; in caso contrario, vengono eseguiti quelli dei rami successivi, e così via.

Il costrutto spiegato viene usato spesso per realizzare operazioni sensibili ad un segnale di clock, come, ad esempio, la scrittura sincrona di un registro. Ad esempio, il registro di uscita del codificatore di Hamming "*ampliato*", ovvero quello illustrato in Figura 1.18, è implementato col seguente processo:

```
process(clk)
begin
    if rising_edge(clk) then
        output_reg <= p1 & p2 & m1 & p3 & m2 & m3 & m4;
    end if;
end process;
```

Il seguente processo, al fronte di salita del segnale di temporizzazione, aggiorna il valore del segnale `output_reg` con il concatenamento dei bit del messaggio e di parità secondo le regole del codice di Hamming.

2.5.3 Statement case

Il costrutto `case` rappresenta l'equivalente sequenziale del costrutto combinatorio `with-select`, offrendo tuttavia una maggiore flessibilità nella descrizione delle condizioni. La sua sintassi generale è la seguente:

```
case signal_name is
  when range_1 =>
    statement_1
  when range_2 =>
    statement_2
  ...
  when range_N =>
    statement_N
  when others =>
    statement_default
end case;
```

Il funzionamento è analogo a quello del costrutto `case` presente in molti linguaggi di programmazione: in base al valore di `signal_name`, viene eseguito il blocco di istruzioni associato alla condizione corrispondente. La clausola `others` consente di gestire tutti i casi non esplicitamente specificati.

Le condizioni associate alle clausole `when` possono essere espresse in forma compatta tramite insiemi di valori o intervalli, secondo la sintassi:

```
values [ | values ] ...
```

dove `values` può rappresentare una costante oppure un intervallo, applicabile sia a variabili di tipo `integer` sia a tipi enumerativi.

È importante che le condizioni associate ai diversi rami siano mutuamente esclusive. In particolare, nel caso di intervalli, eventuali sovrapposizioni costituiscono un errore e violano i vincoli richiesti per una corretta sintesi del circuito.

Si riporta un semplice esempio di utilizzo del seguente costrutto per implementare un decoder a 2 bit:

```
signal sel : std_logic_vector(1 downto 0);
signal y   : std_logic_vector(3 downto 0);
--[...]
process(sel)
begin
  case sel is
    when "00" =>
```

```

y <= "0001";
when "01" =>
y <= "0010";
when "10" =>
y <= "0100";
when "11" =>
y <= "1000";
when others =>
y <= "0000";
end case;
end process;

```

In questo esempio, il valore del segnale `sel` determina quale bit del vettore `y` viene attivato. La clausola `others` gestisce eventuali valori imprevisti, assegnando un vettore nullo come comportamento di default.

2.5.4 Statement for

Il costrutto `for` in VHDL consente di descrivere iterazioni, e sintassi generale è la seguente:

```

for identifier in range loop
    statements
end loop;

```

L'identificatore `identifier` rappresenta la variabile di controllo del ciclo e assume, ad ogni iterazione, i valori definiti dall'intervallo `range`. È importante sottolineare che tale identificatore non è un segnale, ma una variabile locale al ciclo, e non può quindi essere utilizzato in contesti che richiedono esplicitamente segnali.

L'intervallo di iterazione deve essere costante, ovvero i suoi estremi devono essere noti al momento della sintesi. Esso può essere specificato nelle seguenti forme:

```

first to last
last downto first
array_name'range

```

Le prime due modalità definiscono direttamente l'intervallo, mentre la terza utilizza l'attributo `range` di un vettore.

Per ciò che concerne l'indice, è possibile effettuare operazioni aritmetiche su questo, purché i valori risultino determinabili in fase di sintesi. Inoltre, questa variabile è di sola lettura: il linguaggio non consente di modificarne il valore all'interno del corpo del ciclo.

Si riporta un semplice esempio di utilizzo del costrutto `for` per invertire i bit di un vettore di segnali:

```

signal a : std_logic_vector(7 downto 0);
signal b : std_logic_vector(7 downto 0);
--[...]
process(a)
begin
  for i in a'range loop
    b(i) <= a(7 - i);
  end loop;
end process;

```

In questo esempio, la variabile *i* scorre tutti gli indici del vettore *a* con l'ausilio dell'attributo '*range*'), e, per ciascun indice, viene assegnato il bit corrispondente in ordine inverso al vettore *b*.

2.6 Descrizione strutturale

La descrizione strutturale in VHDL consiste nel rappresentare un sistema digitale come un insieme di componenti interconnessi. In questo approccio, l'attenzione non è posta sul comportamento del singolo modulo, ma sulla struttura complessiva del circuito e sulle connessioni tra i suoi elementi costitutivi, già definiti in precedenza nel progetto.

Per usare le *sotto-reti* già descritte, è necessario dichiararne esplicitamente il nome e l'interfaccia nella sezione dichiarativa dell'architecture della rete complessiva che ne farà uso. La dichiarazione di un componente segue la sintassi:

```

component component_name is
  [generic (generic_list);]
  port (port_list);
end component;

```

Questa struttura è analoga a quella di una *entity declaration*, con la differenza che viene utilizzata la parola chiave **component**.

L'utilizzo di un componente avviene tramite la sua istanziazione:

```

instance_name: component_name
  [generic map (generic_assignment_list);]
  port map (port_assignment_list);

```

Ogni istanza è identificata da un nome univoco (*instance_name*) e deve riferirsi a un componente precedentemente dichiarato.

Il costrutto **port map** consente di specificare le connessioni tra le porte del componente e i segnali dell'architecture. Tali associazioni possono essere espresse in forma posizionale:

```

port map (signal_1, ..., signal_N);

```

Oppure in forma nominale:

```
port map (port_1 => signal_1, ..., port_N => signal_N);
```

Nel primo caso l'associazione avviene in base all'ordine, mentre nel secondo caso è esplicitata tramite i nomi delle porte, migliorando la leggibilità del codice.

La descrizione strutturale risulta fondamentale per la realizzazione della CPU nel capitolo successivo, permettendo di rappresentare il sistema modulo per modulo e di evidenziare chiaramente le connessioni tra le componenti.

2.7 Package

Come già parzialmente anticipato, un *package* è un contenitore logico che raccoglie elementi correlati tra loro, come tipi, costanti e sottoprogrammi. Oltre ai precedentemente citati *package* standard forniti dal linguaggio, è possibile crearne di personalizzati, ad esempio per definire tipi di dato specifici, costanti di progetto o sottoprogrammi ricorrenti, favorendo la modularità e la manutenzione del codice.

Un package può essere composto da due parti: la *package declaration* e la *package body*. La *package declaration* definisce l'interfaccia del package, ovvero gli elementi accessibili dall'esterno, mentre la *package body* contiene le implementazioni dei sottoprogrammi o i dettagli dei valori delle costanti; quest'ultima può essere omessa se non sono presenti dettagli da implementare.

La sintassi generale per la dichiarazione di un package è la seguente²:

```
package identifier is
    package_declarative_part
end [package_simple_name];
```

La sezione `package_declarative_part` raccoglie le dichiarazioni di tutti gli elementi visibili agli utilizzatori del package. Questi elementi possono essere utilizzati successivamente in due modi principali:

- Antepoendo al nome dell'elemento il nome del package.
- Importando l'intero package all'inizio del programma o del modulo tramite la clausola `use`, che rende direttamente accessibili tutti gli elementi del suddetto.

In entrambi i casi, gli elementi dichiarati nel package sono disponibili sia all'interno della *package body* sia nelle altre unità che utilizzano il package (parte dello stesso progetto di quest'ultimo), semplificando la gestione di costanti, tipi e sottoprogrammi comuni.

²Tratto da [10]

Capitolo 3

DESIGN DEL SOFT-CORE RISC-V

3.1 Architettura RISC-V

3.1.1 Introduzione

L'architettura di un calcolatore rappresenta il modello astratto del sistema accessibile al programmatore, definita principalmente dall'insieme delle istruzioni (*instruction set*) che la descrivono (e con cui vengono sviluppati tutti i programmi eseguibili su una determinata piattaforma) e dalle modalità di accesso agli operandi.

Nel caso dell'architettura RISC-V, ogni istruzione è rappresentata come una *word* di 32 bit. I microprocessori, che sono sistemi digitali, leggono ed eseguono direttamente queste istruzioni in linguaggio macchina (ergo in binario); tuttavia, per facilitarne la comprensione da parte degli esseri umani, esse vengono spesso espresse tramite un linguaggio simbolico noto come *assembly*. È importante sottolineare che l'architettura non definisce l'implementazione hardware sottostante: differenti realizzazioni fisiche possono condividere la stessa architettura. L'organizzazione concreta dei componenti interni, come registri, memorie e unità aritmetico-logiche, prende il nome di microarchitettura.

RISC-V, introdotta nel 2010 presso l'Università della California, si distingue per la sua natura *open-source*, che ne consente l'utilizzo libero e la diffusione su larga scala. Nonostante ciò, essa offre funzionalità comparabili a quelle delle architetture commerciali più diffuse, rendendola una soluzione moderna e flessibile nel panorama dell'ingegneria dei calcolatori.

3.1.2 Assembly e linguaggio macchina

Come già anticipato, il linguaggio assembly rappresenta una forma leggibile dall'essere umano del linguaggio nativo del calcolatore. Ogni istruzione assembly specifica sia l'operazione da eseguire sia gli operandi su cui essa agisce. Per rendere più chiara la struttura generale di un'istruzione, si consideri l'esempio fittizio `add a, b, c`: il primo elemento, `add`, è detto mnemonico e indica l'operazione

da eseguire; l'operazione viene applicata agli operandi sorgente **b** e **c**, mentre il risultato viene memorizzato nell'operando destinazione **a**.

Un programma scritto in linguaggio assembly viene successivamente tradotto dai mnemonici a una rappresentazione composta esclusivamente da cifre binarie, detta linguaggio macchina. Nel caso dell'architettura RISC-V, tale traduzione segue una struttura ben definita: le istruzioni sono organizzate in quattro principali formati, denominati *R-type*, *I-type*, *S/B-type* e *U/J-type*. Nei paragrafi successivi verrà approfondita la struttura di ciascuno di questi formati.

3.1.3 Registri, memorie ed immediati

3.1.3.1 Introduzione

Come già visto, ogni istruzione opera su uno o più operandi, che devono essere reperiti da specifiche locazioni fisiche del sistema. Gli operandi possono risiedere nei registri, in memoria oppure essere costanti codificate direttamente nell'istruzione. Queste differenze nascono dal compromesso tra velocità e capacità: i registri e gli immediati godono di un accesso molto rapido ma sono limitati in numero e dimensione, mentre la memoria offre grande capacità a fronte di tempi di accesso maggiori.

3.1.3.2 Registri

Per garantire alte prestazioni, le istruzioni accedono principalmente ai registri, che costituiscono una piccola memoria interna molto veloce detta *register file*. RISC-V dispone di 32 registri generali, numerati da 0 a 31, ciascuno dei quali ha anche un nome simbolico che ne riflette l'uso convenzionale. La Tabella 3.1 illustra i 32 registri disponibili, spiegandone brevemente il ruolo.

Nome	Registro	Utilizzo
zero	x0	Valore costante pari al vettore nullo
ra	x1	Indirizzo di ritorno
sp	x2	Puntatore allo stack
gp	x3	Puntatore globale
tp	x4	Puntatore al thread
t0–t2	x5–x7	Registri temporanei
s0/fp	x8	Registro preservato / Puntatore al frame
s1	x9	Registro preservato
a0–a1	x10–x11	Argomenti della funzione / Valori di ritorno
a2–a7	x12–x17	Argomenti della funzione
s2–s11	x18–x27	Registri preservati
t3–t6	x28–x31	Registri temporanei

Tabella 3.1: Registri dell'architettura RISC-V e loro utilizzo [2]

Il registro `x0` (zero) contiene sempre il valore costante 0. I registri `s0-s11` e `t0-t6` sono tipicamente utilizzati per memorizzare variabili temporanee o locali. I registri `a0-a7` sono impiegati per il passaggio degli argomenti e dei valori di ritorno nelle chiamate a funzione, mentre il registro `ra` contiene l'indirizzo dell'istruzione a cui il programma deve tornare al termine della funzione. Inoltre, `sp` (stack pointer) punta alla cima dello stack (area della memoria che vedremo in seguito), `gp` (global pointer) è utilizzato per accedere a variabili globali, e `tp` (thread pointer) è impiegato in contesti multithread.

3.1.3.3 Immediati

Oltre agli operandi contenuti nei registri, le istruzioni RISC-V possono utilizzare costanti, dette *immediati*, codificate direttamente all'interno dell'istruzione. Gli immediati possono essere espressi, nel linguaggio assembly, in formato decimale, esadecimale (prefisso `0x`) o binario (prefisso `0b`). Dal punto di vista hardware, poichè le istruzioni dedicano a questi solo una loro parte dei 32 bit disponibili, gli immediati sono rappresentati solo parzialmente e devono quindi essere adattati alla dimensione della *word*. Inoltre, la posizione ed il numero dei bit che compongono l'immediato dipendono dal tipo di istruzione. Sarà quindi necessaria una ricostruzione ed estensione del valore presente nell'istruzione durante la fase di decodifica, come vedremo dettagliatamente in seguito.

3.1.3.4 Memoria

La memoria rappresenta lo spazio principale per la memorizzazione dei dati ed è caratterizzata da grande capacità ma tempi di accesso più elevati rispetto ai registri. Nell'architettura RISC-V, le istruzioni operano esclusivamente sui registri: pertanto, i dati in memoria devono essere prima caricati nei registri e successivamente eventualmente riscritti in memoria.

La memoria è organizzata come un array di byte indirizzabili singolarmente (*byte-addressable*). In RV32I, gli indirizzi sono a 32 bit e una parola è composta da 4 byte consecutivi: di conseguenza, due word contigue avranno indirizzi che differiscono di 4 bit.

Una porzione logica fondamentale della memoria è lo *stack*, utilizzato come area temporanea per il salvataggio di variabili locali e dati intermedi durante l'esecuzione delle funzioni. Lo stack segue una politica *Last-In First-Out* (LIFO) e cresce verso indirizzi di memoria decrescenti. Il registro `sp` contiene l'indirizzo della cima dello stack e viene aggiornato dinamicamente per allocare o deallocare spazio. Tra i principali utilizzi dello stack vi è il salvataggio e il ripristino dei registri durante le chiamate a funzione.

3.1.4 Tipologie di istruzioni

3.1.4.1 R-Type

Le istruzioni di tipo R (register-type) utilizzano tre registri come operandi: due sorgenti e uno destinazione. Come mostrato in Figura 3.1, il formato dell'istruzione è lungo 32 bit ed è suddiviso in sei campi: **funct7**, **rs2**, **rs1**, **funct3**, **rd** e **op**.

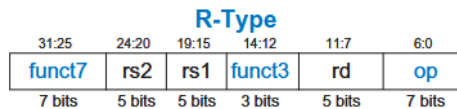


Figura 3.1: Formato delle istruzioni di tipo R [2]

L'operazione eseguita è codificata nei campi **op**, **funct7** e **funct3**, detti anche bit di controllo, mentre i registri sorgente e destinazione sono specificati rispettivamente nei campi **rs1**, **rs2** e **rd**. Rientrano in questa tipologia numerose istruzioni logico-aritmetiche.

3.1.4.2 I-Type

Le istruzioni di tipo I (immediate) utilizzano due registri e un operando immediato. Tra queste rientrano istruzioni aritmetico-logiche (come **addi**, **andi**, **ori**, **xori**), istruzioni di **load** e salti indiretti (**jalr**). Come illustrato in Figura 3.2, il formato è simile a quello R-type, ma include un campo immediato a 12 bit (**imm**) al posto di **funct7** e **rs2**.

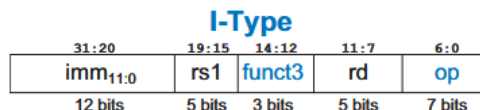


Figura 3.2: Formato delle istruzioni di tipo I [2]

Gli operandi sorgente sono **rs1** e l'immediato, mentre **rd** è il registro destinazione. L'immediato è generalmente un numero con segno rappresentato in complemento a due ed esteso tramite *sign extension*. Fanno eccezione le istruzioni di shift immediato: i bit meno significativi (**imm**[4:0]) rappresentano la quantità (ovviamente senza segno) di shift, mentre i bit più significativi codificano l'operazione e, nel caso dello shift a destra, permettono di distinguere tra shift logico e aritmetico¹.

¹Nella Tabella 3.3, questa particolarità è segnalata dal simbolo '**'

3.1.4.3 S/B-Type

Le istruzioni di tipo S e B utilizzano due registri sorgente (**rs1** e **rs2**) e un operando immediato. Come mostrato in Figura 3.3, il campo immediato è suddiviso in due parti non contigue all'interno dell'istruzione.

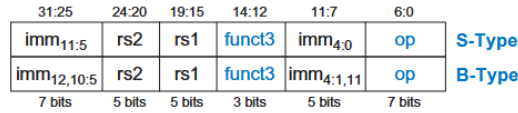


Figura 3.3: Formato delle istruzioni di tipo S/B [2]

Le istruzioni di store utilizzano il formato S-type, mentre le istruzioni di branch utilizzano il formato B-type. Nei formati S-type, l'immediato è un valore a 12 bit con segno, distribuito tra i bit più significativi e meno significativi dell'istruzione.

Nel formato B-type, l'immediato rappresenta l'offset di salto (branch offset) ed è codificato in modo non lineare (caratteristica chiamata *bit swizzling*). Il bit meno significativo è sempre 0, poiché gli indirizzi delle word sono multipli di 4^2 . L'indirizzo di destinazione del branch si ottiene sommando l'offset (esteso con segno) all'indirizzo dell'istruzione corrente.

3.1.4.4 U/J-Type

Le istruzioni di tipo U e J prevedono un solo registro destinazione (**rd**) e un campo immediato a 20 bit. Come illustrato in Figura 3.4, nel formato U-type l'immediato rappresenta i 20 bit più significativi di un valore a 32 bit.

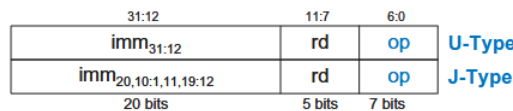


Figura 3.4: Formato delle istruzioni di tipo U/J [2]

Nel formato J-type, invece, l'immediato rappresenta un offset di salto a 21 bit (di cui uno implicito), utilizzato per modificare il flusso di esecuzione. Analogamente al caso B-type, il bit meno significativo non è codificato poiché sempre nullo.

3.1.4.5 Elenco delle istruzioni

La Tabella 3.2 mostra, nel dettaglio, tutte le istruzioni del set RV32I, ovvero il *sub-set* principale del RISC-V che comprende tutte le istruzioni a 32 bit eseguibili con valori interi. In Tabella 3.3, invece, sono spiegate le sigle utilizzate nell'elenco.

²Non si elimina anche il penultimo per la compatibilità con l'istruzione set a 16 bit.

op	funct3	funct7	Tipo	Istruzione	Descrizione	Operazione
0000011 (3)	000	—	I	lb rd, imm(rs1)	carica byte da memoria a registro	rd = SignExt([Address]7:0)
0000011 (3)	001	—	I	lh rd, imm(rs1)	carica half-word da memoria a registro	rd = SignExt([Address]15:0)
0000011 (3)	010	—	I	lw rd, imm(rs1)	carica word da memoria a registro	rd = [Address]31:0
0000011 (3)	100	—	I	lbu rd, imm(rs1)	carica byte senza segno da memoria a registro	rd = ZeroExt([Address]7:0)
0000011 (3)	101	—	I	lhu rd, imm(rs1)	carica half-word senza segno da memoria a registro	rd = ZeroExt([Address]15:0)
0010011 (19)	000	—	I	addi rd, rs1, imm	addizione con immediato	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift logico a sinistra con immediato	rd = rs1 ◀ uimm
0010011 (19)	010	—	I	slti rd, rs1, imm	set se minore di immediato	rd = (rs1 < SignExt(imm))
0010011 (19)	011	—	I	sltiu rd, rs1, imm	set se minore di immediato senza segno	rd = (rs1 < SignExt(imm))
0010011 (19)	100	—	I	xori rd, rs1, imm	XOR con immediato	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srli rd, rs1, uimm	shift logico a destra con immediato	rd = rs1 › uimm
0010011 (19)	101	0100000*	I	srai rd, rs1, uimm	shift aritmetico a destra con immediato	rd = rs1 ›› uimm
0010011 (19)	110	—	I	ori rd, rs1, imm	OR con immediato	rd = rs1 SignExt(imm)
0010011 (19)	111	—	I	andi rd, rs1, imm	AND con immediato	rd = rs1 & SignExt(imm)
0010111 (23)	—	—	U	auipc rd, upimm	somma immediato alto con il PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	—	S	sb rs2, imm(rs1)	salva byte in memoria	[Address]7:0 = rs27:0
0100011 (35)	001	—	S	sh rs2, imm(rs1)	salva half-word in memoria	[Address]15:0 = rs215:0
0100011 (35)	010	—	S	sw rs2, imm(rs1)	salva word in memoria	[Address]31:0 = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	addizione	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sottrazione	rd = rs1 - rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift logico a sinistra	rd = rs1 ◀ rs24:0
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set se minore di	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set se minore senza segno	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	XOR	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl rd, rs1, rs2	shift logico a destra	rd = rs1 › rs24:0
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift aritmetico a destra	rd = rs1 ›› rs24:0
0110011 (51)	110	0000000	R	or rd, rs1, rs2	OR	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	AND	rd = rs1 & rs2
0110111 (55)	—	—	U	lui rd, upimm	carica immediato alto	rd = {upimm, 12'b0}
1100011 (99)	000	—	B	beq rs1, rs2, label	branch se uguali	if (rs1 == rs2) PC = BTA
1100011 (99)	001	—	B	bne rs1, rs2, label	branch se diversi	if (rs1 != rs2) PC = BTA
1100011 (99)	100	—	B	blt rs1, rs2, label	branch se minore	if (rs1 < rs2) PC = BTA
1100011 (99)	101	—	B	bge rs1, rs2, label	branch se maggiore o uguale	if (rs1 >= rs2) PC = BTA
1100011 (99)	110	—	B	bltu rs1, rs2, label	branch se minore senza segno	if (rs1 < rs2) PC = BTA
1100011 (99)	111	—	B	bgeu rs1, rs2, label	branch se maggiore o uguale senza segno	if (rs1 >= rs2) PC = BTA
1101011 (103)	000	—	I	jalr rd, rs1, imm	salto indiretto con link	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	—	—	J	jal rd, label	salto con link	PC = JTA, rd = PC + 4

Tabella 3.2: Istruzioni del sub-set RV32I [2]

3.2 Microarchitettura del soft-core

3.2.1 Introduzione

La seguente sezione si pone l'obiettivo di illustrare l'implementazione della microarchitettura del core, spiegandone le scelte ed il ruolo di ogni componente. Le microarchitetture di base si dividono in tre grandi categorie:

1. La microarchitettura a ciclo singolo esegue un'intera istruzione in un unico ciclo di clock. Essa è concettualmente semplice e presenta un'unità di controllo di facile implementazione. Tuttavia, la frequenza massima di clock è vincolata dall'istruzione più lenta, riducendo le prestazioni complessive. Inoltre, richiede memorie separate per istruzioni e dati, una soluzione spesso poco realistica nei sistemi moderni.
2. La microarchitettura multiciclo suddivide l'esecuzione delle istruzioni in più cicli di clock più brevi. Le istruzioni semplici richiedono meno cicli rispetto a quelle più complesse, migliorando l'efficienza complessiva. È inoltre sufficiente una singola memoria, utilizzata in cicli distinti per il fetch dell'istruzione e per l'accesso ai dati. Per questo motivo, tali architetture sono state storicamente utilizzate in sistemi a basso costo.
3. La microarchitettura pipeline estende il modello a ciclo singolo introducendo l'esecuzione concorrente di più istruzioni. In questo modo, migliora significativamente il *throughput* del processore e permette, grazie a vincoli temporali

Sigla	Descrizione
imm	immediato con segno a 12 bit in <code>imm[11:0]</code>
uimm	immediato senza segno a 5 bit in <code>imm[4:0]</code>
upimm	20 bit più significativi di un immediato a 32 bit, in <code>imm[31:12]</code>
Address	indirizzo di memoria: <code>rs1 + SignExt(imm[11:0])</code>
[Address]	dato memorizzato all'indirizzo Address
BTA	indirizzo di destinazione del branch: <code>PC + SignExt({imm[12:1], 1'b0})</code>
JTA	indirizzo di destinazione del salto: <code>PC + SignExt({imm[20:1], 1'b0})</code>
label	etichetta che identifica un indirizzo di istruzione
SignExt	estensione del segno a 32 bit
ZeroExt	estensione con zeri a 32 bit

Tabella 3.3: Legenda delle sigle utilizzate nelle istruzioni descritte [2]

meno stringenti, maggiori frequenze di clock. Tuttavia, è necessario introdurre logiche aggiuntive per gestire le dipendenze tra istruzioni simultanee e registri di pipeline appositi. Nonostante questa complessità, i benefici prestazionali sono tali che tutte le moderne CPU ad alte prestazioni adottano tecniche di pipelining.

La seguente microarchitettura è del terzo tipo, ed è mostrata nel suo complesso in Figura 3.5. Nella seguente sezione, si procederà con l'illustrazione delle varie componenti di base, per poi approfondire successivamente l'implementazione delle tecniche di pipelining.

3.2.2 Componenti di base

3.2.2.1 ALU

La *Arithmetic Logi Unit* è quell'unità della CPU che si occupa di svolgere, come suggerisce il nome, tutte le operazioni aritmetiche e logiche previste dall'Instruction Set. Il seguente modulo presenta tre ingressi: **SrcA** e **SrcB** sono i due operandi di 32 bit, mentre **ALUControl** sarà invece il segnale di controllo utile per la selezione dell'operazione da svolgere. Più precisamente, in base al valore di questo vettore, verranno eseguite diverse operazioni:

1. "0000": addizione
2. "0001": sottrazione
3. "0010": and
4. "0011": or
5. "0100": xor
6. "0101": shift logico a sinistra
7. "0110": shift logico a destra

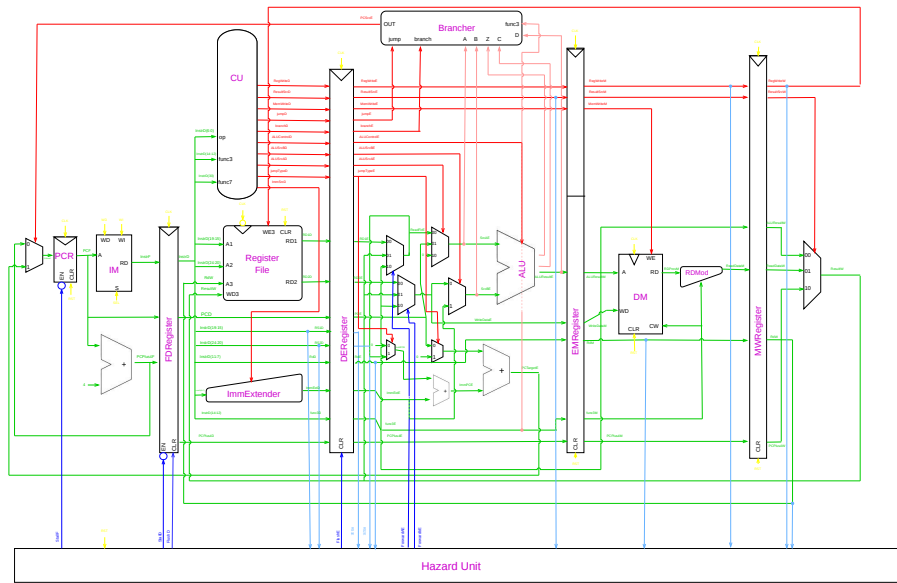


Figura 3.5: Microarchitettura del soft-processor. In nero le componenti; in giallo gli ingressi; in verde i segnali del datapath; in rosso i segnali di controllo; in blu i segnali per la gestione degli hazard.

8. "0111": shift aritmetico a destra
9. "1000": set less than
10. "1001": set less than (operandi senza segno)

In uscita, invece, è presente il risultato `ALUResult`, il flag `Zero` che segnala un eventuale risultato nullo ed il riporto/prestito `Cout`, generato per le sole addizioni e sottrazioni.

3.2.2.2 Control Unit

La CU (*Control Unit*) è quel modulo che si occupa di decodificare l'istruzione appena prelevata dall'Instruction Memory (che vedremo in seguito) e di generare, conseguentemente, tutti i segnali di controllo necessari per gestire il comportamento di tutte le altre componenti, il cui collegamento complessivo prende invece il nome di *datapath*. Questo modulo presenta tre ingressi:

1. `op`: vettore composto dai primi 7 bit dell'istruzione corrente.
2. `func3`: vettore composto dal 15esimo, 14esimo e 13esimo bit dell'istruzione corrente.
3. `func7`: segnale che memorizza il 31esimo bit dell'istruzione corrente.

In risposta, dopo aver trasformato in intero, memorizzandolo nel segnale intero ed interno `op_i`, il primo vettore in input, genera numerosi segnali di controllo in uscita (indicando il valore di `op_i`, per semplicità, `op`):

- **JumpType**, cambia il comportamento della componentistica relativa al calcolo dell'indirizzo dell'istruzione da eseguire se l'istruzione corrente è la particolare `jalr`:
 - '1' con `op = 103`, altrimenti '0'
- **ResultSrc**, definisce quale uscita del datapath dovrà essere eventualmente scritta all'interno del registro considerato:
 - "01" con `op = 3`
 - "10" con `op = (103, 111)`
 - "00" altrimenti
- **RegWrite**, abilita o meno la scrittura nel *Register File*:
 - '1' con `op = (3, 19, 23, 51, 55, 103, 111)`
 - '0' altrimenti
- **MemWrite**, abilita o meno la scrittura in memoria (valida solo per le istruzioni S-Type):
 - '1' con `op = 35`
 - '0' altrimenti
- **ALUSrcA**, definisce quale sarà il primo operando della ALU:
 - "01" con `op = 23`
 - "10" con `op = 55`
 - "00" altrimenti
- **ALUSrcB**, definisce quale sarà il secondo operando della ALU:
 - '1' con `op = (3, 19, 23, 35, 55)`
 - '0' altrimenti
- **ImmSrc**, definisce come codificare l'eventuale immediato:
 - "000" con `op = (3, 103)` oppure `op = 19` e `func3 = (000, 010, 011, 100, 110, 111)`
 - "001" con `op = 19` e `func3 = (001, 101)`
 - "010" con `op = 35`
 - "011" con `op = 99`

- "100" con `op` = 111
- "101" con `op` = (23, 55)
- `ALUOp`, segnale interno per scremare alcuni casi nel calcolo successivo del definitivo `ALUControl`:
 - "00" con `op` = (3, 23, 35, 55)
 - "01" con `op` = 99
 - "10" con `op` = (19, 51)
- `ALUControl`, definisce l'operazione da far eseguire alla ALU:
 - "0000" con `ALUOp` = "00" oppure (`ALUOp` = "10" e `func3` = "000" e `op[5]` = '0')
 - "0001" con `ALUOp` = "01"
 - "0000" con `ALUOp` = "10" e `func3` = "000" e `op[5]` = '1' e `func7` = '0'
 - "0001" con `ALUOp` = "10" e `func3` = "000" e `op[5]` = '1' e `func7` = '1'
 - "0101" con `ALUOp` = "10" e `func3` = "001"
 - "1000" con `ALUOp` = "10" e `func3` = "010"
 - "1001" con `ALUOp` = "10" e `func3` = "011"
 - "0100" con `ALUOp` = "10" e `func3` = "100"
 - "0110" con `ALUOp` = "10" e `func3` = "101" e `func7` = '0'
 - "0111" con `ALUOp` = "10" e `func3` = "101" e `func7` = '1'
 - "0011" con `ALUOp` = "10" e `func3` = "110"
 - "0010" con `ALUOp` = "10" e `func3` = "111"
- `Branch`, indica se si sta eseguendo un'istruzione di tipo B:
 - '1' con `op` = 99
 - '0' altrimenti
- `Jump`, indica se si sta eseguendo un'istruzione di tipo J:
 - '1' con `op` = (103, 111)
 - '0' altrimenti

Ovviamente, le brevi descrizioni illustrate per i suddetti output non risultano particolarmente significative. Illustrando le varie componenti del datapath sarà più chiaro il ruolo che, ognuno di questi segnali, assume nel progetto complessivo: ad esempio, abbiamo già illustrato come `ALUControl` gestisca la selezione dell'operazione di interesse.

3.2.2.3 Brancher

Questo modulo, che appartiene concettualmente alla CU ma viene separato per necessità legate alla pipeline, si occupa di gestire i salti condizionali e non all'interno della CPU. Riceve in ingresso gli operandi entranti nella ALU **A**, **B**, le sue uscite **D³**, **Zero**, **Cout**, le due flag prodotte dalla CU **Jump**, **Branch** ed il segnale **func3** già visto precedentemente. Prima della selezione del tipo di branch da eseguire, viene calcolato il segnale interno **Bs**, che segnala o meno se, con segno, il primo operatore è minore del secondo. Per le operazioni senza segno, tale verifica è effettuata attraverso **Cout** senza ulteriori calcoli. Per le verifiche di uguaglianza, infine, si può ricorrere alla semplice consultazione del segnale **Zero**. Successivamente, si effettua il calcolo del segnale interno **tbr** che, in base al tipo di branch (segnalato da **func3**), verifica se la condizione è soddisfatta o meno:

1. **tbr** = **Zero** quando **func3** = "000" (**beq**)
2. **tbr** = not **Zero** quando **func3** = "001" (**bneq**)
3. **tbr** = **Bs** quando **func3** = "100" (**blt**)
4. **tbr** = not **Bs** quando **func3** = "101" (**bge**)
5. **tbr** = not **Cout** quando **func3** = "110" (**bltu**)
6. **tbr** = **Cout** quando **func3** = "111" (**bgeu**)
7. **tbr** = '0' per tutti gli altri casi

Infine viene generata la flag **PCSrc**, che segnerà alla CPU se verrà effettuato il salto o meno: ciò avviene quando si ha un'istruzione di salto incondizionato oppure si ha un'istruzione di tipo B con condizione verificata (indicata da un valore alto di **tbr**).

3.2.2.4 Data Memory

Il seguente blocco di memoria viene utilizzato per memorizzare e riutilizzare i dati elaborati dal programma. Presenta come ingressi il segnale di clock **clk**, il segnale di reset **rst**, l'indirizzo **A**, la word **WD**, la flag **WE** (collegato al già visto **MemWrite**) ed il segnale di controllo **CW** direttamente collegato a **func3**. Come uscite, presenta il contenuto eventualmente letto **RD** ed il segnale **OutSign**, che vedremo in seguito. Tale memoria è asincrona in lettura (non viene controllata dal segnale di clock) ma sincrona in scrittura, ovvero sotto il controllo di un fronte di clock, in questo caso quello di salita. In particolare, su questo fronte:

1. Se il segnale di reset è alto, riporta a 0 tutto il contenuto della memoria

³Abbreviazione di "*Differenza*", in quanto le operazioni di branch eseguono sempre la suddetta operazione per effettuare le relative verifiche.

2. se **WE** è alto, avviene la scrittura nell'indirizzo indicato della word **WD**. In base al valore di **CW**, viene progressivamente scritto, partendo dal vettore nullo, il **LSB**, la half-word bassa o l'intera word; in modo da implementare correttamente le istruzioni **sb**, **sh**, **sw**.

3.2.2.5 Instruction Memory

Tale memoria viene utilizzata per memorizzare il codice macchina dei programmi da eseguire e, pertanto, la sua implementazione dipende fortemente dalle scelte realizzative su **FPGA**. Poichè lo scopo di questa sezione è quello di illustrare il comportamento di base del processore, ci limiteremo ad una trattazione incompleta del modulo, salvo poi chiuderla successivamente. Questa memoria asincrona riceve in ingresso un indirizzo (**A**, collegato direttamente al **PC**) e riporta in uscita il contenuto indicato dall'input.

3.2.2.6 Immediate Extender

Come abbiamo visto, un numero elevato di istruzioni fornisce al suo interno indicazioni sull'immediato da utilizzare: questo blocco ha lo scopo di decodificare i bit forniti dall'istruzione corrente per creare il valore numerico da usare nell'esecuzione. Riceve in ingresso i 25 bit superiori dell'istruzione (lunghezza massima in cui si può estendere un immediato codificato) ed il segnale di controllo generato dalla **CU ImmSrc**; in uscita invece restituisce l'immediato completo **ImmRes**. In base al valore di **ImmSrc**, avremo una diversa trasformazione dell'input.

- "000": $20\text{Instr}[31] + \text{Instr}[31:20]$ (tipo I ad eccezione degli shift)
- "001": $27('0') + \text{Instr}[24:20]$ (shift di tipo I)
- "010": $20\text{Instr}[31] + \text{Instr}[31:25] + \text{Instr}[11:7]$ (Tipo S)
- "011": $20\text{Instr}[31] + \text{Instr}[7] + \text{Instr}[30:25] + \text{Instr}[11:8] + ('0')$ (Tipo B)
- "100": $12\text{Instr}[31] + \text{Instr}[19:12] + \text{Instr}[20] + \text{Instr}[30:21] + ('0')$ (Tipo J)
- "101": $\text{Instr}[31:12] + 12('0')$ (Tipo U)

3.2.2.7 Read-Data Modifier

Questo modulo serve per modificare l'uscita della Data Memory in base al tipo di istruzione (**lb**, **lh**, **lw**, **lbu**, **lhu**), indicata, come al solito, da **func3**. In base a quest'ultimo segnale (memorizzato in **RDCtrl**) viene conservato, progressivamente, il **LSB**, la half-word bassa o l'intera word, estendendo eventualmente il risultato col segno o con lo zero in base al tipo di istruzione.

3.2.2.8 Register File

Questo blocco implementa il banco registri del processore, utilizzato per memorizzare temporaneamente gli operandi e i risultati delle istruzioni. La struttura contiene i 32 registri dell'architettura da (ovviamente) 32 bit ciascuno.

Presenta come ingressi il segnale di clock **CLK**, il segnale di reset **rst**, i tre indirizzi **A1**, **A2** e **A3**, il segnale di abilitazione alla scrittura **WE3** (direttamente collegato a **RegWrite**) e il dato da scrivere eventualmente **WD3**. Come uscite, restituisce i contenuti dei registri selezionati tramite **RD1** e **RD2**.

La lettura dei registri è asincrona: il contenuto dei registri indicati dagli indirizzi **A1** e **A2** viene immediatamente riportato in uscita senza attendere un fronte di clock.

La scrittura, invece, è sincrona sul fronte di discesa del clock (scelta legata alla pipeline, che verrà spiegata in seguito). In particolare, su tale fronte:

1. Se il segnale di reset è alto, tutto il contenuto del banco registri viene riportato a zero.
2. Se **WE3** è alto e l'indirizzo **A3** è diverso da quello nullo, il dato **WD3** viene scritto nel registro selezionato.

La condizione sull'indirizzo **A3** impedisce la scrittura nel registro **x0**, che nell'architettura RISC-V deve mantenere costantemente il valore nullo.

3.2.2.9 Program Counter Register

Questo modulo implementa il registro del Program Counter (**PC**), utilizzato per memorizzare l'indirizzo della prossima istruzione da eseguire.

Presenta come ingressi il segnale di clock **clk**, il valore del prossimo Program Counter **PcNext**, il segnale di abilitazione **en** (necessario per la pipeline) ed il segnale di reset **clr**. Come uscita, restituisce il valore corrente del Program Counter tramite il segnale **Pc**.

Il registro è sincrono rispetto al fronte di salita del clock. In particolare:

1. Se il segnale di reset **clr** è alto, il contenuto del Program Counter viene riportato a zero.
2. Se **en** è alto, il valore **PcNext** viene caricato nel registro.
3. In caso contrario, il contenuto del registro rimane invariato.

3.2.2.10 Componenti per il calcolo del PC

Numerosi componenti minori sono utilizzate per il corretto calcolo del valore prossimo del **PC**. Nell'E-State (vedremo prossimamente il significato di questo termine) sono presenti quattro blocchi dedicati:

1. Il primo multiplexer viene utilizzato per il calcolo del segnale `sumPC`: se `jumpType` è alto (solo nel caso di `jalr`), viene lasciato passare il valore corretto del primo registro dell'istruzione corrente; viceversa viene lasciato passare il vettore nullo.
2. Il secondo multiplexer, sempre a seconda del valore basso o alto della precedente flag, fa passare, rispettivamente, il valore del PC corrente o il vettore nullo.
3. Il primo sommatore effettua la somma tra `sumPC` ed il valore dell'immediato corrente.
4. Infine, un sommatore finale effettua la somma tra l'uscita del secondo multiplexer ed `ImmPC`, in uscita dal primo sommatore, restituendo `PCTarget`.

Questa circuiteria è necessaria per via della presenza dell'istruzione `jalr` che, a differenza delle altre, non lavora attraverso offset. Nel F-State, invece, sono presenti ulteriori componenti:

1. Viene calcolato `PCPlus4`, sommando, come suggerisce il nome, 4 al PC corrente.
2. Un multiplexer controllato da `PCSrc` assegna a `PCNext`, a seconda del valore del flag di controllo, rispettivamente `PCPlus4` o `PCTarget`.

3.2.2.11 Altre componenti

Nell'E-State sono presenti due multiplexer, necessari per la corretta selezione degli operandi della ALU.

1. Il primo, controllato da `ALUSrcA`, assegna a `SrcA`, progressivamente, il valore del primo registro, il PC od il vettore nullo.
2. Il secondo, controllato da `ALUSrcB`, assegna a `SrcB`, progressivamente, il secondo registro o l'immediato.

Infine, nel W-State, è presente un multiplexer, controllato da `ResultSrc`, che controlla il risultato del datapath, poi spedito al RF in `WD3`. In particolare, in base al segnale di controllo, viene assegnato progressivamente il valore in uscita dalla ALU, il valore in uscita dalla memoria od il valore incrementato di 4 del PC.

3.2.3 Pipeline e gestione degli hazard

3.2.3.1 Introduzione

La progettazione di un processore pipeline, come già parzialmente anticipato, si basa sulla suddivisione del datapath del processore single-cycle in più stadi consecutivi, consentendo l'esecuzione simultanea di più istruzioni. In particolare, l'architettura considerata adotta una pipeline a cinque stadi, permettendo l'elaborazione contemporanea di fino a cinque istruzioni, ciascuna in una diversa fase di esecuzione.

Poiché ogni stadio contiene soltanto una porzione della logica complessiva del processore, i vincoli di clock risultano ben inferiori rispetto al modello a singolo ciclo, permettendo una frequenza di clock maggiore ed un conseguente *throughput* migliore.

La pipeline è suddivisa nei seguenti cinque stadi:

1. *Fetch (F)*: viene prelevata dalla memoria l'istruzione puntata dal Program Counter.
2. *Decode (D)*: l'istruzione viene decodificata, vengono letti gli operandi dal Register File e vengono generati i segnali di controllo necessari all'esecuzione.
3. *Execute (E)*: l'ALU esegue l'operazione richiesta dall'istruzione.
4. *Memory (M)*: vengono effettuate eventuali operazioni di lettura o scrittura sulla memoria dati.
5. *Writeback (W)*: il risultato finale viene scritto nel Register File, qualora previsto dall'istruzione.

La divisione in 5 stadi viene effettuata mediante l'inserimento di registri di pipeline. In particolare, vengono introdotti quattro registri intermedi il cui scopo è memorizzare temporaneamente dati e segnali di controllo tra uno stadio e il successivo, garantendo che ciascun segnale rimanga sincronizzato con l'istruzione a cui appartiene durante tutte le fasi di esecuzione.

3.2.3.2 Definizione degli hazard

La sovrapposizione delle operazioni in processori di questa categoria introduce tuttavia possibili situazioni di conflitto, denominate *hazard*, che si verificano quando il corretto avanzamento di una istruzione dipende dal completamento di un'altra istruzione ancora in esecuzione.

Per gestire questi problemi, sono necessarie soluzioni da implementare nell'hardware: per esempio, come era già stato fatto notare, il Register File viene scritto sul fronte di discesa di clock e letto sul fronte di salita. Tale scelta consente ad una istruzione di leggere un valore appena scritto nello stesso ciclo di clock, evitando alcune dipendenze tra istruzioni consecutive.

Gli hazard si suddividono principalmente in due categorie:

- *Data Hazard*: si verifica quando una istruzione tenta di leggere un registro il cui valore non è stato ancora scritto da una istruzione precedente. In questo caso, l'istruzione potrebbe utilizzare un dato non aggiornato, compromettendo la correttezza dell'esecuzione.
- *Control Hazard*: si verifica quando il processore non ha ancora determinato correttamente quale sarà la prossima istruzione da prelevare dalla memoria al momento del fetch. Questa situazione si presenta tipicamente nelle istruzioni di branch.

Per gestire gli hazard, viene creato un modulo dedicato, chiamato *Hazard Unit*, che ha lo scopo di prelevare i segnali di interesse dal processore per produrre conseguentemente segnali di controllo volti a risolvere questi conflitti.

3.2.3.3 Gestione dei *Data Hazard*

Una parte dei data hazard può essere risolta mediante una tecnica denominata *forwarding*, che consiste nell'inoltrare direttamente il risultato prodotto da una istruzione verso una successiva che ne necessita.

In particolare, il forwarding consente di trasferire un risultato proveniente dagli stadi M o W direttamente allo stadio E, evitando che l'istruzione dipendente utilizzi dati non aggiornati. Per implementare questo meccanismo vengono introdotti opportuni multiplexer subito prima di quelli dedicati alla scelta degli operandi della ALU, in grado di selezionare, come valore effettivo dei due registri, sia i valori letti dal Register File sia i risultati provenienti dagli stadi successivi della pipeline.

La gestione del forwarding è affidata, come si può facilmente immaginare, alla *Hazard Unit*, che confronta i registri sorgente dell'istruzione presente nello stadio E, **Rs1E** e **Rs2E**, con i registri destinazione delle istruzioni presenti negli stadi M e W, indicati rispettivamente come **RdM** e **RdW** nel datapath.

L'unità riceve inoltre i segnali **RegWriteM** e **RegWriteW**, necessari per verificare se le istruzioni considerate produrranno effettivamente una scrittura nel Register File.

Quando il registro destinazione di una istruzione negli stadi successivi coincide con uno dei registri sorgente dell'istruzione nello stadio Execute, la Hazard Unit abilita il forwarding del dato corretto verso l'ALU.

È tuttavia necessario escludere il registro **x0** dal meccanismo di forwarding, poiché, come già detto, nell'architettura RISC-V esso è permanentemente vincolato al valore zero e non può essere modificato.

Nel caso in cui sia presente una corrispondenza sia nello stadio M sia nello stadio W, viene data priorità al primo, in quanto contiene il risultato prodotto dall'istruzione eseguita più recentemente.

Il meccanismo di forwarding è sufficiente a risolvere gran parte dei data hazard. Tuttavia, le istruzioni di caricamento da memoria, come **lw**, rappresentano un caso particolare. Infatti, il dato richiesto non viene reso disponibile fino al termine dello stadio M; di conseguenza, esso non può essere inoltrato in tempo allo stadio E dell'istruzione immediatamente successiva. Si dice quindi che l'istruzione **lw** (come tutte le altre della sua categoria) possiede una latenza di due cicli, poiché una istruzione dipendente può utilizzare il dato corretto solamente due cicli dopo il caricamento.

Per risolvere questo problema è necessario introdurre uno *stall* della pipeline, ovvero sospendere temporaneamente l'avanzamento delle istruzioni fino a quando il dato richiesto non diventa disponibile.

Durante uno stall viene introdotta nella pipeline una istruzione fittizia denominata *bubble*, il cui comportamento equivale a quello di una istruzione **nop** (istruzione nulla). La bubble viene generata azzerando i segnali di controllo dello

stadio Execute durante lo stall, in modo tale da non produrre alcun effetto sullo stato architetturale del processore.

In generale, lo stall di uno stadio viene realizzato disabilitando il corrispondente registro di pipeline, impedendo così l'aggiornamento dei suoi ingressi. Quando uno stadio viene bloccato, anche tutti gli stadi precedenti devono essere fermati, evitando la perdita delle istruzioni già presenti nella pipeline. Inoltre, il registro di pipeline immediatamente successivo allo stadio bloccato deve essere svuotato (*flush*) per impedire la propagazione di informazioni non valide.

Poiché gli stall degradano le prestazioni del processore, essi devono essere introdotti solamente quando strettamente necessari.

In particolare, la Hazard Unit introduce uno stall quando si verificano simultaneamente le seguenti condizioni:

1. una istruzione di caricamento da memoria si trova nello stadio E, condizione identificata tramite il segnale **ResultSrcE(0) = '1'**;
2. il registro destinazione dell'istruzione di load, **RdE**, coincide con uno dei registri sorgente dell'istruzione presente nello stadio Decode, ossia **Rs1D** oppure **Rs2D**.

Per supportare questo meccanismo vengono introdotti segnali di abilitazione (**en**) nei registri di pipeline degli stadi Fetch e Decode, oltre ad un segnale di clear sincrono (**CLR**) nel registro dello stadio Execute.

Quando si verifica una dipendenza di questo tipo, la Hazard Unit attiva i segnali **StallF** e **StallD**, bloccando rispettivamente gli stadi Fetch e Decode e mantenendo invariati i loro contenuti. Contemporaneamente viene attivato il segnale **FlushE**, che azzerà il registro dello stadio Execute introducendo la bubble nella pipeline.

La condizione complessiva di stall viene identificata tramite il segnale **lwStall**, generato dalla Hazard Unit ogni volta che è necessario sospendere temporaneamente l'avanzamento della pipeline a causa di una dipendenza da istruzioni di load.

3.2.3.4 Gestione dei Control Hazard

Come già parzialmente spiegato, i *control hazard* rappresentano un ulteriore problema nei processori pipeline e si verificano principalmente in presenza di istruzioni di branch o salto condizionato. In tali situazioni, il processore non conosce immediatamente quale sarà il prossimo valore del Program Counter e quindi quale istruzione dovrà essere prelevata nello stadio di Fetch.

Una possibile soluzione consiste nel bloccare temporaneamente la pipeline fino a quando non viene presa la decisione sul branch, ovvero fino al calcolo del segnale **PCSrcE**. Poiché tale decisione viene ottenuta solamente nello stadio E, questa tecnica introdurrebbe uno stall di due cicli per ogni branch eseguito. Considerando l'elevata frequenza delle istruzioni di salto nei programmi reali, questa soluzione comporterebbe una significativa degradazione delle prestazioni.

Per questo motivo, si preferisce utilizzare una tecnica di *branch prediction*. Nel processore considerato viene adottata una strategia molto semplice, che assume inizialmente che il branch non venga preso. Di conseguenza, il processore continua a prelevare ed eseguire sequenzialmente le istruzioni successive.

Successivamente, quando nello stadio Execute viene effettivamente calcolato il valore di PCSrcE, il processore verifica se la predizione effettuata era corretta. Se il branch risulta effettivamente preso, le istruzioni già caricate nella pipeline dopo il branch risultano errate e devono quindi essere eliminate.

Nel caso specifico della pipeline a cinque stadi considerata, quando il branch viene preso devono essere eliminate le due istruzioni presenti negli stadi Decode ed Execute attraverso la già introdotta tecnica di *flush*.

Per implementare il flush vengono introdotti ingressi di clear sincrono (CLR) nei registri di pipeline interessati e viene aggiunto il segnale FlushD alla Hazard Unit (FlushE, come già visto, è già stato introdotto). Quando il segnale PCSrcE viene posto a valore alto, indicando che il branch è stato preso, la Hazard Unit attiva simultaneamente i segnali FlushD e FlushE, azzerando i registri di pipeline degli stadi Decode ed Execute ed eliminando così le istruzioni non valide.

Anche in questo caso, il flush introduce una perdita prestazionale, poiché alcune istruzioni già parzialmente elaborate vengono scartate. Tuttavia, tale soluzione risulta significativamente più efficiente rispetto allo stall completo della pipeline ad ogni istruzione di branch.

3.2.3.5 Implementazione tecnica della Hazard Unit

Di seguito vengono riportate le condizioni logiche implementate all'interno della Hazard Unit per la gestione dei meccanismi di forwarding, stall e flush della pipeline, spiegate nel dettaglio nella precedente trattazione teorica.

- Forwarding per la risoluzione dei data hazard quando possibile:
 - Se $(Rs1E == RdM) \ \& \ RegWriteM \ \& \ (Rs1E \neq 0)$, allora:
 - * ForwardAE = "10"
 - Altrimenti, se $(Rs1E == RdW) \ \& \ RegWriteW \ \& \ (Rs1E \neq 0)$, allora:
 - * ForwardAE = "01"
 - Altrimenti:
 - * ForwardAE = "00"

In modo analogo viene generato il segnale ForwardBE, considerando, al posto di Rs1E, Rs2E.

- Stall quando si verifica un load hazard:
 - $lwStall = ResultSrcE0 \ \& \ ((Rs1D == RdE) \ | \ (Rs2D == RdE))$
 - StallF = lwStall

- StallD = lwStall
- Flush quando un branch viene preso oppure quando una load introduce una bubble:
 - FlushD = PCSrcE
 - FlushE = lwStall | PCSrcE

3.2.4 Architettura strutturale della CPU in VHDL

Nel presente capitolo si è scelto di omettere lo sviluppo dettagliato dei singoli moduli in VHDL, poiché esso risulta strettamente aderente alle descrizioni già fornite e non presenta elementi di particolare rilievo rispetto agli schemi illustrati nel capitolo precedente. Tale scelta consente inoltre di evitare un'eccessiva prolissità nell'esposizione.

Si ritiene tuttavia opportuno fornire al lettore una rappresentazione dell'architettura strutturale della CPU, così da chiarire in maniera più immediata il modo in cui le diverse componenti risultano interconnesse. Poiché non vengono riportate nel dettaglio le `entity` dei singoli moduli né i segnali interni impiegati, il seguente frammento di codice deve essere interpretato come uno schema puramente indicativo del progetto:

```
-- IMPLEMENTAZIONE DATAPATH E CONNESSIONE CON LA CONTROL UNIT

-- ALU
AU: ALU
  port map(
    SrcAE,
    SrcBE,
    ALUControlE,
    ALUResultE,
    ZeroE,
    CoutE
  );

-- Control Unit
CU: ControlUnit
  port map(
    InstrD(6 downto 0),
    InstrD(14 downto 12),
    InstrD(30),
    RegWriteD,
    MemWriteD,
    jumpTypeD,
    ResultSrcD,
```

```

        ALUSrcAD,
        ALUSrcBD,
        ImmSrcD,
        ALUControlD,
        JumpD,
        BranchD
    );

-- Data Memory
DM: DataMemory
    port map(
        clk,
        rst,
        AluResultM,
        WriteDataM,
        MemWriteM,
        func3M,
        RDprovM,
        outSig
    );

-- Instruction Memory
IM: InstructionMemory
    port map(
        PCF,
        sel,
        InstrF
    );

-- Immediate Extender
IE: ImmExt
    port map(
        InstrD(31 downto 7),
        ImmSrcD,
        ImmExtD
    );

-- Read Data Extender
RE: RDext
    port map(
        RDprovM,
        func3M,
        ReadDataM
    );

```

```

-- Register File
RF: RegisterFile
  port map(
    clk,
    rst,
    InstrD(19 downto 15),
    InstrD(24 downto 20),
    RdW,
    RegWriteW,
    ResultW,
    RD1D,
    RD2D
  );

-- PC Register
notStallF <= not(StallF);
PR: PcReg
  port map(
    clk,
    PCNextF,
    notStallF,
    rst,
    PCF
  );

-- FD Register
notStallD <= not(StallD);
FD: FDRegister
  port map(
    clk,
    notStallD,
    FlushD,
    InstrF,
    PCF,
    PCPlus4F,
    InstrD,
    PCD,
    PCPlus4D
  );

-- DE Register
DE: DERegister
  port map(
    clk,
    FlushE,

```

```

    RegWriteD,
    ResultSrcD,
    MemWriteD,
    JumpD,
    BranchD,
    ALUControlD,
    ALUSrcAD,
    ALUSrcBD,
    jumpTypeD,
    RD1D,
    RD2D,
    PCD,
    InstrD(19 downto 15),
    InstrD(24 downto 20),
    InstrD(11 downto 7),
    ImmExtD,
    InstrD(14 downto 12),
    PCPlus4D,
    RegWriteE,
    ResultSrcE,
    MemWriteE,
    JumpE,
    BranchE,
    ALUControlE,
    ALUSrcAE,
    ALUSrcBE,
    jumpTypeE,
    RD1E,
    RD2E,
    PCE,
    RS1E,
    RS2E,
    RdE,
    ImmExtE,
    func3E,
    PCPlus4E
);

```

```

-- EM Register
EM: EMRegister
port map(
    clk,
    rst,
    RegWriteE,
    ResultSrcE,

```

```

    MemWriteE,
    ALUResultE,
    WriteDataE,
    RdE,
    func3E,
    PCPlus4E,
    RegWriteM,
    ResultSrcM,
    MemWriteM,
    ALUResultM,
    WriteDataM,
    RdM,
    func3M,
    PCPlus4M
);

-- MW Register
MW: MWRegister
port map(
    clk,
    rst,
    RegWriteM,
    ResultSrcM,
    ALUResultM,
    ReadDataM,
    RdM,
    PCPlus4M,
    RegWriteW,
    ResultSrcW,
    ALUResultW,
    ReadDataW,
    RdW,
    PCPlus4W
);

-- Hazard Unit
HU: HazardUnit
port map(
    rst,
    InstrD(19 downto 15),
    InstrD(24 downto 20),
    RS1E,
    RS2E,
    RdE,
    RdM,

```

```

    RdW,
    RegWriteM,
    RegWriteW,
    ResultSrcE,
    PCSrcE,
    ForwardAE,
    ForwardBE,
    StallF,
    StallD,
    FlushD,
    FlushE
);

-- Brancher
BR: Brancher
port map(
    SrcAE(31),
    SrcBE(31),
    ALUResultE(31),
    ZeroE,
    CoutE,
    JumpE,
    BranchE,
    func3E,
    PCSrcE
);

-- Multiplexer per il PC
PCNextF <= PCTargetE
    when PCSrcE = '1'
    else PCPlus4F;

-- Incremento di 4 per il PC
PCplus4F <= std_logic_vector(unsigned(PCF) + 4);

-- Multiplexer per il forwarding del primo registro
ReadFirE <= ResultW
    when ForwardAE = "01" else
    ALUResultM
    when ForwardAE = "10" else
    RD1E;

-- Multiplexer per il forwarding del secondo registro
WriteDataE <= ResultW
    when ForwardBE = "01" else

```

```

    ALUResultM
    when ForwardBE = "10" else
    RD2E;

-- Multiplexer per la SrcA della ALU
SrcAE <= ReadFirE
    when ALUSrcAE = "00" else
    PCE
    when ALUSrcAE = "01" else
    (others => '0');

-- Multiplexer per la SrcB della ALU
SrcBE <= ImmExtE
    when ALUSrcbE = '1'
    else WriteDataE;

-- Multiplexer per la somma del PC
sumPCE <= ReadFirE
    when jumpTypeE = '1'
    else (others => '0');

-- Quantità da sommare al PC
ImmPCE <= std_logic_vector(
    unsigned(sumPCE) + unsigned(ImmExtE)
);

-- Target eventuale per il PC
PCTargetE <= ImmPcE
    when jumpTypeE = '1' else
    std_logic_vector(
        unsigned(ImmPCE) + unsigned(PCE)
    );

-- Multiplexer per il risultato finale
ResultW <= ReadDataW
    when ResultSrcW = "01" else
    PCPlus4W
    when ResultSrcW = "10" else
    ALUResultW;

```

3.3 Implementazione del soft-processor su FPGA

3.3.1 Inizializzazione dei segnali

Il passaggio dalla simulazione all'esecuzione di un programma su una board fisica presenta un maggior numero di difficoltà di quelle che ci si può attendere. In primis, è di vitale importanza inizializzare tutti i segnali compresi nel design: infatti, se in simulazione verranno considerati come dei *don't care* senza creare problemi, su FPGA portano inevitabilmente al non funzionamento della rete progettata.

3.3.2 Gestione delle memorie

Un'ulteriore necessità progettuale consiste nella definizione delle dimensioni e della struttura delle memorie utilizzate dal processore. Per risolvere questa problematica viene realizzato un package dedicato, denominato **memTypes**, contenente al proprio interno i tipi necessari alla rappresentazione delle memorie del sistema.

In particolare, vengono definiti:

- il tipo **mem_type**, utilizzato per rappresentare una memoria composta da 128 word da 32 bit, ed utilizzato per la Data Memory e l'Instruction Memory;
- il tipo **reg_array**, utilizzato per implementare il Register File del processore, costituito da 32 registri da 32 bit.

All'interno del package vengono inoltre definite cinque ROM differenti, denominate **ROM0_C**, **ROM1_C**, **ROM2_C**, **ROM3_C** e **ROM4_C**.

Ciascuna ROM contiene una sequenza di istruzioni macchina precaricate (e quindi 5 programmi), utilizzate per verificare il corretto funzionamento del processore e delle diverse componenti architetturali.

L'utilizzo di un package dedicato permette inoltre, come già illustrato nel precedente capitolo, di centralizzare la definizione delle memorie del sistema, semplificando la manutenzione e l'estendibilità del progetto.

A tal proposito, avevamo lasciato in sospeso la trattazione di alcune scelte implementative relative alla DM e alla IM:

1. Per quanto riguarda la prima, l'output **outSign** viene utilizzato per mandare in uscita il contenuto della memoria presente all'indirizzo 100. Come vedremo successivamente, tale segnale verrà anche utilizzato come output generale (indirettamente) della CPU.
2. Per quanto riguarda la seconda, viene introdotto un segnale di ingresso di tre bit **sel**, con cui sarà possibile selezionare progressivamente uno dei 5 programmi presenti.

Infine è presente un segnale globale di reset sincrono, collegato a tutti i registri e le memorie del design (fatta eccezione per la IM), che permette di cancellare tutte le tracce di qualsiasi esecuzione compiuta fino a quel momento. Questo

segnale viene passato anche alla HU in modo da considerarlo nel calcolo di `FlushD` e `FlushE`.

3.3.3 Implementazione della CPU

Come modulo, il processore presenta, come ingressi:

- Il segnale di clock (`clk`), utilizzato per tutti i blocchi sincroni e collegato ad uno dei segnali di temporizzazione presenti sulla board;
- Il segnale `switchon`, collegato ad un *slide switch* della scheda, che permette o meno il "*display*" del risultato;
- Il segnale `ctrl`, utilizzato per la selezione sicura del programma da eseguire e collegato ad uno dei push button presenti;

Come uscite, presenta invece 6 segnali da 7 bit ciascuno, collegati rispettivamente ai 6 display esadecimali messi a disposizione dalla scheda.

L'architettura del processore, oltre a presentare l'implementazione standard spiegata nel dettaglio precedentemente, presenta alcune sezioni dedicate all'implementazione su FPGA:

- Viene implementato un processo combinatorio controllato da `ctrl`: ogni volta che il bottone viene premuto, viene incrementato `sel` (da "000" a "100", per poi riportarsi al valore di partenza se viene superato suddetto limite) ed attivato il segnale di reset `rst`. Nel momento in cui viene rilasciato, `rst` viene riportato al valore basso.
- Viene implementato un processo sincrono per il display del risultato. Sul fronte di salita di clock, se `switchon` è alto, viene il segnale interno `outSign` collegato alla DM viene codificato, seguendo le regole presenti nel datasheet [6], nei segnali di uscita `num`, collegati ai pin dei display a 7 segmenti. Quando invece `switchon` è basso, i display vengono lasciati spenti: questo segnale di ingresso, in sostanza, funge da "*interruttore*" che permette, o meno, di mostrare il risultato.

Capitolo 4

TEST E VERIFICA DEL SOFT-PROCESSOR

4.1 Verifica del funzionamento della CPU

4.1.1 Introduzione

Come spiegato nella Sezione 1.1, il flusso di lavoro è diviso in diverse fasi: nel Capitolo precedente, è stato realizzato il design del processore, mentre la sintesi di quest'ultimo è stata fatta in automatico da Quartus durante la compilazione. Nella seguente sezione, si procederà quindi con la simulazione funzionale, l'analisi temporale e di potenza e la verifica del funzionamento del circuito su FPGA, secondo le implementazioni definite precedentemente.

4.1.2 Simulazione funzionale

Come già spiegato in precedenza, per effettuare una simulazione funzionale quanto più vicina alla realtà, sarà necessario scrivere una testbench. Come possibile implementazione di quest'ultima, si è scelta la seguente opzione: la CPU esegue indisturbata il primo programma caricato nell'Instruction Memory per 500 ns. Al concludersi di quest'ultimi, il segnale d'ingresso `ctrl` passa a '0' (bottone premuto) per 10 ns, per poi tornare successivamente al valore alto (bottone rilasciato) per altri 10 ns aggiuntivi ai 500 ns successivi: in questo modo, come spiegato nel precedente Capitolo, verranno eseguiti in ordine i 5 programmi caricati. Per quanto concerne quest'ultimi, sono stati realizzati in modo da contenere, complessivamente, tutte le istruzioni disponibili nell'Instruction Set: il primo è stato preso da un libro di testo [2] ed è mostrato in Figura 4.1; i successivi, invece, sono stati assemblati opportunamente con la toolchain RISC-V del noto compilatore GCC in ambiente Linux, di cui i codici sono invece i presenti.¹

¹L'istruzione `li` è una "pseudo-istruzione", ovvero rappresenta una o più particolari istruzioni standard messe insieme. Nel caso specifico, questa corrisponde ad un `addi` tra il registro `x0` e l'immediato presente.

#	RISC-V Assembly	Description	Address	Machine Code
main:	addi x2, x0, 5	# x2 = 5	0	00500113
	addi x3, x0, 12	# x3 = 12	4	00C00193
	addi x7, x3, -9	# x7 = (12 - 9) = 3	8	FF718393
	or x4, x7, x2	# x4 = (3 OR 5) = 7	C	0023E233
	and x5, x3, x4	# x5 = (12 AND 7) = 4	10	0041F2B3
	add x5, x5, x4	# x5 = 4 + 7 = 11	14	004282B3
	beq x5, x7, end	# shouldn't be taken	18	02728863
	slt x4, x3, x4	# x4 = (12 < 7) = 0	1C	0041A233
	beq x4, x0, around	# should be taken	20	00020463
	addi x5, x0, 0	# shouldn't execute	24	00000293
around:	slt x4, x7, x2	# x4 = (3 < 5) = 1	28	0023A233
	add x7, x4, x5	# x7 = (1 + 11) = 12	2C	005203B3
	sub x7, x7, x2	# x7 = (12 - 5) = 7	30	402383B3
	sw x7, 84(x3)	# [96] = 7	34	0471AA23
	lw x2, 96(x0)	# x2 = [96] = 7	38	06002103
	add x9, x2, x5	# x9 = (7 + 11) = 18	3C	005104B3
	jal x3, end	# jump to end, x3 = 0x44	40	008001EF
	addi x2, x0, 1	# shouldn't execute	44	00100113
end:	add x2, x2, x9	# x2 = (7 + 18) = 25	48	00910133
	sw x2, 0x20(x3)	# [100] = 25	4C	0221A023
done:	beq x2, x2, done	# infinite loop	50	00210063

Figura 4.1: Primo programma per il test della CPU [2].

#SECONDO PROGRAMMA

00000000 <_start>:

```

0: ffc10113      addi sp,sp,-4      # sp = sp - 4
4: 0fc00293      li t0,252        # t0 = 252
8: 00010337      lui t1,0x10       # t1 = 0x00010000
c: ffe30313      addi t1,t1,-2     # t1 = 0x0000FFFE
10: 00510023      sb t0,0(sp)        # Mem[sp] = 0xFC
14: 006110a3      sh t1,1(sp)        # Mem[sp+1] = 0xFFFE
18: 00010383      lb t2,0(sp)        # t2 = -4
1c: 00014e03      lbu t3,0(sp)       # t3 = 252
20: 00111e83      lh t4,1(sp)        # t4 = -2
24: 00115f03      lhu t5,1(sp)       # t5 = 65534
28: 00f00f93      li t6,15           # t6 = 15
2c: 004f9f93      slli t6,t6,0x4     # t6 = 240
30: 002fdf93      srli t6,t6,0x2     # t6 = 60
34: 401fdf93      srai t6,t6,0x1     # t6 = 30
38: 12345537      lui a0,0x12345     # a0 = 0x12345000
3c: 67850513      addi a0,a0,1656    # a0 = 0x12345678
40: 00004597      auipc a1,0x4       # a1 = PC + 0x4000
44: 10058593      addi a1,a1,256     # a1 = a1 + 256
48: 06400293      li t0,100          # t0 = 100
4c: 01f2a023      sw t6,0(t0)        # Mem[100] = 30
50: 00410113      addi sp,sp,4       # sp = sp + 4
54: 00000513      li a0,0            # a0 = 0
58: 05d00893      li a7,93           # a7 = 93

```

0000005c <loop>:

```

5c: 00210063      beq sp,sp,5c <loop> # loop infinito

```

```

#-----
#TERZO PROGRAMMA
0:  li t0,-5           # t0 = -5
4:  slti t1,t0,0       # t1 = 1
8:  sltiu t2,t0,10     # t2 = 0
c:  lui t3,0xb         # t3 = 0x0000B000
10: addi t3,t3,-1366   # t3 = 0x0000AAAA
14: not t4,t3          # t4 = 0xFFFF5555
18: ori t5,t3,255      # t5 = 0x0000AAFF
1c: lui a6,0x1         # a6 = 0x1000
20: addi a6,a6,-16     # a6 = 0xFF0
24: and t6,t3,a6       # t6 = 0x0AA0
28: li t0,100          # t0 = 100
2c: sw t3,0(t0)        # Mem[100] = 0xAAAA
30: li a0,0            # a0 = 0
34: li a7,93           # a7 = 93
38: beq sp,sp,38       # loop infinito

#-----
#QUARTO PROGRAMMA
0:  li t0,5            # t0 = 5
4:  li t1,10           # t1 = 10
8:  bne t0,t1,10       # salto se t0 != t1
10: blt t0,t1,18        # salto se t0 < t1
18: li t2,15           # t2 = 15
1c: bge t2,t1,24        # salto se t2 >= t1
24: li t3,-1           # t3 = -1
28: bltu t3,t0,30      # confronto unsigned
30: li t0,100          # t0 = 100
34: sw t3,0(t0)        # Mem[100] = t3
38: li a0,0            # a0 = 0
40: li a7,93           # a7 = 93
44: beq sp,sp,44       # loop infinito

#-----
#QUINTO PROGRAMMA
0:  li t0,1            # t0 = 1
4:  sll t1,t0,t0        # t1 = 2
8:  srl t2,t1,t0        # t2 = 1
c:  sra t3,t1,t0        # t3 = 1
10: li t4,-1           # t4 = -1

```

```

14: li t5,1          # t5 = 1
18: sltu t6,t4,t5    # t6 = 0
1c: xor t6,t4,t5     # t6 = 0xFFFFFFFF
20: auipc t0,0x0      # t0 = PC
24: addi t0,t0,16     # t0 = address store
28: jalr t0          # jump store
30: li t0,100        # t0 = 100
34: sw t6,0(t0)      # Mem[100] = t6
38: li a0,0          # a0 = 0
3c: li a7,93         # a7 = 93
40: beq sp,sp,40     # loop infinito

```

I seguenti programmi hanno tra loro due principali caratteristiche comuni:

1. Tutti si concludono con un loop infinito, che "*congela*" il flusso della CPU nel suo stato finale.
2. Tutti salvano uno dei loro registri all'interno dell'indirizzo 100 della Data Memory, corrispondente, come sappiamo, al segnale `outSign`, che viene poi "*tradotto*" per i display a sette segmenti.

Si può procedere quindi, con i passaggi spiegati nel primo Capitolo, con la compilazione del programma e l'apertura di Questa. Per visualizzare al meglio il funzionamento della CPU, si procede inserendo nella finestra delle Waveform i segnali interni relativi ad alcuni registri (si potrebbe inserirli tutti, ma per le immagini allegate, per una questione di ordine ed estetica, si è deciso di selezionare solo alcuni da esempio), al PC uscente dal suo registro e al segnale `outSig`. Dopo il classico *Restart*, si può procedere con la simulazione direttamente nel software ausiliario: per esempio, per comodità, possiamo effettuare ciò digitando, su terminale, il comando `run 3000 ns`, fermando la simulazione al valore temporale indicato. I risultati sono mostrati qui in ordine: si può osservare, confrontandoli con i codici commentati, che avviene una corretta esecuzione per ciascuno di essi, con la conseguente scrittura in `outSig` del valore atteso.

Infine, sarà possibile visualizzare il contenuto delle memorie alla conclusione della simulazione, ovvero dopo i 3000 ns: la Figura 4.8 mostra il contenuto del Register File, mentre la Figura 4.7 mostra invece il contenuto della Data Memory. Come ci si aspettava, si avranno i valori corrispondenti al primo programma, in quanto, come già spiegato, col concludersi del quinto, il segnale `sel` torna ciclicamente all'inizio della lista².

²Poichè per via della durata di ogni esecuzione e i brevi cambi di valore del segnale `ctrl` ogni programma impiega 520 ns, all'istante finale si ha, in esecuzione, il primo, seguendo la logica proposta precedentemente.

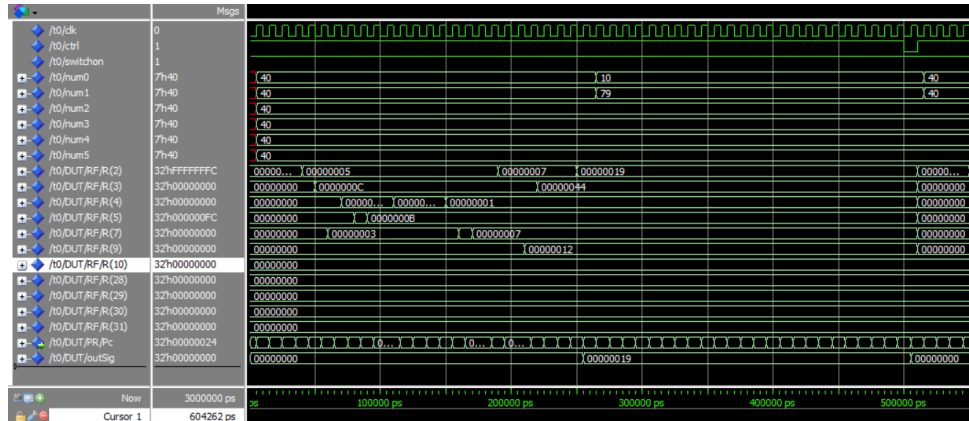


Figura 4.2: Waveform relative al test del primo programma.

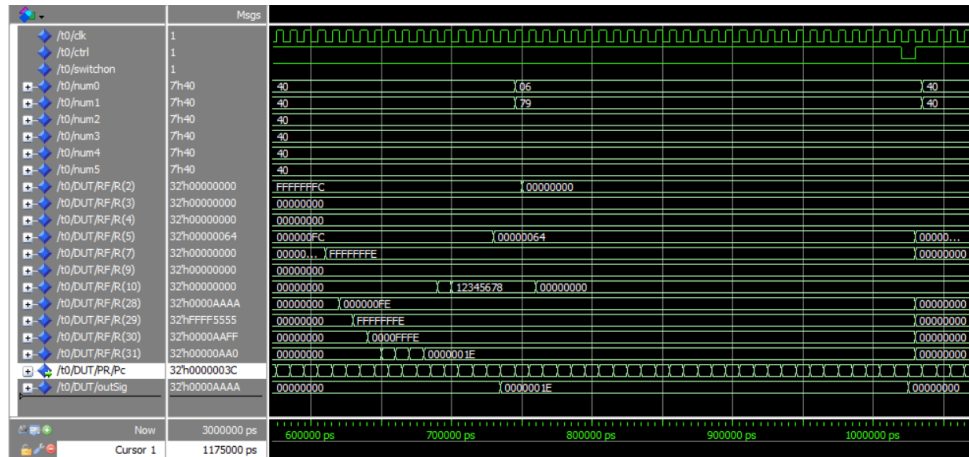


Figura 4.3: Waveform relative al test del secondo programma.

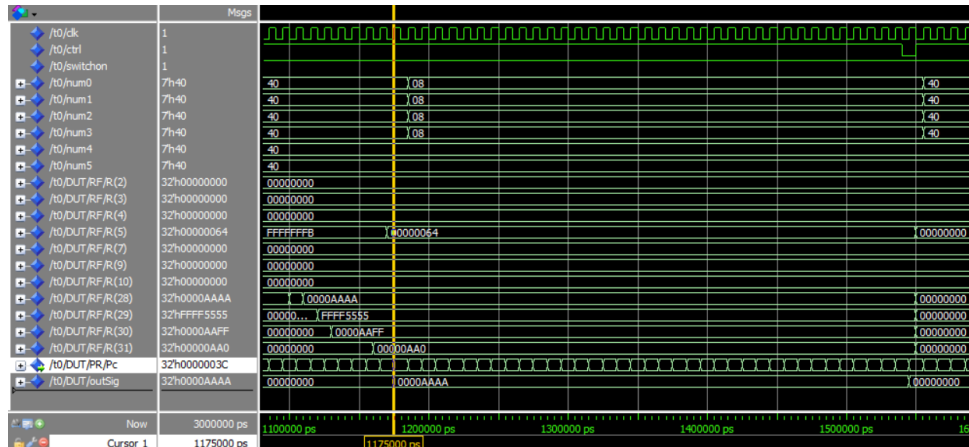


Figura 4.4: Waveform relative al test del terzo programma.

4.1.3 Analisi temporale

Come già sappiamo, dopo la prima compilazione viene effettuata un'analisi temporale con una frequenza di clock di default pari a 1000 MHz. Tuttavia, i clock

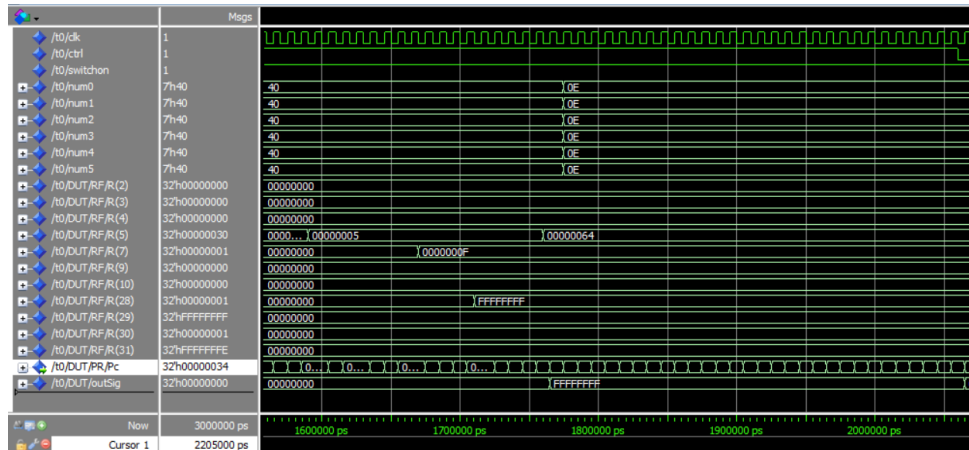


Figura 4.5: Waveform relative al test del quarto programma.

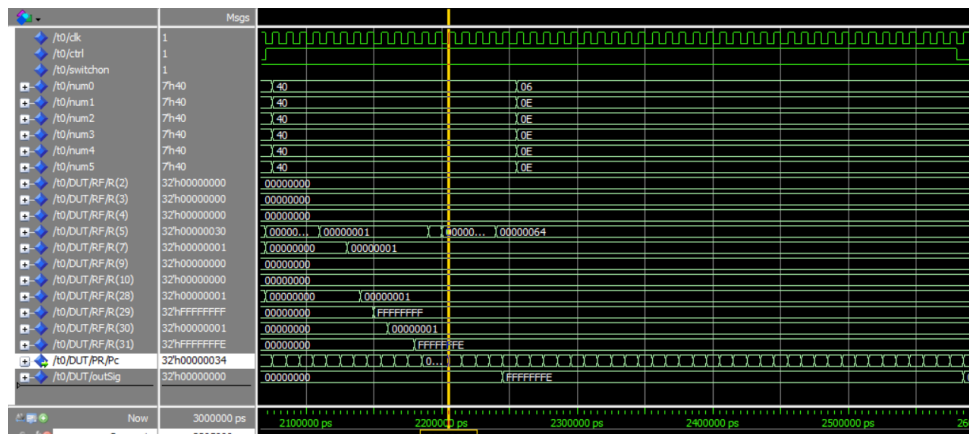


Figura 4.6: Waveform relative al test del quinto programma.

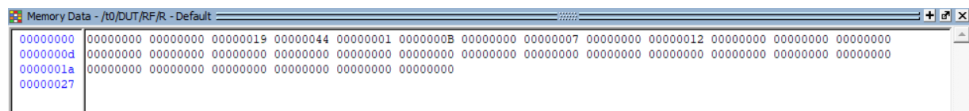


Figura 4.7: Contenuto del Register File al concludersi della simulazione.

messi a disposizione dalla scheda sono caratterizzati da una pari a 50 MHz: di conseguenza, l'obiettivo di questa analisi temporale sarà quello di verificare, con la procedura spiegata nel primo capitolo, se la nostra CPU è in grado di lavorare in modo stabile a questa frequenza, senza quindi richiedere l'implementazione di un "clock divider"³. Procediamo quindi all'apertura del "Timing Analyzer" e, con gli stessi identici passaggi già visti, assegniamo un periodo di 20 ns al segnale clk. Successivamente, si salva il file *.sdc creato, si assegna quest'ultimo al progetto e si ricompila il tutto. Infine, nel Report dedicato⁴, sarà possibile leggere i risultati

³Circuito utilizzato per ridurre di multipli di 2 la frequenza di un clock dato.

⁴In quest'ultimo, nel caso di inserimento di un file *.stp legato a *Signal Tap*, sarà presente anche un ulteriore clock chiamato *altera_reserved_tck*, ovvero il segnale di temporizzazione

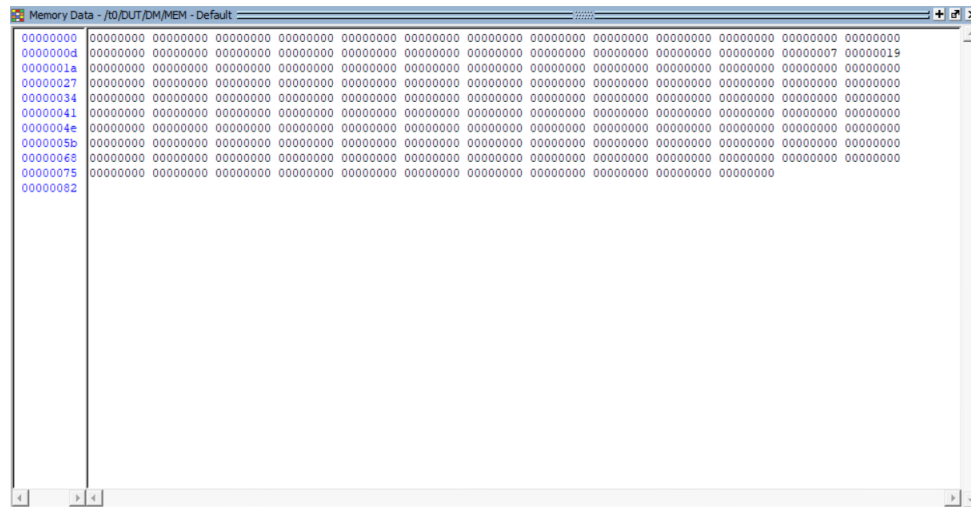


Figura 4.8: Contenuto della Data Memory al concludersi della simulazione.

ottenuti:

- La frequenza massima supportata dalla rete è pari a 64.41 MHz;
- Il Setup Slack è positivo, pari a 3.893 ns;
- L'Hold Slack è positivo, pari a 0.315 ns;
- Sia il Recovery Summary che il Removal Summary risultano vuoti, sintomo del fatto che tutti i segnali potenzialmente problematici (come i vari reset) sono sincronizzati.

In conclusione, gli slack positivi ci permettono di poter utilizzare in sicurezza il clock fornito dalla board, senza la necessità di ridurlo.

4.1.4 Analisi di potenza

Per avere un'analisi di potenza quanto più vicina alla realtà, procediamo, come già parzialmente anticipato, con la generazione di un file *.vcd a partire dalla testbench simulata: dopo aver concluso la simulazione secondo i passaggi indicati, digitare nel terminale il comando `vcd file prova1.vcd`. Successivamente, si può chiudere Questa ed aprire i *Settings* del progetto come abbiamo già fatto più volte: nella sezione dedicata al *Power Analyzer*, si abilita il tool in questione durante la compilazione e l'utilizzo dei file per l'inizializzazione (le prime due voci, come mostrato in Figura 4.9); infine, attraverso la voce *Add*, si inserisce il file appena creato.

Si procede con la compilazione del programma e la lettura del relativo Report, mostrato in Figura 4.10.

della porta JTAG interna, aggiunto automaticamente dal Quartus nel caso di utilizzo del tool menzionato. Tale segnale, così come tutti i dati relativi, possono essere ignorati, in quanto non utili allo scopo del lavoro.

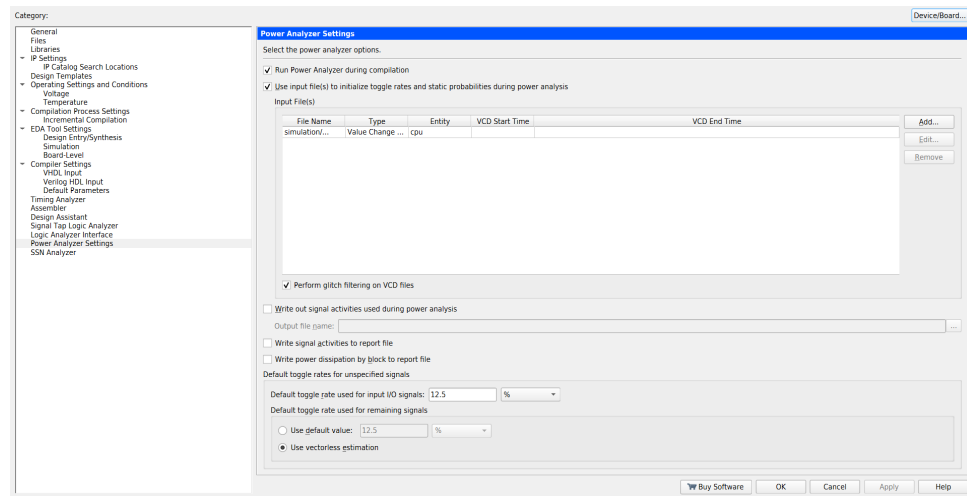


Figura 4.9: Impostazioni del *Power Analyzer* per la CPU.

Power Analyzer Status	Successful - Mon May 25 17:43:48 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	cpu
Top-level Entity Name	CPU
Family	Cyclone V
Device	5CSEMA5F31C6
Power Models	Final
Total Thermal Power Dissipation	468.70 mW
Core Dynamic Thermal Power Dissipation	39.78 mW
Core Static Thermal Power Dissipation	411.53 mW
I/O Thermal Power Dissipation	17.39 mW

Figura 4.10: Report dell'analisi di potenza della CPU.

4.1.5 Programmazione della FPGA

L'ultimo step rimanente è quello di caricare il progetto sulla board attraverso, come abbiamo già visto, il *Programmer*. I display a 7 segmenti saranno inizialmente spenti, come mostrato in Figura 4.11: non appena lo switch associato al segnale **switchon** sarà portato a valore alto, verranno accesi, mostrando il contenuto in memoria.

Ad esempio, in Figura 4.12 è mostrato il valore scritto durante il primo programma, che è il primo ad essere eseguito.

Non appena verrà premuto il bottone associato al segnale **ctrl1**, la CPU entrerà in stato di reset e il valore risulterà quindi nullo, come mostrato in Figura 4.13.

Appena questo verrà rilasciato, si passerà al programma successivo: nelle quattro figure successive sono mostrati i rispettivi risultati dei programmi dal secondo al quinto.

Infine, potremo utilizzare il tool *Signal Tap* per osservare il comportamento dei segnali reali durante l'esecuzione su FPGA. Immaginiamo, ad esempio, di voler verificare che il nostro processore stia effettivamente eseguendo il programma, e che quindi i display a 7 segmenti non stiano mostrando un risultato casuale: inseriamo nella lista dei segnali da osservare gli input **switchon**, **ctrl1**, il segnale interno **sel** ed il valore del PC uscente dal suo registro. Come condizione di trigger, impostiamo semplicemente il valore alto di **switchon**, ovvero l'abilitazione dei

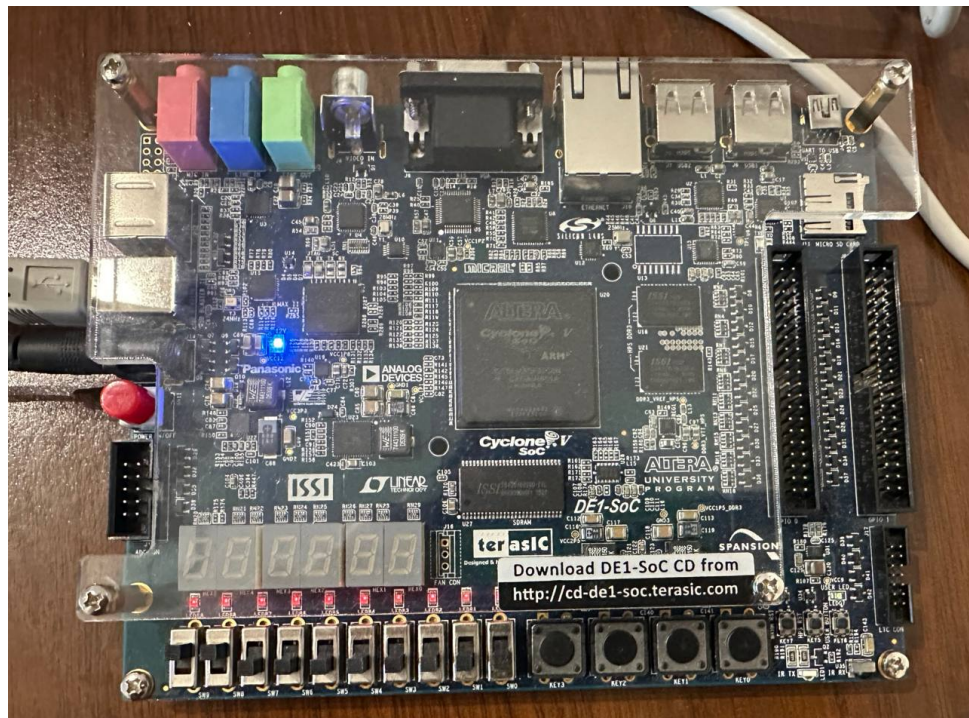


Figura 4.11: FPGA con display a 7 segmenti inizialmente spenti.

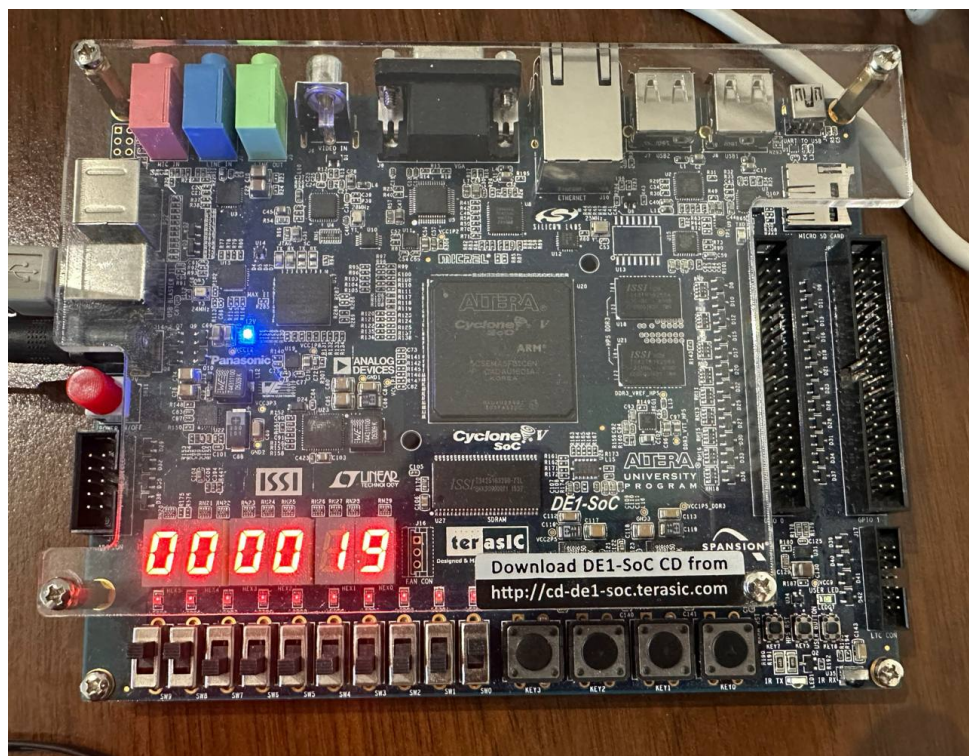


Figura 4.12: Valore scritto durante l'esecuzione del primo programma, visibile sui display a 7 segmenti.

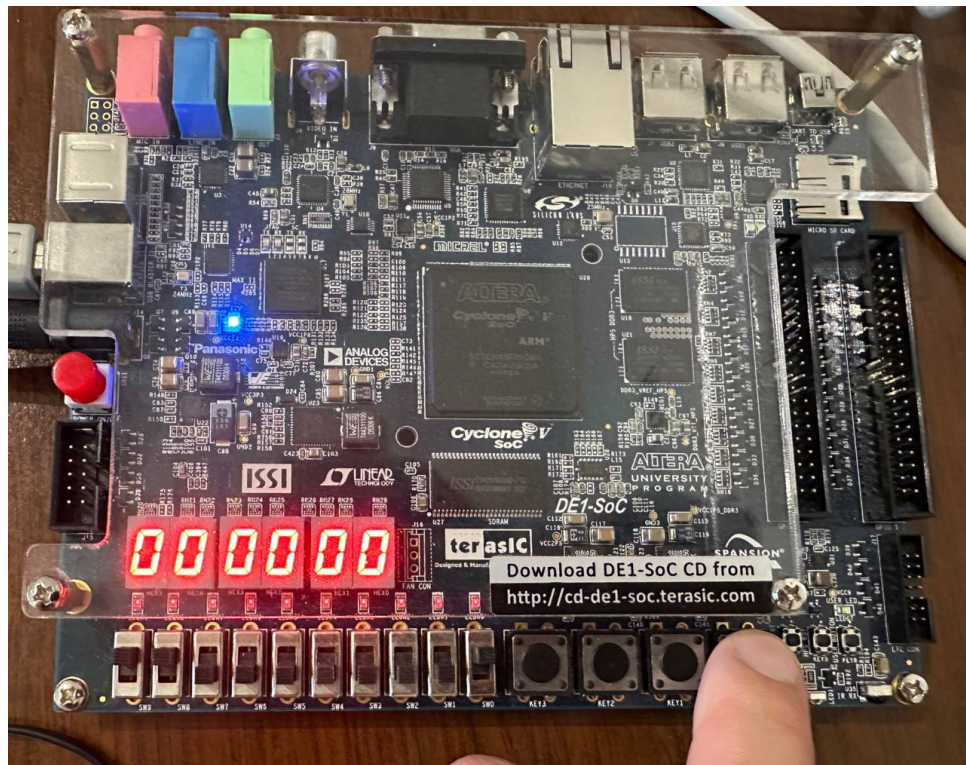


Figura 4.13: Stato di reset della CPU dopo la pressione del bottone `ctrl`, con valore nullo sul display.

display a 7 segmenti. Effettuando l'analisi nello stato mostrato in Figura 4.12 e facendo accadere la condizione necessaria, otterremo il risultato mostrato in Figura 4.18. Da questo risultato, si possono fare diverse affermazioni positive:

- Il segnale `ctrl` è alto, sintomo del fatto che il bottone non è premuto e la CPU sta eseguendo un determinato programma.
- Il segnale `sel` è nullo, valore a cui corrisponde, come ci si aspettava, il primo programma.
- Il PC cambia in modo iterativo i suoi valori, passando da `0x50` a `0x58`. Se si osserva il codice relativo mostrato in Figura 4.1, si può notare come l'ultima istruzione sia proprio all'indirizzo `0x50`: come previsto dal meccanismo di *branch prediction* utilizzato per la gestione dei *control hazard*, la CPU eseguirà le due istruzioni successive (in questo caso due `nop`⁵, in quanto il contenuto di tutte le memorie esclusa la IM è inizializzato a zero), per poi prendere il branch previsto dal loop infinito e ripetere il ciclo.

Queste osservazioni dimostrano, oltre alla testimonianza "*visiva*" fornita dai display, il funzionamento corretto dell'intero sistema.

⁵Una *nop* è un'istruzione che lascia completamente invariato l'intero stato del processore e dei suoi componenti. Nell'istruzione set in questione, essa sarà costituita da 32 zeri come codice macchina corrispondente.

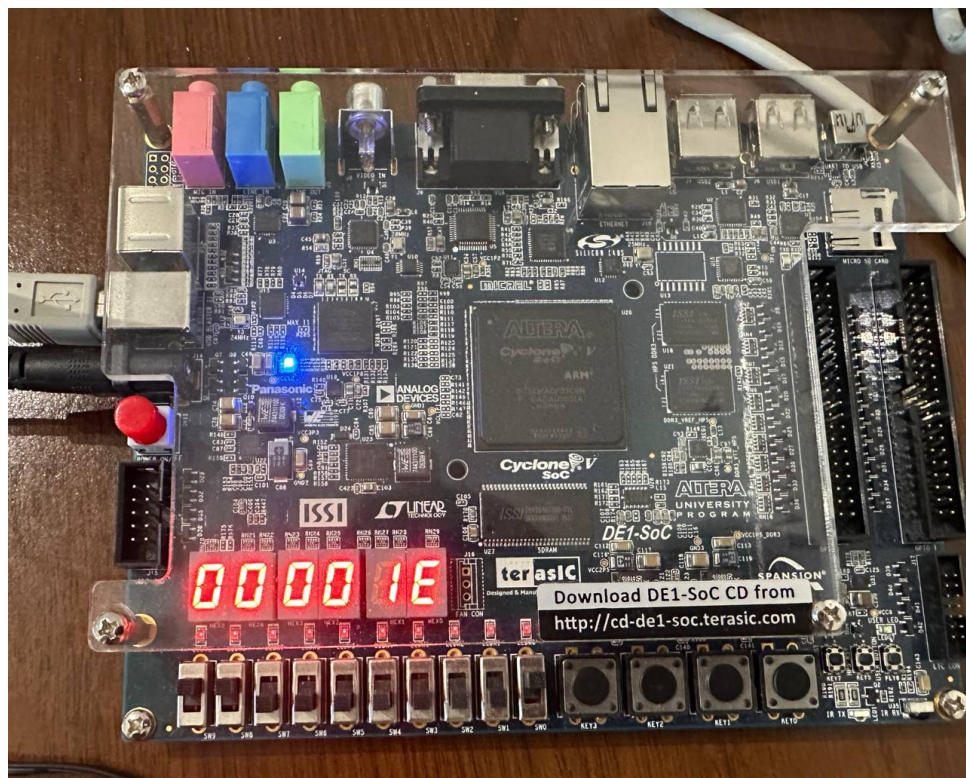


Figura 4.14: Output del secondo programma visualizzato sui display a 7 segmenti.

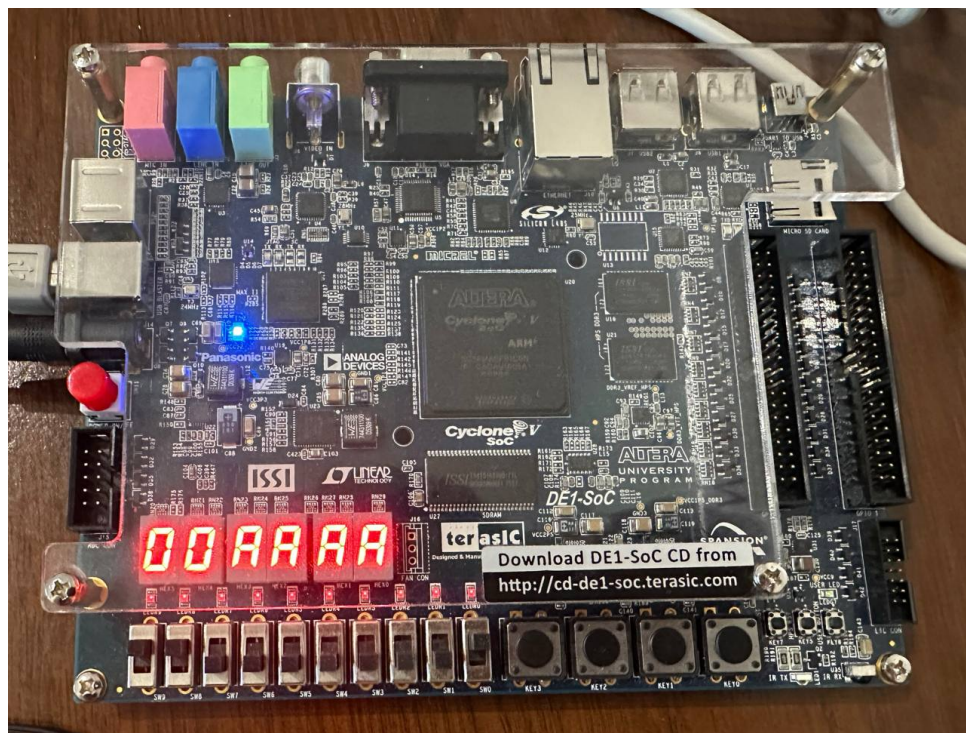


Figura 4.15: Output del terzo programma mostrato sui display a 7 segmenti.

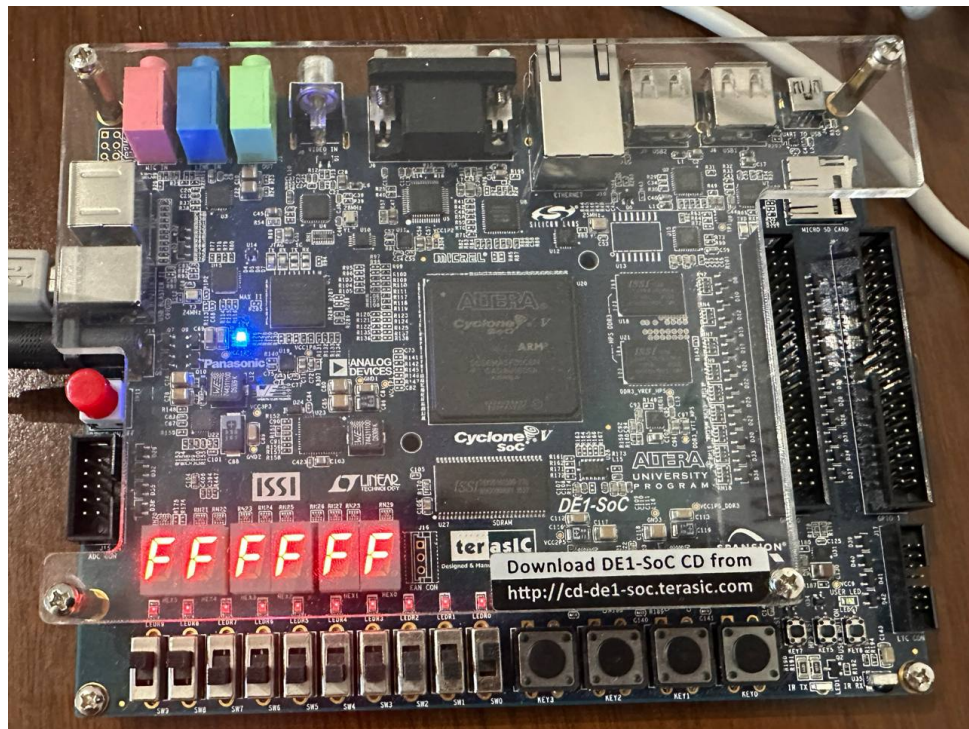


Figura 4.16: Output del quarto programma visualizzato sui display a 7 segmenti.

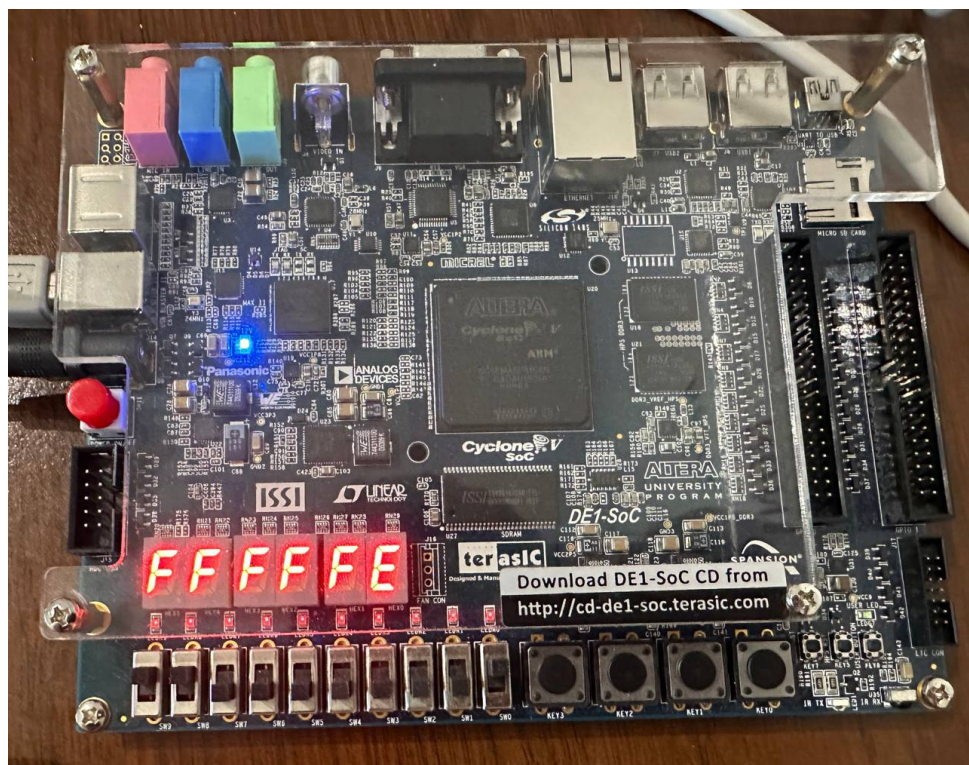


Figura 4.17: Output del quinto programma visualizzato sui display a 7 segmenti.

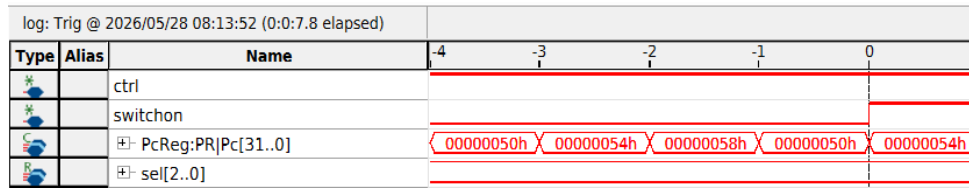


Figura 4.18: Analisi di alcuni dei segnali reali della CPU attraverso *Signal Tap*.

4.2 Utilizzo del soft-core *Nios V*

4.2.1 Introduzione

Fin'ora, è stato illustrato un progetto su una CPU realizzata autonomamente: tuttavia, Altera mette a disposizione una famiglia di *soft-core* già pronti chiamati *Nios V*, utilizzabili direttamente in ambiente Quartus. Questi processori si dividono in tre categorie [11]:

1. La versione *NIOS V/c (compact)* è la più semplice, e supporta il solo set di istruzioni RV32I;
2. La versione *NIOS V/m (microcontroller)* è la soluzione intermedia, ed oltre al set precedente, include anche istruzioni speciali di controllo (*Zicsr*);
3. La versione *NIOS V/g (general-purpose)* è la scelta più completa ed include, oltre alle istruzioni dei casi precedenti, anche quelle relative alle moltiplicazioni e divisioni (*M*).

Per quanto riguarda i registri messi a disposizione, oltre ai 32 già visti precedentemente, saranno presenti (per le ultime due soluzioni) anche dei registri di controllo, utilizzati dalle istruzioni dell'omonima categoria, non facenti parte del normale spazio degli indirizzi utilizzato dalla memoria ed elencati brevemente in Tabella 4.1.

In modo da contestualizzare il contenuto di questo elenco, è bene chiarire il significato di alcuni termini tecnici utilizzati. Una *trap* è un evento che interrompe il normale flusso di esecuzione del processore e trasferisce il controllo a una routine di gestione. Comprende sia le *eccezioni* (eventi sincroni causati dall'istruzione in esecuzione) sia gli *interrupt* (eventi asincroni esterni al flusso del programma).

Per quanto concerne le istruzioni eseguibili, oltre a quelle già studiate del set RV32I, saranno disponibili (ovviamente considerando le versioni che le supportano):

- Le istruzioni *M*, mostrate in Tabella 4.2.
- Le istruzioni di controllo *Zicsr*, elencate in Tabella 4.3. Ognuna di queste istruzioni fa riferimento ad un indirizzo di controllo utilizzando il suo indirizzo, indicato simbolicamente in questo elenco con *csr*.
- Istruzioni aggiuntive di natura generica mostrate in Tabella 4.4.

Registro	Indirizzo	Descrizione
mstatus	0x300	Riporta e controlla lo stato operativo del processore
misa	0x301	Riporta l'ISA della macchina
mie	0x304	Registro di abilitazione degli interrupt della macchina
mtvec	0x305	Registro che contiene l'indirizzo di salto al momento di una trap
mepc	0x341	Registro che contiene l'indirizzo dell'istruzione corrente al momento della trap
mcause	0x342	Registro che fornisce informazioni riguardo la causa della trap
mtval	0x343	Registro utilizzato per conservare informazioni addizionali riguardo una trap
mip	0x344	Registro degli interrupt pendenti (ovvero richiesti ma non ancora gestiti) della macchina

Tabella 4.1: Elenco dei registri di controllo e relativi indirizzi CSR.

Istruzione	Descrizione
mul rd, rs1, rs2	Moltiplicazione (parte bassa del risultato)
mulh rd, rs1, rs2	Moltiplicazione (parte alta del risultato, con segno)
mulhsu rd, rs1, rs2	Moltiplicazione (parte alta, operandi con segno e senza segno)
mulhu rd, rs1, rs2	Moltiplicazione (parte alta del risultato, senza segno)
div rd, rs1, rs2	Divisione tra interi con segno
divu rd, rs1, rs2	Divisione tra interi senza segno
rem rd, rs1, rs2	Resto della divisione con segno
remu rd, rs1, rs2	Resto della divisione senza segno

Tabella 4.2: Istruzioni della *M-extension* [3].

4.2.2 Inclusione del soft-processor all'interno di un progetto Quartus

In quest'ultima sezione del lavoro, si provvede a creare un nuovo progetto con le stesse funzionalità del precedente (ovvero che sia in grado di restituire in uscita un valore interno al processore e mostrarlo sul display a 7 segmenti) utilizzando, al posto della CPU originale, il core Nios V. Per prima cosa, si provvede a creare un nuovo progetto, procedendo successivamente con l'apertura del tool chiamato *Platform Designer*. La schermata di tale strumento all'apertura è mostrata in Figura 4.19.

Si inizia aggiungendo il processore, che sarà disponibile, insieme a tutte le altre componenti, nell'*IP Catalog*: come esempio, sceglieremo l'opzione intermedia, cercando e selezionando quindi la voce *Nios V/m Microcontroller IP*. Dopo aver

Istruzione	Descrizione
<code>csrrw rd, csr, rs1</code>	Lettura e scrittura di un registro CSR
<code>csrrwi rd, csr, uimm</code>	Lettura e scrittura di un registro CSR con immediato
<code>csrrs rd, csr, rs1</code>	Lettura e set dei bit del registro CSR
<code>csrrsi rd, csr, uimm</code>	Lettura e set dei bit del registro CSR con immediato
<code>csrrc rd, csr, rs1</code>	Lettura e cancellazione (clear) dei bit del registro CSR
<code>csrrci rd, csr, uimm</code>	Lettura e cancellazione (clear) dei bit del registro CSR con immediato

Tabella 4.3: Istruzioni di controllo del set *Zicsr* [3].

Istruzione	Descrizione
<code>nop</code>	Nessuna operazione; non ha effetti sul processore e incrementa semplicemente il program counter. È una pseudoinstruzione implementata come <code>addi x0, x0, 0</code> .
<code>ecall</code>	Genera una eccezione di ambiente (system call) con codice eccezione pari a 3.
<code>mret</code>	Ritorna da una routine di gestione della trap e ripristina l'esecuzione del programma interrotto.
<code>wfi</code>	Sospende l'esecuzione fino al verificarsi di un interrupt; in alcuni casi può comportarsi come una <code>nop</code> .

Tabella 4.4: Istruzioni miscellanee in Nios V [3].

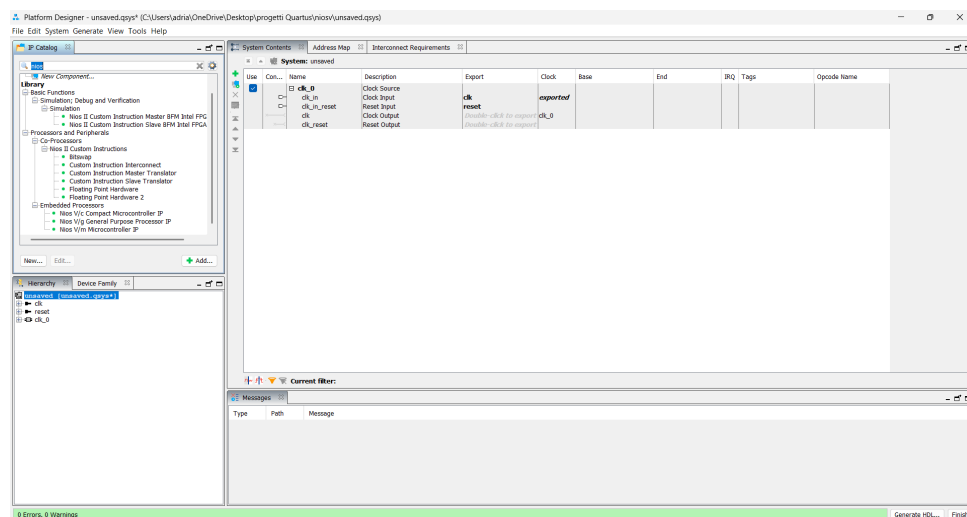


Figura 4.19: Finestra principale del *Platform Designer*.

lasciato inalterate le impostazioni di default, si aggiungo successivamente, con lo stesso procedimento:

- Una *On-chip memory*, ovvero un banco di memoria da fornire al processore. Si seleziona, come dimensione, 65536 byte, oltre a spuntare l'opzione (il cui

significato sarà reso chiaro successivamente) *Enable non-default initialization files*;

- Un *Parallel Input-Output (PIO)*, selezionando come tipologia *output* e dimensione 32 bit.

Si procede successivamente connettendo le varie componenti: si farà condividere alle tre lo stesso clock e lo stesso segnale di reset, oltre a provvedere alla connessione corretta dell'*Instruction Manager* e del *Data Manager* del processore. Si procede successivamente a cambiare, per il processore, il *Reset agent*. Infine, si assegnano in automatico gli indirizzi di memoria con *System > Assign Base Address*, in modo da evitare possibili *overlap*. Il risultato finale è mostrato in Figura 4.20.

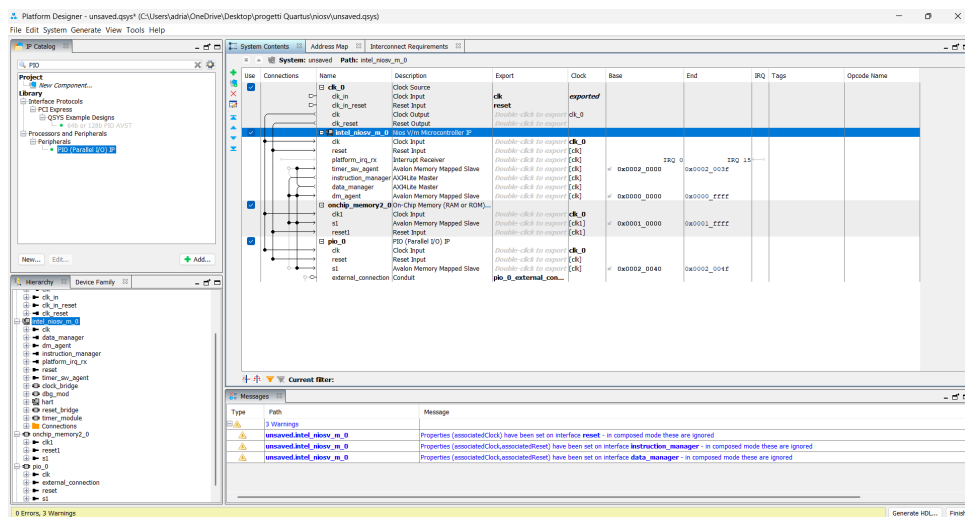


Figura 4.20: Finestra principale del *Platform Designer* dopo aver effettuato i procedimenti necessari.

Si conclude generando i moduli del sistema con *Generate HDL* e scegliendo, come HDL di riferimento, VHDL. Usciti dal tool, come segnalato dallo stesso Quartus, sarà necessario includere nel progetto i file **.qip* e **.sip* in modo che, durante la compilazione, l'IDE sia in grado di ricostruire l'intero sistema correttamente.

4.2.3 Generazione di un programma eseguibile dal soft-processor

Se nel processore personale era possibile conoscere, nel minimo dettaglio, come veniva elaborata ogni singola istruzione, il Nios V sarà invece assimilabile ad una *black box*, utilizzabile nei progetti ma non modificabile manualmente (a meno delle funzionalità del *Platform Designer*) in quanto IP proprietario. Di conseguenza, non sarà possibile inizializzare la memoria in modo affidabile con programmi compilati normalmente, ma sarà invece necessario seguire un procedimento preciso che permetterà, a partire da un generico programma scritto in C, di ottenere il corrispettivo file **.hex* con cui si andrà ad inizializzare la *On-chip memory*. Si inizia creando, all'interno della cartella di lavoro, una sottocartella dedicata, che verrà

chiamata **programmi**: in quest'ultima, saranno create e generate tutte le directory necessarie per completare il suddetto compito. Per prima cosa, sarà necessario generare il *Board Support Package (BSP)* in modo da permettere, per il codice che andremo a scrivere successivamente, una compilazione coerente con l'architettura: si procede quindi avviando la shell del Nios V (mostrata in Figura 4.21), entrando (attraverso il comando `cd`) nella cartella di lavoro e digitando il comando⁶:

```
niosv-bsp -c -p=niosv.qpf -s=niosv.qsys -t=hal programmi /bsp/settings.bsp
```

Alternativamente, si può utilizzare direttamente il file `*.sopcinfo`, generato per questo scopo, con i comandi:

```
niosv-bsp -c -t=hal -s=niosv.sopcinfo programmi/bsp/settings.bsp
niosv-bsp -g programmi/bsp/settings.bsp
```

In generale, con queste direttive, sarà possibile creare ed inizializzare, all'interno di **programmi**, la cartella **bsp**.

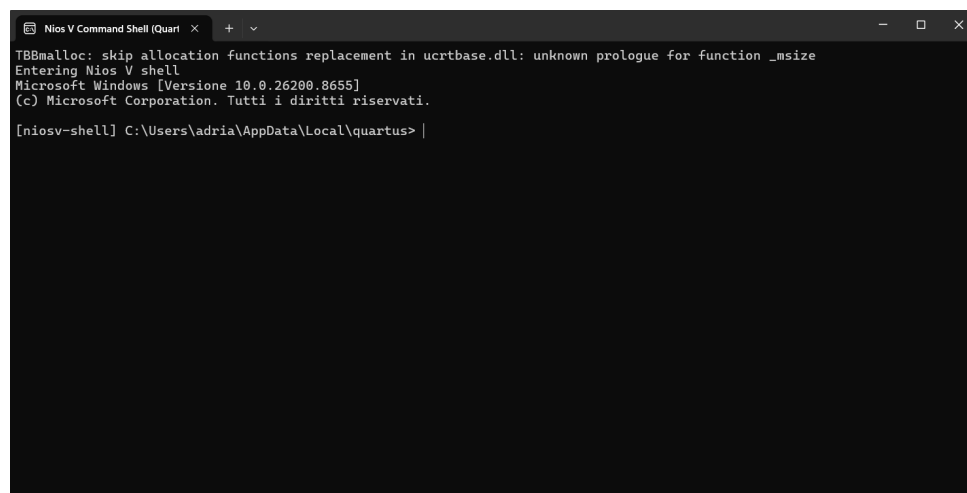


Figura 4.21: Finestra della *Nios V Command Shell*.

Concluso ciò, si procede con la creazione, sempre dentro **programmi**, della cartella **app**, in cui andremo a scrivere il nostro programma `onchip_mem.c`:

```
// Librerie generate dal BSP
// La prima contiene le macro con gli indirizzi base delle periferiche
// La seconda definisce le macro di accesso ai registri come IOWR
#include "system.h"
#include "io.h"
// Libreria standard del C
#include <stdint.h>

int main() {
```

⁶I seguenti comandi sono stati tratti, in larga parte, da [12]

```

// Variabili di esempio
uint32_t a = 12;
uint32_t b = 8;
uint32_t c = 3;

// Operazione: (12 + 8)*3 = 60 = 0x3C
uint32_t result = (a + b) * c;

// Scrittura sul PIO a 32 bit
IOWR(PIO_0_BASE, 0, result);

// loop infinito
while (1) {

}

return 0;
}

```

Come si può osservare, tale codice d'esempio si limita ad eseguire una semplice operazione aritmetica⁷ e a salvarne il risultato nell'uscita PIO. Per ottenere una compilazione corretta, sarà necessario fornire delle indicazioni al compilatore attraverso un *"file guida"*: per tale motivo, si procede generando il file `CMakeLists.txt` con il seguente comando:

```

nios-app --bsp-dir=programmi/bsp --app-dir=programmi/app
--srcs=programmi/app/onchip_mem.c --elf-name=onchip_mem.elf

```

Con questo comando, si può chiudere definitivamente il terminale dedicato ed aprire l'IDE, messo sempre a disposizione da Altera, chiamato *Ashling Riscfree IDE*. Subito dopo l'apertura, verrà mostrata una finestra come quella di Figura 4.22, in cui si chiederà all'utente di scegliere una cartella di lavoro: ovviamente, procederemo selezionando la directory dedicata `programmi`.

Dopodichè sarà possibile accedere all'IDE completo, mostrato in Figura 4.23.

Si continua selezionando da *File*, l'opzione *Import Nios V CMake Project*, scegliendo come cartella la *subdirectory app*. Infine, dal menù in cui saranno presenti i file correnti, si clicca col tasto destro sulla cartella principale e si seleziona *Build Project*, in modo da compilare il programma: concluso il processo, sarà possibile trovare il file `*.hex` dentro la cartella appena generata `build`. Si concluderà questa sezione tornando nel *Platform Designer* e scegliendo per la *On-chip memory*, come *User initialization file*, quello appena generato, in modo da inizializzare la

⁷Si osserva come, nonostante il Nios V/m non metta a disposizione la *M-extension*, sia presente un'operazione di moltiplicazione. Al momento della compilazione, l'assembler provvederà a ricostruire tale sviluppo matematico attraverso più operazioni del set RV32IZicsr concatenate, al posto di utilizzarne una singola.

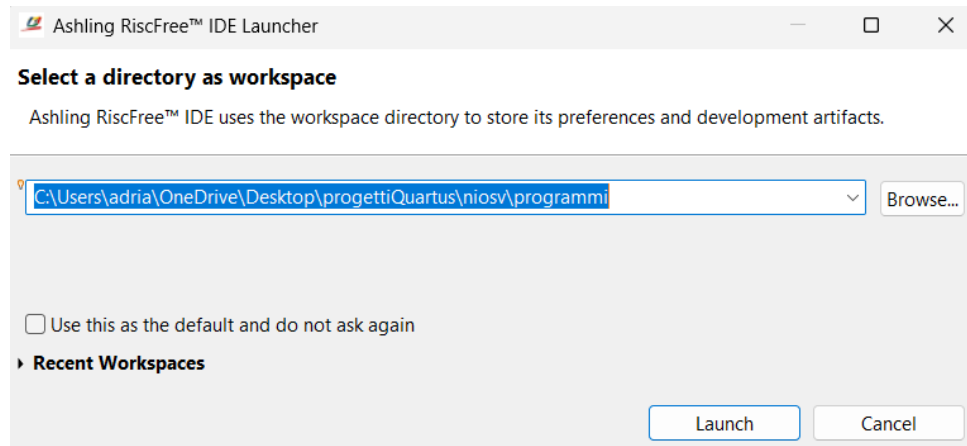


Figura 4.22: Finestra iniziale del Riscfree IDE relativa alla selezione del *workspace*.

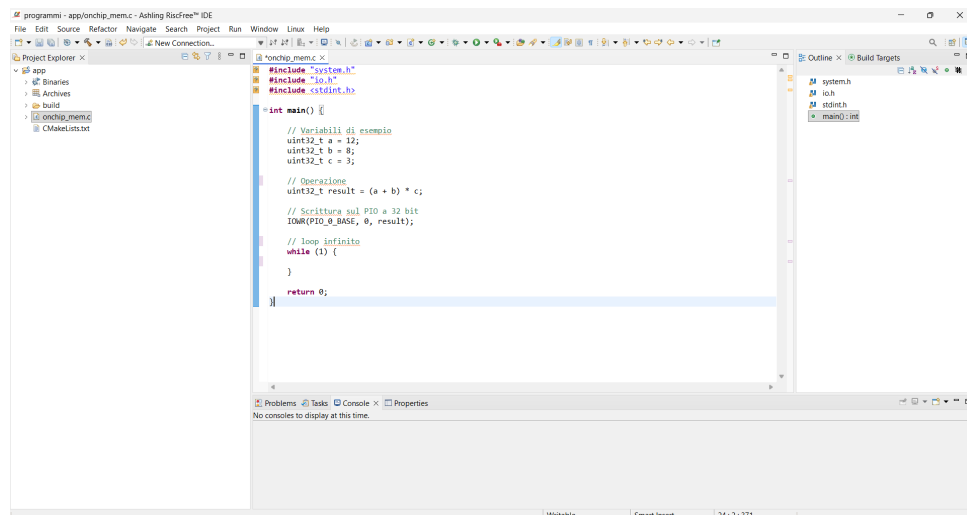


Figura 4.23: Schermata principale del *Riscfree IDE*.

memoria con il programma scritto. Le impostazioni selezionate sono mostrate in Figura 4.24.

4.2.4 Integrazione del soft-core nel progetto

Per realizzare un sistema quanto più simile all'originale, sarà necessario implementare un progetto complessivo in cui il Nios V rappresenti un modulo connesso agli ingressi e ad eventuali segnali intermedi, con cui si andranno a pilotare i display. In particolare, si realizzerà un file VHDL (che sarà, ovviamente, la *Top entity* del progetto) dalle seguenti caratteristiche:

1. Il codice avrà come ingresso un clock (connesso ovviamente al segnale di

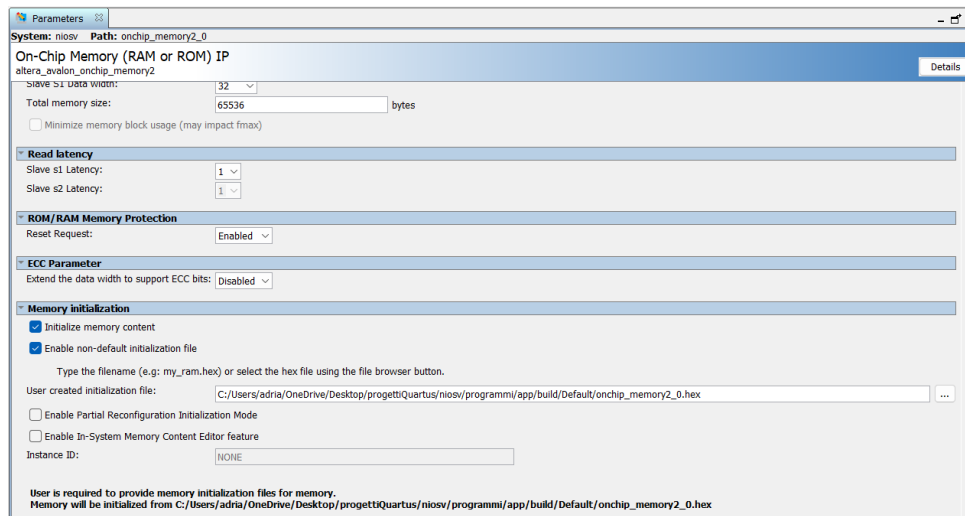


Figura 4.24: Impostazioni relative all’inizializzazione della memoria del Nios V.

temporizzazione di 50 MHz fornito dalla scheda) ed un segnale di reset⁸ (connesso al primo *slide switch*); mentre presenterà come uscite 6 segnali a 7 bit, con cui si andranno a pilotare i corrispettivi display.

2. Verrà istanziato il processore e collegato al clock ed al reset globali, mentre l’uscita sarà salvata su un segnale a 32 bit chiamato PI0.
3. Attraverso questo segnale verranno generati, con la stessa logica del progetto precedente, i 6 segnali di uscita. Nel caso in cui il segnale di reset sia basso, per evitare di mostrare il *garbage value* eventualmente presente in memoria, si spengono tutti i LED.

Si procede successivamente, dopo aver utilizzando correttamente l’*Assignment Editor*, con la compilazione ed il caricamento del programma nella FPGA. I risultati sono mostrati in Figura 4.25 ed in Figura 4.26: come si può facilmente osservare, i LED sono inizialmente spenti, mentre non appena si abilita il segnale di reset con l’interruttore dedicato, si avrà in uscita il valore atteso 0x3C.

4.2.5 Analisi temporale e di potenza

Come in tutti i progetti realizzati in Quartus, sarà possibile effettuare analisi temporali ed energetiche. Per quanto riguarda la prima, dopo la compilazione l’IDE non riconoscerà, inizialmente, il clock del sistema come segnale di temporizzazione: per tale motivo, sarà necessario creare un file dedicato *.sdc. Per mostrare una soluzione diversa da quelle fin’ora basate sull’utilizzo della GUI del *Timing Analyzer*, si procede creando suddetto file in modo verboso, scrivendoci all’interno il seguente testo:

⁸Per la logica utilizzata dal Nios V, tale segnale porta il sistema ad uno stato di reset quando è basso, mentre lo abilita quando è alto.

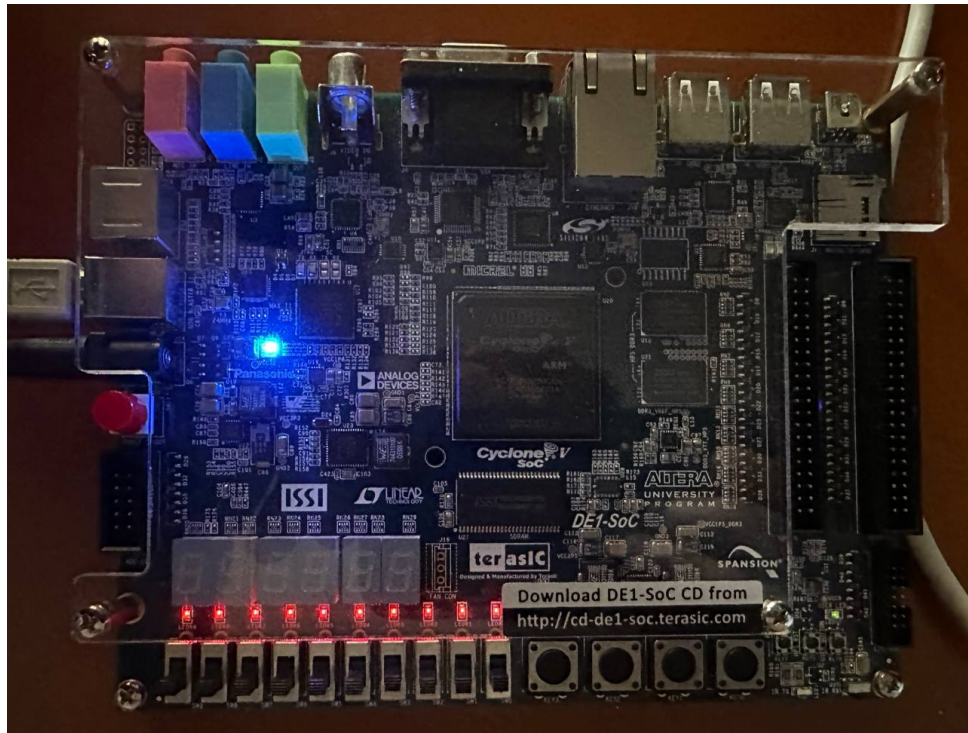


Figura 4.25: Implementazione fisica del progetto con Nios V con segnale di reset basso.

```
create_clock -name CLOCK_50 -period 20.000 [get_ports {CLOCK_50}]
```

Questo file *.sdc, così come gli altri realizzati con la GUI, indicherà al compilatore di effettuare l'analisi temporale considerando, come clock, il segnale `CLOCK_50` (ovvero il corrispettivo nel file VHDL) dal periodo pari a 20 ns (e frequenza conseguentemente di 50 MHz). Dopo averlo incluso nel progetto con lo stesso procedimento già visto, si può procedere con la compilazione, in modo da ottenere il report dedicato. I risultati ottenuti sono i seguenti:

- La frequenza massima supportata dal segnale è di 86.4 MHz, leggermente maggiore rispetto al soft-core personale;
- Il *Setup Slack* è di 8.426 ns;
- L'*Hold Slack* è di 0.114 ns;
- Il *Recovery Slack* è di 14.146 ns;
- Il *Removal Slack* è di 0.858 ns.

La positività di tutti i risultati indica il rispetto di tutti i vincoli temporali presenti. Per quanto riguarda l'analisi di potenza, sarà possibile seguire il procedimento già visto, ottenendo il risultato in Figura 4.27.

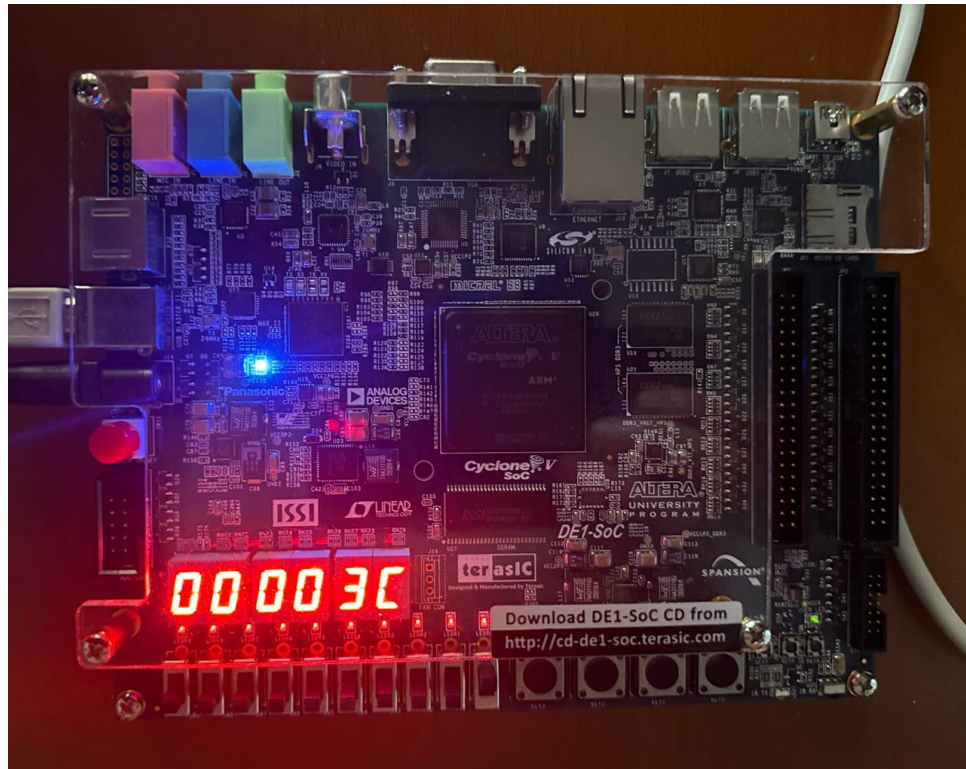


Figura 4.26: Implementazione fisica del progetto con Nios V con segnale di reset alto.

Power Analyzer Summary	
<<Filter>>	
Power Analyzer Status	Successful - Thu Jul 2 11:21:25 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	niosv
Top-level Entity Name	top_ent
Family	Cyclone V
Device	5CSEMA5F31C6
Power Models	Final
Total Thermal Power Dissipation	455.48 mW
Core Dynamic Thermal Power Dissipation	26.09 mW
Core Static Thermal Power Dissipation	412.09 mW
I/O Thermal Power Dissipation	17.30 mW

Figura 4.27: Analisi di potenza del progetto con Nios V.

4.2.6 Verifica del funzionamento tramite *Signal Tap*

Come in tutti gli altri progetti, sarà possibile creare ed integrare un file *.stp in modo da effettuare il debug della scheda direttamente sull'hardware. In particolare, come segnali associati (mostrati in Figura 4.28) selezioniamo il segnale di reset RESET_N e l'uscita del Nios V. Ovviamente, come clock del sistema, sarà scelto CLOCK_50

Come condizione di trigger impostiamo, come mostrato in Figura 4.29, che il 4° LSB del PIO sia alto: in questo modo, sarà possibile osservare se effettivamente, in uscita dal Nios V, ci sia il valore corretto, oltre a poter osservare la commutazione dal *garbage value* al valore atteso.

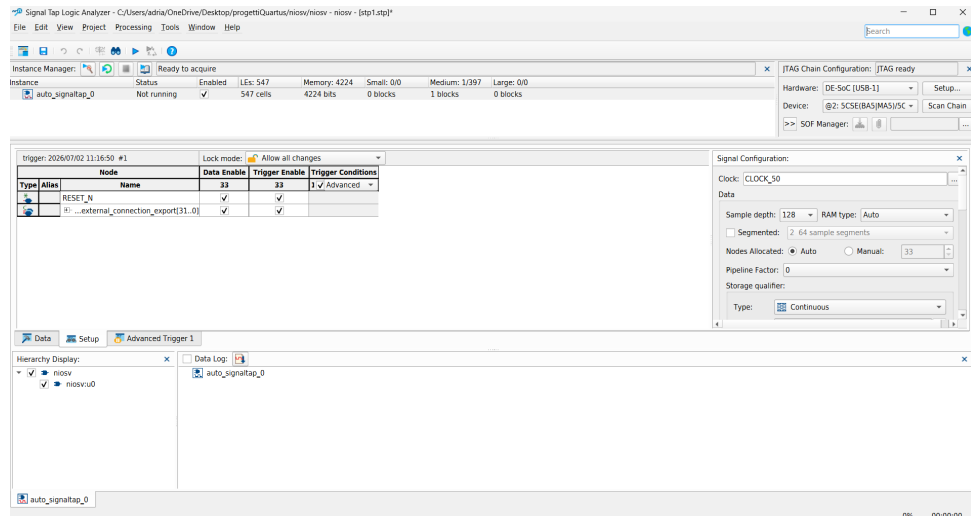


Figura 4.28: Elenco dei segnali d'interesse per *Signal Tap* nel progetto con Nios V.



Figura 4.29: Condizione di trigger per *Signal Tap* nel progetto con Nios V.

Infine, si procede salvando il file creato, includendolo nel progetto e ricompilando il tutto. Concluso ciò, si riapre *Signal Tap* e si fa partire l'analisi: dopo aver abilitato il segnale **RESET_N**, non appena il processore scriverà sul PIO, compariranno le waveform attese, mostrate in Figura 4.30.

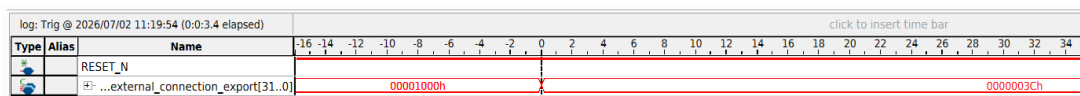


Figura 4.30: Waveform rilevate con *Signal Tap* nel progetto con Nios V.

Come si può osservare, con **RESET_N** alto, si avrà la commutazione tra il valore "spazzatura" 0x1000 al valore atteso 0x3C.

CONCLUSIONI

In conclusione, è stato dimostrato con successo il corretto funzionamento del *soft-core* realizzato, le cui prestazioni sono state successivamente confrontate con quelle della controparte proprietaria *Nios V*. In particolare, è stata verificata la corretta esecuzione di tutte le istruzioni dell'*Instruction Set RV32I*, sia in simulazione che in esecuzione su FPGA, ed è stato inoltre possibile sfruttare il segnale di temporizzazione da 50 MHz fornito dalla scheda senza violare i vincoli temporali. Il sistema basato sulla CPU proprietaria consente tuttavia di raggiungere frequenze di lavoro superiori, oltre a supportare un set di istruzioni più ampio; ciononostante, la realizzazione di un processore personale garantisce un controllo completo sul sistema e su ogni dettaglio del processo di esecuzione delle istruzioni. Il presente progetto potrebbe essere ulteriormente migliorato includendo nuovi subset di istruzioni appartenenti alla famiglia RISC-V, come la *M-extension* (RVM), le istruzioni a virgola mobile RVF/D e le istruzioni di controllo *Zicsr*. Inoltre, si potrebbe implementare un sistema gerarchico di memoria in cui il processore comunichi direttamente solo con due *cache* (una per la *Data Memory* e una per la *Instruction Memory*), allocando la memoria principale nella SDRAM da 64 MB messa a disposizione dal *System on Chip* utilizzato.

BIBLIOGRAFIA E SITOGRAFIA

- [1] Numeric_std package. https://www.hdlworks.com/hdl_corner/vhdl_ref/VHDLContents/NumericStd.htm.
- [2] David Harris Sarah L. Harris. *Digital Design and Computer Architecture - RISC-V Edition*. Morgan Kaufmann - Elsevier, 2022.
- [3] Fpgacademy. <https://fpgacademy.org>.
- [4] Timing analyzer: Introduction to timing analysis. <https://youtu.be/HMAqjjCuDEI?si=WYSpIr21m7iLNX-F>.
- [5] Intel fpga power and thermal calculator for intel fpga devices. https://youtu.be/jV17C-VWz1o?si=X0TCVE_e-9dqJ5vb.
- [6] De1-soc user manual (rev.f/rev.g board). www.terasic.com.tw.
- [7] Agostino Giorgio. *Elettronica digitale, teoria e pratica: dalle reti logiche all'intelligenza artificiale*. Gruppo Editoriale GEDI, 2023.
- [8] Signal tap logic analyzer: Introduction and getting started. <https://youtu.be/R8vUERKTDzg?si=5LL-7bDqjliG0-HD>.
- [9] Carlo Brandolese. Introduzione al linguaggio vhdl. <http://brandolese.faculty.polimi.it>.
- [10] Peter J. Ashenden. *The VHDL Cookbook*, 1990.
- [11] Nios v processor reference manual. <https://docs.altera.com/r/docs/683632/25.3/nios-v-processor-reference-manual/overview>.
- [12] An 985: Nios v processor tutorial. <https://docs.altera.com/r/docs/784468/current/an-985-nios-v-processor-tutorial/an-985-nios-v-processor-tutorial>.

RINGRAZIAMENTI

Per prima cosa, ringrazio il Prof. Agostino Giorgio per l'immensa disponibilità, pazienza e competenza dimostrate durante tutto il percorso di realizzazione del lavoro di tesi.

Ringrazio i miei genitori per il loro sostegno di ogni tipo; mio fratello Giulio per essere stato il miglior coinquilino che abbia avuto durante la mia esperienza universitaria ed i miei nonni, insieme a mia sorella Flavia, per tutto il loro supporto emotivo. Ringrazio anche mio cugino Pierpaolo per gli innumerevoli passaggi e mia cugina Roberta per avermi aiutato più volte durante la ricerca dell'alloggio e con alcuni contrattempi burocratici.

Ringrazio inoltre i miei colleghi di corso, grazie ai quali ho trovato, durante questi tre anni, un ambiente sereno e collaborativo: in particolare, ringrazio gli amici di *30 che in inglese si dice I.P.* e quelli di *Poor professor's*. Mi permetto di fare una menzione d'onore per Paolo, presente in entrambi e compagno di corso e fatiche per tutta l'esperienza universitaria.

Infine ringrazio Gianni, il *Custode delle Moto*.